

**POLYTECHNIQUE MONTRÉAL**

affiliée à l'Université de Montréal

**Génération de trajectoire sans collision par Flow Matching et fonctions  
barrières de contrôle pour la manipulation robotique.**

**FRÉDÉRICK LANTHIER**

Département de génie mécanique

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*  
Génie mécanique

Septembre 2026

**POLYTECHNIQUE MONTRÉAL**

affiliée à l'Université de Montréal

Ce mémoire intitulé :

**Génération de trajectoire sans collision par Flow Matching et fonctions  
barrières de contrôle pour la manipulation robotique.**

présenté par **Frédéric LANTHIER**

en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*  
a été dûment accepté par le jury d'examen constitué de :

**Bowen YI**, présidente

**Sofiane ACHICHE**, membre et directeur de recherche

**Jérôme LE NY**, membre et codirecteur de recherche

**Cédric LEBLOND-MÉNARD**, membre externe

## DÉDICACE

*À ma blonde que j'aime, mes parents,  
et mes amis des labs...*

## REMERCIEMENTS

Texte / Text.

*Je tiens à souligner l'utilisation de l'intelligence artificielle générative pour soutenir la révision et l'édition des sections rédigées de ce mémoire. Son usage s'est limité à une assistance linguistique et ne s'est pas étendu à la génération ni à l'analyse du contenu de recherche. L'ensemble des idées, des résultats et des conclusions présentés dans ce mémoire demeurent les miens.*

## RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

## ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

## TABLE DES MATIÈRES

|                                                                                          |      |
|------------------------------------------------------------------------------------------|------|
| DÉDICACE . . . . .                                                                       | iv   |
| REMERCIEMENTS . . . . .                                                                  | v    |
| RÉSUMÉ . . . . .                                                                         | vi   |
| ABSTRACT . . . . .                                                                       | vii  |
| LISTE DES TABLEAUX . . . . .                                                             | xii  |
| LISTE DES FIGURES . . . . .                                                              | xiii |
| LISTE DES SIGLES ET ABRÉVIATIONS . . . . .                                               | xvi  |
| LISTE DES ANNEXES . . . . .                                                              | xvii |
| CHAPITRE 1 INTRODUCTION . . . . .                                                        | 1    |
| CHAPITRE 2 REVUE DE LITTÉRATURE . . . . .                                                | 3    |
| 2.1 Génération de trajectoires robotisées . . . . .                                      | 3    |
| 2.1.1 Planification classique par échantillonnage et optimisation . . . . .              | 3    |
| 2.1.2 Apprentissage par démonstration, clonage de comportement et renforcement . . . . . | 4    |
| 2.1.3 Modèles génératifs, politiques de diffusion et Flow Matching . . . . .             | 5    |
| 2.2 Perception visuelle et mémoire spatiale . . . . .                                    | 7    |
| 2.2.1 Vision-langage, segmentation zéro-shot et suivi visuel . . . . .                   | 8    |
| 2.2.2 Reconstruction RGB-D et représentation par nuages de points . . . . .              | 8    |
| 2.2.3 Cartes persistantes face aux occlusions et pertes de détection . . . . .           | 9    |
| 2.3 Représentations géométriques pour la collision . . . . .                             | 10   |
| 2.3.1 Champs de distance de la scène et limites computationnelles . . . . .              | 10   |
| 2.3.2 Primitives conservatives et modèles géométriques simplifiés . . . . .              | 11   |
| 2.3.3 Champs de distance signée du robot . . . . .                                       | 12   |
| 2.4 Sécurité réactive et fonctions de contrôle barrières . . . . .                       | 13   |
| 2.4.1 Champs de potentiel et méthodes réactives classiques . . . . .                     | 13   |
| 2.4.2 Fonctions barrières de contrôle et invariance avant . . . . .                      | 13   |
| 2.4.3 Filtres de sécurité prédictifs et méthodes d'atteignabilité . . . . .              | 15   |
| 2.4.4 Limites des CBF sous perception bruitée et contrôle discret . . . . .              | 16   |

|                                                           |                                                                        |    |
|-----------------------------------------------------------|------------------------------------------------------------------------|----|
| 2.5                                                       | Couplage entre modèles génératifs et filtres de sécurité . . . . .     | 17 |
| 2.5.1                                                     | Contraintes intégrées à la génération . . . . .                        | 18 |
| 2.5.2                                                     | Filtres de sécurité appliqués à l'exécution . . . . .                  | 18 |
| 2.5.3                                                     | Découplage entre intention nominale et sécurité réactive . . . . .     | 19 |
| 2.6                                                       | Synthèse critique et positionnement . . . . .                          | 20 |
| 2.6.1                                                     | Lacunes identifiées dans la littérature . . . . .                      | 20 |
| 2.6.2                                                     | Positionnement de l'architecture proposée . . . . .                    | 21 |
| 2.6.3                                                     | Vers les objectifs de recherche . . . . .                              | 21 |
| CHAPITRE 3 MOTIVATION DE RECHERCHE ET OBJECTIFS . . . . . |                                                                        | 24 |
| 3.1                                                       | Positionnement de recherche et motivation . . . . .                    | 24 |
| 3.2                                                       | Objectifs de recherche . . . . .                                       | 25 |
| 3.2.1                                                     | Objectif principal . . . . .                                           | 25 |
| 3.2.2                                                     | Objectifs spécifiques . . . . .                                        | 25 |
| 3.3                                                       | Portée et exclusions de la recherche . . . . .                         | 27 |
| CHAPITRE 4 DESIGN DU SYSTÈME . . . . .                    |                                                                        | 28 |
| 4.1                                                       | Plateforme Expérimentale Et Architecture Globale . . . . .             | 28 |
| 4.1.1                                                     | Plateforme matérielle et logicielle . . . . .                          | 28 |
| 4.1.2                                                     | Architecture modulaire du système . . . . .                            | 29 |
| 4.2                                                       | Perception Sémantique et Reconstruction 3D . . . . .                   | 31 |
| 4.2.1                                                     | Extraction Sémantique . . . . .                                        | 32 |
| 4.2.2                                                     | Suivi temporel de la cible . . . . .                                   | 32 |
| 4.2.3                                                     | Projection RGB-D vers l'espace 3D . . . . .                            | 33 |
| 4.2.4                                                     | Séparation cible, obstacles et outil visible . . . . .                 | 33 |
| 4.2.5                                                     | Nettoyage spatial du nuage d'obstacles . . . . .                       | 33 |
| 4.3                                                       | Cartes Persistantes de la scène . . . . .                              | 34 |
| 4.3.1                                                     | Motivation : robustesse aux pertes de détection et occlusion . . . . . | 34 |
| 4.3.2                                                     | Génération des cartes persistantes . . . . .                           | 34 |
| 4.3.3                                                     | Mise à jour des cartes persistantes . . . . .                          | 35 |
| 4.4                                                       | Acquisition des données et représentation des démonstrations . . . . . | 35 |
| 4.4.1                                                     | Protocole de collecte des démonstrations . . . . .                     | 35 |
| 4.4.2                                                     | Prétraitement des données . . . . .                                    | 38 |
| 4.5                                                       | Génération de trajectoire par Flow Matching . . . . .                  | 42 |
| 4.5.1                                                     | Formulation mathématique du Flow Matching conditionnel . . . . .       | 42 |
| 4.5.2                                                     | Architecture du modèle . . . . .                                       | 43 |
| 4.5.3                                                     | Entraînement du modèle . . . . .                                       | 45 |



|            |                                                                        |    |
|------------|------------------------------------------------------------------------|----|
| 4.6        | Conversion et exécution de la trajectoire nominale . . . . .           | 45 |
| 4.6.1      | Conversion outil–TCP . . . . .                                         | 46 |
| 4.6.2      | Cinématique inverse pour obtenir la trajectoire articulaire nominale . | 46 |
| 4.6.3      | Réajustement temporel de la trajectoire articulaire . . . . .          | 48 |
| 4.7        | Filtre de sécurité réactif SDF-CBF . . . . .                           | 49 |
| 4.7.1      | Modèle de distance du robot appris hors ligne . . . . .                | 50 |
| 4.7.2      | Fonction barrière . . . . .                                            | 52 |
| 4.7.3      | Gradient analytique de la contrainte . . . . .                         | 55 |
| 4.7.4      | Contrainte de sécurité et vitesse nominale . . . . .                   | 57 |
| 4.7.5      | De la vitesse nominale à la commande sécurisée . . . . .               | 60 |
| 4.7.6      | Évitement des minima locaux du filtre . . . . .                        | 65 |
| 4.7.7      | Portée des garanties . . . . .                                         | 66 |
| CHAPITRE 5 | RÉSULTATS ET DISCUSSION . . . . .                                      | 71 |
| 5.1        | Protocole expérimental . . . . .                                       | 71 |
| 5.1.1      | Bancs d’essai . . . . .                                                | 71 |
| 5.1.2      | Scénarios . . . . .                                                    | 71 |
| 5.1.3      | Métriques . . . . .                                                    | 72 |
| 5.2        | Validation des composants . . . . .                                    | 73 |
| 5.2.1      | Précision du SDF du robot . . . . .                                    | 73 |
| 5.2.2      | Temps d’évaluation du SDF et du gradient . . . . .                     | 76 |
| 5.2.3      | Sélection des points critiques . . . . .                               | 76 |
| 5.2.4      | Validation du planificateur nominal . . . . .                          | 77 |
| 5.3        | Sécurité et évitement d’obstacles . . . . .                            | 78 |
| 5.3.1      | Résultats agrégés avec et sans filtre . . . . .                        | 80 |
| 5.3.2      | Étude détaillée : tâche d’alimentation . . . . .                       | 80 |
| 5.3.3      | Étude détaillée : tâche de saisie et dépôt . . . . .                   | 86 |
| 5.4        | Ablations des mécanismes . . . . .                                     | 86 |
| 5.4.1      | Carte persistante contre perception instantanée . . . . .              | 86 |
| 5.4.2      | Métrique de tâche contre métrique euclidienne . . . . .                | 86 |
| 5.4.3      | Dégagement postural : neutralité de sécurité et progression . . . . .  | 86 |
| 5.4.4      | Resserrement dynamique face à un obstacle mobile . . . . .             | 87 |
| 5.4.5      | Réparation de la commande filtrée . . . . .                            | 87 |
| 5.5        | Généralisation inter-embodiment . . . . .                              | 87 |
| 5.6        | Performance et temps de calcul . . . . .                               | 88 |
| 5.7        | Cas d’échec et limites . . . . .                                       | 88 |

|                                                                 |     |
|-----------------------------------------------------------------|-----|
| 5.8 Discussion . . . . .                                        | 89  |
| CHAPITRE 6 CONCLUSION . . . . .                                 | 90  |
| 6.1 Synthèse des travaux / Summary of Works . . . . .           | 90  |
| 6.2 Limitations de la solution proposée / Limitations . . . . . | 90  |
| 6.3 Améliorations futures / Future Research . . . . .           | 90  |
| RÉFÉRENCES . . . . .                                            | 91  |
| ANNEXES . . . . .                                               | 100 |

## LISTE DES TABLEAUX

|             |                                                                                                                                                                                                                                                                                                                                                                                  |     |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Tableau 2.1 | Tableau comparatif des frameworks de planification et de sécurité de l'état de l'art. . . . .                                                                                                                                                                                                                                                                                    | 22  |
| Tableau 3.1 | Critères de validation associés aux objectifs de recherche. . . . .                                                                                                                                                                                                                                                                                                              | 26  |
| Tableau 4.1 | Hierarchie des mécanismes du système selon leur rôle vis-à-vis des objectifs de recherche. . . . .                                                                                                                                                                                                                                                                               | 31  |
| Tableau 4.2 | Hyperparamètres d'entraînement du modèle de Flow Matching. . . .                                                                                                                                                                                                                                                                                                                 | 46  |
| Tableau 4.3 | Temps d'évaluation conjointe des barrières $h_l$ et de la Jacobienne de contrainte complète ( $L \times 7$ ) pour 100 points obstacles, en fonction du nombre $L$ de liens protégés inclus : gradient analytique en forme close contre différentiation automatique (moyenne $\pm$ écart-type en ms, 100 répétitions), avec l'erreur absolue maximale entre les deux Jacobiennes. | 57  |
| Tableau 4.4 | Paramètres du filtre de sécurité SDF-CBF. . . . .                                                                                                                                                                                                                                                                                                                                | 70  |
| Tableau 5.1 | Bancs d'essai utilisés pour la validation expérimentale. . . . .                                                                                                                                                                                                                                                                                                                 | 72  |
| Tableau 5.2 | Matrice des scénarios évalués par banc d'essai ( $N$ essais par cellule). .                                                                                                                                                                                                                                                                                                      | 72  |
| Tableau 5.3 | Comparaison avec et sans filtre SDF-CBF, agrégée sur $N$ essais par cellule : taux de succès, collisions, marge minimale $h_{\min}$ et déviation de trajectoire (métriques de la Section 5.1.3). . . . .                                                                                                                                                                         | 80  |
| Tableau 5.4 | Comparaison Panda et xArm7 sur les scénarios communs de la tâche de saisie et dépôt ( $N$ essais par cellule, métriques de la Section 5.1.3).                                                                                                                                                                                                                                    | 87  |
| Tableau 5.5 | Statistiques descriptives des temps de calcul par étape du pipeline (durées en millisecondes). . . . .                                                                                                                                                                                                                                                                           | 89  |
| Tableau C.1 | Hyperparamètres d'entraînement du modèle de distance du robot. . .                                                                                                                                                                                                                                                                                                               | 110 |

## LISTE DES FIGURES

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                     |    |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 2.1 | Trajectoire hors distribution en clonage de comportement . . . . .                                                                                                                                                                                                                                                                                                                                                                  | 5  |
| Figure 2.2 | Champ de vitesse du <i>Flow Matching</i> . . . . .                                                                                                                                                                                                                                                                                                                                                                                  | 6  |
| Figure 4.1 | Architecture globale du système asynchrone. . . . .                                                                                                                                                                                                                                                                                                                                                                                 | 30 |
| Figure 4.2 | Exemple de masque obtenu par le modèle de segmentation SAM3 pour le prompt <i>Pink Block</i> . . . . .                                                                                                                                                                                                                                                                                                                              | 32 |
| Figure 4.3 | Protocole d'échantillonnage des cibles, vu de dessus dans le repère de base du robot $\mathcal{F}_b$ . Les assiettes (26 cm de diamètre) se chevauchant, seules trois des neuf positions de la grille $3 \times 3$ sont illustrées (supérieur droit, centre, inférieur gauche), le centre de chacune étant marqué par une croix noire. Autour de chaque centre, les 5 points indiquent les positions de cible en quinconce. . . . . | 36 |
| Figure 4.4 | Trois phases de la politique experte simulée pour la tâche d'alimentation.                                                                                                                                                                                                                                                                                                                                                          | 38 |
| Figure 4.5 | Trois phases de la politique experte simulée pour la tâche de saisie et dépôt. . . . .                                                                                                                                                                                                                                                                                                                                              | 38 |
| Figure 4.6 | Représentation schématique des différents repères du système . . . .                                                                                                                                                                                                                                                                                                                                                                | 40 |
| Figure 5.1 | Visualisation qualitative du Signed Distance Field (SDF) de Bernstein pour trois configurations articulaires : gradients $\nabla d$ évalués le long des normales à la surface du robot. . . . .                                                                                                                                                                                                                                     | 74 |
| Figure 5.2 | Précision de la distance du SDF de Bernstein. (a) Profil de distance le long des normales à la surface du robot ; (b) distribution de l'erreur signée par rapport à la distance exacte calculée sur les maillages. . . .                                                                                                                                                                                                            | 75 |
| Figure 5.3 | Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale). . . . .                                                                                                                                                                                                                                            | 75 |
| Figure 5.4 | Temps de calcul de la distance et du gradient : modèle de Bernstein (Graphics Processing Unit (GPU)) contre vérité terrain sur maillages (Central Processing Unit (CPU)), en fonction du nombre de points de requête. . . . .                                                                                                                                                                                                       | 76 |
| Figure 5.5 | Sélection des $K$ points de la carte persistante les plus proches du robot (en rouge) qui contraignent le filtre Control Barrier Function (CBF) ; l'isosurface $d_{\text{robot}} = 1,5$ cm forme l'enveloppe de sécurité autour de l'outil (en bleu). . . . .                                                                                                                                                                       | 77 |

|             |                                                                                                                                                                                                                                                                                                                                                               |     |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Figure 5.6  | Fidélité du planificateur nominal aux démonstrations expertes sur le jeu de validation. (a) Distribution de la déviation de position (DTW) ; (b) distribution de l’erreur de rotation finale. . . . .                                                                                                                                                         | 78  |
| Figure 5.7  | Erreur de prédiction en fonction du point $k$ de l’horizon. (a) Erreur de position ; (b) erreur de rotation. Bandes : percentiles 10–90 et écart interquartile. . . . .                                                                                                                                                                                       | 79  |
| Figure 5.8  | Générations multiples du planificateur pour une même observation, superposées à la vérité terrain. (a) Plan $XY$ ; (b) plan $XZ$ . . . . .                                                                                                                                                                                                                    | 79  |
| Figure 5.9  | Intégration du flot pour une scène de la tâche d’alimentation : segment $X_t$ à $t = 0$ (bruit), $t = 0,5$ et $t = 1$ (généré), dans les plans $XZ$ (haut) et $YZ$ (bas). . . . .                                                                                                                                                                             | 80  |
| Figure 5.10 | Trajectoire de la pointe de l’outil avec et sans filtre SDF-CBF pour un essai représentatif, superposée au nuage d’obstacles de la carte persistante et à la cible. . . . .                                                                                                                                                                                   | 81  |
| Figure 5.11 | Évolution de la barrière $h_{\text{soft}}$ et de sa valeur compensée $h_{\text{corr}}$ pour une tâche en présence d’obstacles ; les bandes vertes indiquent les cycles où la projection corrige la vitesse, les bandes jaunes ceux où seule la réparation intervient. . . . .                                                                                 | 82  |
| Figure 5.12 | Contrainte de sécurité évaluée sur la vitesse nominale (avant projection) et sur la commande finale (après réparation). Les valeurs négatives de la courbe nominale correspondent aux violations qu’aurait commises la trajectoire générée sans filtre. . . . .                                                                                               | 83  |
| Figure 5.13 | Vitesse nominale, correction de projection et vitesse commandée pour chacune des sept articulations. . . . .                                                                                                                                                                                                                                                  | 84  |
| Figure 5.14 | Taux de variation de la barrière commandé ( $\nabla h^\top \dot{\mathbf{q}}_{\text{final}}$ ) et réellement exécuté ( $\nabla h^\top \dot{\mathbf{q}}_{\text{mes}}$ ) avec l’interface de commande retenue. . . . .                                                                                                                                           | 85  |
| Figure A.1  | Architecture du système d’acquisition des données d’entraînement. . .                                                                                                                                                                                                                                                                                         | 101 |
| Figure A.2  | Structure des dossiers du jeu de données brut. . . . .                                                                                                                                                                                                                                                                                                        | 102 |
| Figure A.3  | Structure des dossiers du jeu de données prétraité. . . . .                                                                                                                                                                                                                                                                                                   | 103 |
| Figure B.1  | Architecture du réseau de vitesse <b>ConditionalUnet1D</b> : encodeur-décodeur 1D à trois niveaux de résolution, connexions de saut et injection du conditionnement global dans chaque bloc résiduel. Chaque triplet indique la forme du tenseur transitant entre les blocs : taille de lot $B$ , nombre de canaux, longueur temporelle ( $H = 16$ ). . . . . | 107 |

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                                        |     |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Figure B.2 | Architecture du bloc résiduel conditionnel ( <b>ConditionalResidualBlock1D</b> )<br>du réseau de vitesse, avec sa modulation affine par le conditionnement<br>global. $B$ désigne la taille du lot, $C_{\text{in}}$ et $C_{\text{out}}$ les nombres de canaux<br>à l'entrée et à la sortie du bloc, $H$ la longueur temporelle du niveau<br>de résolution courant et $D_{\text{cond}} = 384$ la dimension du conditionnement<br>global $c_g$ . . . . . | 108 |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|

## LISTE DES SIGLES ET ABRÉVIATIONS

|       |                                            |
|-------|--------------------------------------------|
| API   | Application Programming Interface          |
| ARA   | Assistive Robotic Arm                      |
| BC    | Behavior Cloning                           |
| CBF   | Control Barrier Function                   |
| CFM   | Conditional Flow Matching                  |
| CPU   | Central Processing Unit                    |
| DDL   | Degré de liberté                           |
| FM    | Flow Matching                              |
| GPU   | Graphics Processing Unit                   |
| IA    | Intelligence Artificielle                  |
| JSON  | JavaScript Object Notation                 |
| LLM   | Large Language Model                       |
| QP    | Quadratic Program                          |
| RAF   | Robotic Assisted Feeding                   |
| RDF   | Robot Distance Field                       |
| RGB-D | Red-Green-Blue + Depth                     |
| RL    | Reinforcement Learning                     |
| RLHF  | Reinforcement Learning from Human Feedback |
| ROS   | Robot Operating System                     |
| SAM   | Segment Anything Model                     |
| SDF   | Signed Distance Field                      |
| SO    | Specific objective                         |
| VLM   | Vision Language Model                      |
| YAML  | YAML Ain't Markup Language                 |

**LISTE DES ANNEXES**

|          |                                                               |     |
|----------|---------------------------------------------------------------|-----|
| Annexe A | Protocole détaillé d’acquisition des démonstrations . . . . . | 100 |
| Annexe B | Détails de l’architecture du réseau de vitesse . . . . .      | 105 |
| Annexe C | Entraînement du champ de distance signée du robot . . . . .   | 109 |
| Annexe D | Compléments du filtre de sécurité SDF-CBF . . . . .           | 111 |
| Annexe E | Résultats complémentaires . . . . .                           | 113 |



## CHAPITRE 1 INTRODUCTION

Présenter la problématique de recherche, comme quoi l'utilisation des modèles d'IA générative pour des tâches robotiques réelles ne sont pas couramment utiliser puisqu'ils nécessite le positionnement d'une caméra externe, parce que c'est dans des environnements dynamiques et que si on veut qu'ils soient réactifs aux collisions, il faut entraîner le modèle sur plusieurs centaines de données au lieu d'une quarantaine pour une tâche afin de lui faire faire apprendre l'évitement de collision. cela complexifie alors parler aussi des robots collaboratifs qui permettent de réaliser des tâches, mais que ces tâches sont pour la plupart hardcoder et ne réagissent pas aux stimuli environnants, ce qui peut générer des trajectoires dangereuses.

Parler de la motivation de la recherche. Pour moi, serait que la génération de trajectoire par intelligence artificielle, pour ne pas accumuler des centaines de données pour faire de l'évitement de collision doit avoir un système d'évitement de collision sur la trajectoire réalisée. Ce système peut se trouver lors de l'inférence du modèle ou bien après l'inférence du modèle. Lors de l'inférence du modèle, c'est trop long pour des environnements dynamiques et après il faut considérer l'environnement, ce qui nécessite l'entraînement d'un réseau que pour l'environnement. De plus, les systèmes actuels nécessite une caméra externe calibrée adéquatement, ce qui n'est pas utilisable pour un usage quotidien. Ce qu'on propose c'est un système d'évitement de collision pour des environnements dynamiques pour des environnements non vues pendant l'entraînement avec une seule caméra au niveau de l'effecteur du robot.

Faire un paragraphe par rapport à la structure du mémoire. Celle de Félix c'était : Chapter 2 reviews the literature on robot-assisted feeding, highlighting current challenges in perception, task planning, and manipulation. Chapter 3 defines the specific scope and research objectives of this project. Chapter 4 details the design of the system, describing the system architecture, the integration of Vision-Language Models (VLMs) for autonomous food acquisition planning, and the implementation of the food perception and manipulation modules. Chapter 5 presents the experimental results, including an analysis of VLM-based food acquisition planning performance and a validation of the system on a physical robot. Finally, Chapter 6 summarizes the main contributions, discusses the system's limitations, and proposes directions for future research.

Donc nous cela pourrait être "Le chapitre 2 présente la revue de littérature en lien avec la génération de trajectoires robotiques en utilisant l'IA générative mettant en évidence l'exécution des trajectoires la vision par ordinateur et le suivi de cible ainsi que l'évitement de

collision. Le chapitre 3 présente la motivation, les objectifs de recherche ainsi que la portée du projet. Le chapitre 4 présente le design du système, mettant de l'avant son architecture, l'intégration des systèmes de vision par ordinateur pour détecter et suivre la cible, l'acquisition des données pour entraîner le modèle de Flow Matching, la génération des trajectoires générées par IA ainsi, la capture des obstacles, le filtre de sécurité utilisé et l'intégration du temps réel. Le chapitre 5 présente les résultats expérimentaux, la validation du modèle géométrique de sécurité, la qualité de la trajectoire nominale, l'analyse des vitesses, la sécurité et l'évitement d'obstacle, les performances computationnelles ainsi que les limites du système.

"

## CHAPITRE 2 REVUE DE LITTÉRATURE

Cette revue de littérature présente une analyse de l'état de l'art pour la génération de trajectoires robotisées sûres en environnement quasi statique et non structuré. Elle est organisée selon l'architecture en chaîne « perception, planification, action » proposée dans ce travail. Nous abordons d'abord les méthodes de génération de mouvement, en situant l'émergence du *Flow Matching* conditionnel retenu comme planificateur nominal. Ensuite, nous examinons les techniques de perception visuelle et de mémoire spatiale qui fournissent le conditionnement géométrique nécessaire. Les représentations géométriques permettant d'évaluer les collisions en temps réel sont ensuite analysées, suivies par les lois de commande sécuritaires réactives fondées sur la théorie du contrôle. Enfin, nous étudions les stratégies de couplage entre modèles génératifs et filtres de sécurité, ce qui prépare le positionnement de l'architecture découplée proposée dans ce mémoire.

### 2.1 Génération de trajectoires robotisées

Cette première section retrace le développement théorique et les limites connues des méthodes de génération de mouvement, afin de situer le choix du *Flow Matching* conditionnel comme planificateur d'intention nominale.

#### 2.1.1 Planification classique par échantillonnage et optimisation

La génération de mouvements sans collision a d'abord été abordée par des algorithmes de recherche explicites, raisonnant directement dans l'espace des configurations sans apprentissage préalable. Deux familles s'y côtoient, selon que l'espace des configurations est exploré par tirages aléatoires ou qu'une trajectoire y est raffinée par optimisation continue. Les méthodes par échantillonnage, à l'instar de l'algorithme *Rapidly-exploring Random Tree star* (RRT\*) de Karaman et Frazzoli [1], construisent incrémentalement un arbre de configurations valides. Bien que ces planificateurs, largement implémentés au sein de la bibliothèque *Open Motion Planning Library* (OMPL) [2], garantissent une complétude probabiliste, voire une optimalité asymptotique pour les variantes étoilées comme RRT\*, ils produisent généralement des trajectoires géométriquement rugueuses et saccadées. Ce défaut impose une étape de lissage a posteriori qui dépend fortement de la complexité géométrique de la scène [3].

Les approches par optimisation locale adoptent la démarche inverse et raffinent une trajectoire initiale par minimisation d'une fonction de coût. Par exemple, CHOMP (*Covariant Hamil-*

*tonian Optimization for Motion Planning*) [4] exploite le gradient d'un champ de distance signé pré-calculé pour repousser la trajectoire d'initialisation (typiquement une interpolation rectiligne dans l'espace des configurations) hors des obstacles tout en la lissant. De même, TrajOpt [5] formule le problème sous forme d'une séquence d'optimisations convexes sous contraintes relaxées de non-collision, avec des taux de réussite élevés sur des bras articulés complexes. Ces méthodes restent toutefois sensibles à la trajectoire d'initialisation et tendent à converger vers des minima locaux sous-optimaux, ce qui peut conduire à des violations de contraintes si le champ de distance est bruité. Pour pallier cette dépendance à l'initialisation, les approches classiques d'amorçage (*warm-starting*) proposent d'utiliser des heuristiques géométriques ou des chemins sans collision issus d'algorithmes par échantillonnage, comme RRT-Connect [6] ou des cartes routières probabilistes (PRM) [7], pour enseigner l'optimiseur. Ce couplage hybride échantillonnage-optimisation combine la complétude globale du premier et la régularité locale du second. Il annonce déjà les méthodes fondées sur l'apprentissage : plutôt que d'ensemencer un optimiseur par une recherche géométrique explicite, celles-ci apprennent le mouvement directement de démonstrations expertes.

### 2.1.2 Apprentissage par démonstration, clonage de comportement et renforcement

L'apprentissage par démonstration (*Learning from Demonstration*, LfD), dont les fondations théoriques ont été posées par Atkeson et Schaal [8,9] en 1997 et 1999, avant d'être synthétisées par Argall et al. [10] en 2009, vise à reproduire un comportement expert sans programmer explicitement la loi de commande. La formulation la plus directe, le clonage de comportement (*Behavior Cloning*, BC), dont le concept remonte aux travaux pionniers d'ALVINN de Pomerleau [11] et a été formalisé par Bain et Sammut [12], traite le problème comme une régression supervisée des actions expertes à partir des observations. Bien qu'appliqué avec succès à des tâches d'acquisition alimentaire par imitation visuelle ou de manipulation fine moderne [13–15], le BC sous perte quadratique (*Mean Squared Error*, MSE) souffre de deux limites structurelles majeures : l'erreur cumulative (*compounding errors*), où de petites déviations entraînent le robot hors de la distribution d'entraînement, et l'effondrement de la multimodalité, la moyenne de démonstrations divergentes produisant des actions aberrantes ou invalides. La figure 2.1 représente un cas simple de trajectoire générée par BC hors distribution qui ne reflète pas les trajectoires expertes.

Pour surmonter ces écueils, Florence et al. [17] ont introduit le clonage de comportement implicite (*Implicit BC*) représentant la politique par un modèle d'énergie. Cette formulation permet de capturer des distributions d'actions multivaluées et discontinues, surpassant la

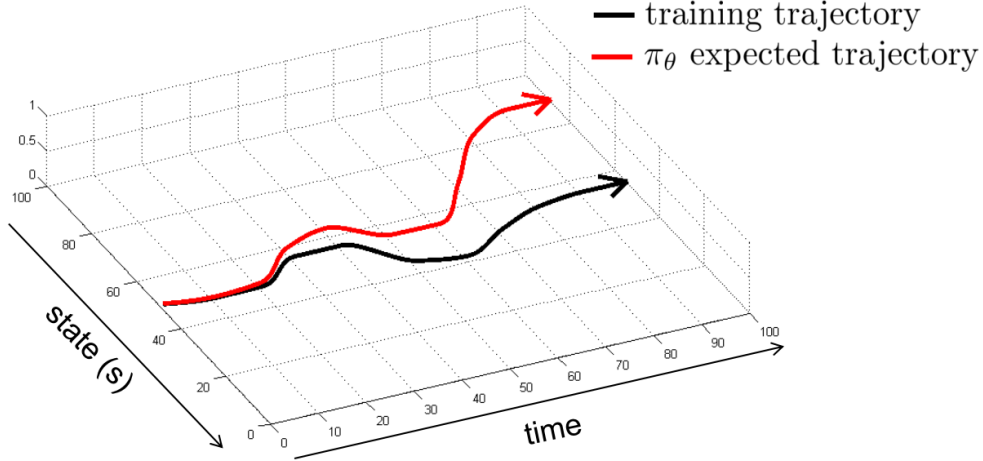


Figure 2.1 Représentation d’une trajectoire obtenue par *Behavior Cloning* hors distribution, tirée de [16]

régression directe sur des tâches nécessitant une grande précision géométrique. Des travaux connexes étendent cette logique à l’imitation strictement hors ligne [18] ou exploitent des représentations auto-supervisées pour accroître la généralisation du système [19]. L’apprentissage par renforcement (*Reinforcement Learning*, RL) contourne quant à lui ces limites de l’imitation en optimisant directement une mesure de récompense par interaction avec l’environnement, plutôt qu’en régressant vers des démonstrations expertes potentiellement divergentes [20, 21]. Son inefficacité d’échantillonnage prohibitive et les risques de collision inhérents à l’exploration compliquent cependant son déploiement direct sur des plateformes physiques réelles. Une dernière voie encode le mouvement sous forme de systèmes dynamiques stables par construction (tels que l’estimateur SEDS [22]) afin de garantir la convergence vers la cible, au détriment de l’expressivité par rapport aux modèles génératifs récents.

### 2.1.3 Modèles génératifs, politiques de diffusion et Flow Matching

Les politiques génératives modélisent explicitement la distribution des trajectoires expertes, ce qui leur permet de représenter naturellement la multimodalité inhérente aux comportements humains [23, 24]. En 2023, la *Diffusion Policy* de Chi et al. [25], inspirée des modèles probabilistes de débruitage (*Denoising Diffusion Probabilistic Models*) [26] et de la planification générative [27, 28], a démontré de solides performances en manipulation d’objets par un processus de débruitage stochastique itératif. Sa variante tridimensionnelle (*3D Diffusion Policy*) conditionne cette génération sur un nuage de points [29], renforçant la robustesse géométrique. Le principal inconvénient des politiques de diffusion réside dans leur latence d’inférence élevée : la génération d’une trajectoire requiert de nombreuses évaluations séquentielles

du réseau de neurones, limitant leur application en boucle fermée à haute fréquence. Bien que des techniques de distillation de modèle (telles que ManiCM [30] ou d'autres approches accélérées [31, 32]) parviennent à ramener l'inférence à une seule étape, elles nécessitent des phases d'entraînement complexes et peuvent dégrader la stabilité des trajectoires générées.

Pour offrir une alternative plus directe et efficace au débruitage stochastique itératif, Lipman et al. [33] ont introduit le formalisme du *Flow Matching*. L'objectif est d'apprendre un champ de vitesses  $u_t^\theta(X)$ , indexé par un temps d'écoulement  $t \in [0, 1]$ , qui déplace continûment un échantillon initial issu d'une distribution simple  $p_0$  (typiquement une gaussienne standard  $\mathcal{N}(\mathbf{0}, \mathbf{I})$ ) vers la distribution cible des trajectoires expertes  $p_1$ . La figure 2.2 permet de visualiser le champ de vitesses déplaçant une distribution initiale vers une distribution finale.

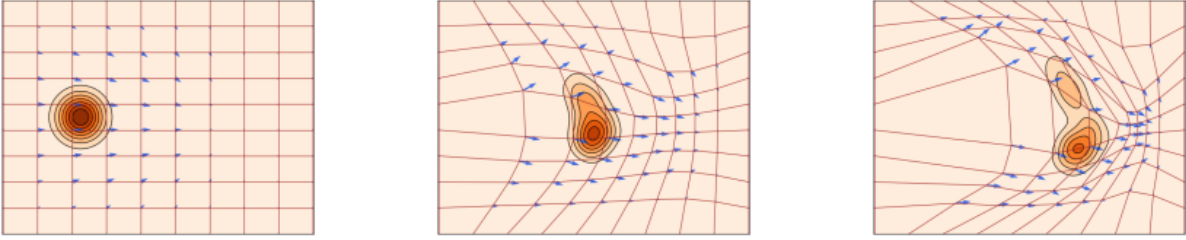


Figure 2.2 Représentation du champ de vitesses  $u_t$  transportant la distribution initiale  $p_0$  vers la distribution cible  $p_1$  pour  $t \in [0, 1]$ , tirée de [34]

Une trajectoire candidate  $X_t$  est générée en intégrant l'équation différentielle ordinaire (ODE) associée :

$$\frac{dX_t}{dt} = u_t^\theta(X_t). \quad (2.1)$$

Le champ de vitesses marginal engendrant la distribution de probabilité  $p_t$  étant généralement incalculable analytiquement, le *Conditional Flow Matching* (CFM) [35] résout ce problème en définissant un chemin de probabilité conditionnel  $p_t(X | X_0, X_1) = \mathcal{N}(X | \mu_t(X_0, X_1), \sigma_t^2 \mathbf{I})$  et son champ de vitesses associé  $u_t(X | X_0, X_1)$  qui sont analytiquement traitables. Pour des distributions gaussiennes conditionnelles, ce champ cible s'exprime par :

$$u_t(X | X_0, X_1) = \frac{\dot{\sigma}_t}{\sigma_t} (X - \mu_t(X_0, X_1)) + \dot{\mu}_t(X_0, X_1), \quad (2.2)$$

où  $\dot{\mu}_t$  et  $\dot{\sigma}_t$  désignent les dérivées temporelles des paramètres du chemin. Le réseau de neurones  $u_t^\theta$  est alors entraîné en minimisant la perte de régression du champ de vitesses conditionnel :

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1], X_1 \sim p_1, X_0 \sim p_0, X_t \sim p_t(\cdot | X_0, X_1)} \left[ \left\| u_t^\theta(X_t) - u_t(X_t | X_0, X_1) \right\|^2 \right]. \quad (2.3)$$

Dans le cadre du *Rectified Flow* proposé par Liu et al. [36], le chemin est défini par interpolation linéaire simple entre le bruit et la donnée réelle, avec une variance constante :

$$\mu_t(X_0, X_1) = tX_1 + (1 - t)X_0, \quad \sigma_t = \sigma_{\min}, \quad (2.4)$$

ce qui réduit le champ cible  $u_t$  au vecteur constant reliant le point initial au point final :

$$u_t(X_t | X_0, X_1) = X_1 - X_0. \quad (2.5)$$

Lorsque  $\sigma_{\min} = 0$ , le chemin se réduit à l'interpolation déterministe  $X_t = tX_1 + (1 - t)X_0$  ; une variance résiduelle  $\sigma_{\min} > 0$  régularise l'entraînement, au prix d'atteindre non pas la cible exacte mais sa convolution par un bruit gaussien d'écart-type  $\sigma_{\min}$ . Cette formulation linéaire favorise des trajectoires d'intégration quasi rectilignes, permettant d'intégrer l'ODE en un nombre réduit d'étapes de discrétisation tout en préservant la qualité des trajectoires ; l'intégration est déterministe conditionnellement à l'échantillon initial  $X_0$ , la génération demeurant stochastique à travers son tirage. Pour adapter ce formalisme à la robotique, il suffit de conditionner le champ  $u_t^\theta(X | c)$  sur une observation sémantique ou géométrique  $c$ , ce qui permet d'obtenir un planificateur de trajectoire réactif, comme l'illustre le framework PointFlowMatch [37].

Plus récemment, de nombreuses extensions ont cherché à accroître l'efficacité et la généralisation géométrique de ces modèles. Des approches telles que ManiFlow [38], MeanFlow [39, 40] et les politiques à flux continu [41] visent la génération de mouvements en une ou deux étapes d'inférence. D'autres travaux, à l'instar de FlowMP [42, 43], étendent le *Flow Matching* aux dynamiques du second ordre afin de générer des trajectoires respectant les contraintes d'accélération et de couple. La politique riemannienne (*Riemannian Flow Matching Policy*, RFMP) de Braun et al. [44] transpose quant à elle ce formalisme directement sur des variétés non euclidiennes telles que  $SE(3)$  par interpolation géodésique, réduisant les saccades de rotation et le temps de calcul. Cette combinaison d'expressivité multimodale, d'intégration rapide et reproductible à échantillon initial fixé, et de robustesse géométrique fait du *Flow Matching* un candidat de premier plan pour la génération d'intention nominale en manipulation réactive.

## 2.2 Perception visuelle et mémoire spatiale

Le planificateur nominal qui précède suppose un signal de conditionnement géométrique stable. Cette section documente les mécanismes qui convertissent un flux de données visuelles bidimensionnelles et bruitées en un tel signal, statique et persistant.

### 2.2.1 Vision-langage, segmentation zéro-shot et suivi visuel

La première étape de l’architecture consiste à extraire de la scène l’objet pertinent pour la tâche et à l’ancrer sémantiquement. Deux paradigmes associent perception et action : les modèles monolithiques, qui les fusionnent, et les architectures modulaires, qui séparent la compréhension sémantique de la génération de mouvement. Les architectures *Vision-Language-Action* (VLA) monolithiques intègrent la perception sémantique et le contrôle dans un unique réseau de neurones entraîné de bout en bout. Des modèles tels que RT-2 de Brohan et al. [45] ou *OpenVLA* de Kim et al. [46] en sont particulièrement représentatifs. Ce dernier, fort de sept milliards de paramètres, surpasse significativement les modèles propriétaires antérieurs sur une grande variété de tâches de manipulation [47, 48]. Leur latence d’inférence élevée demeure toutefois peu compatible avec un contrôle réactif en temps réel ; des versions compressées comme TinyVLA [49] tentent d’accélérer ce processus, sans lever complètement ce verrou.

Les approches modulaires, à l’inverse, découplent l’exécution en confiant l’interprétation sémantique de l’invite textuelle (par exemple, identifier l’aliment à acquérir) à un modèle de vision-langage ou à un grand modèle de langage (LLM) [50–53], laissant la localisation géométrique fine à des modules dédiés. Ce découplage préserve l’interprétabilité du système et permet de remplacer indépendamment chaque composant. Pour la segmentation fine des objets, le modèle fondateur *Segment Anything* (SAM) de Kirillov et al. [54], sorti en 2023, a démontré de fortes capacités de généralisation zéro-shot sans réentraînement. Son successeur, le modèle SAM2 sorti en 2024 par Ravi et al. [55], a étendu cette logique au domaine temporel en assurant la propagation cohérente et le suivi des masques sémantiques le long du flux d’images. Plus récemment, en 2025, la version *Segment Anything 3* (SAM3) [56] a introduit la segmentation par concept, permettant de localiser toutes les instances correspondant à une requête sémantique en une seule inférence. C’est ce masque, maintenu cohérent au fil des trames et des requêtes, que la reconstruction 3D examinée ci-dessous convertit en représentation géométrique de la scène.

### 2.2.2 Reconstruction RGB-D et représentation par nuages de points

Une fois le masque sémantique obtenu en deux dimensions, la rétroprojection géométrique de ses pixels dans l’espace cartésien, facilitée par les caméras RGB-D, permet de générer, par un calcul direct et sans apprentissage, des nuages de points tridimensionnels de la cible et des obstacles. Se pose alors la question de la représentation géométrique optimale pour conditionner les politiques génératives. Le conditionnement direct sur des nuages de points s’est imposé comme une alternative solide aux images brutes, car il capture explicitement les



relations spatiales tridimensionnelles de la scène [57, 58] ; la politique *3D Diffusion Policy* [29] et le modèle fondationnel FP3 [59] généralisent ainsi à de nouvelles géométries avec un nombre réduit de démonstrations expertes.

L’acquisition d’un nuage de points à partir d’une unique caméra embarquée reste cependant vulnérable aux occlusions physiques induites par les mouvements de l’effecteur ou du bras articulé lui-même. Pour y faire face, des méthodes comme CordViP de Fu et al. [60] proposent de reconstruire les zones masquées en combinant proprioception et estimation de pose 6D, restaurant ainsi la géométrie de la zone d’interaction. Bien que certains travaux suggèrent que des approches bidimensionnelles enrichies en pseudo-3D puissent parfois suffire [61, 62], l’utilisation de nuages de points 3D bruts présente l’avantage de ne requérir aucun modèle CAO préalable des objets de la scène, ce qui s’avère indispensable pour manipuler des formes complexes ou non structurées.

### 2.2.3 Cartes persistantes face aux occlusions et pertes de détection

La perception instantanée, aussi précise soit-elle, ne suffit pas lors de tâches robotiques interactives dans des environnements dynamiques. Dès que le robot effectue une approche, son propre corps engendre des occlusions physiques massives, masquant temporairement la cible ou les obstacles proches [63]. Une politique fonctionnant purement en boucle ouverte et sans mémoire spatiale serait alors incapable de maintenir sa consigne de mouvement. Pour résoudre cette limite, la cartographie d’environnements dynamiques propose d’intégrer des graphes de facteurs (à l’instar de DynoSAM [64]) afin de séparer la géométrie statique des trajectoires d’objets mobiles. Des approches probabilistes comme SE(3)-PoseFlow de Jin et al. [65] utilisent quant à elles le *Flow Matching* pour estimer les distributions de poses sous forte occlusion.

Afin de stabiliser durablement le signal de conditionnement fourni au planificateur nominal, l’intégration d’une couche de mémoire spatiale persistante, telle qu’une grille de voxels locale ou une carte d’occupation historique, s’appuie classiquement sur des méthodes de cartographie par grille d’occupation [66, 67]. En accumulant et en filtrant les nuages de points capturés au fil du temps par fusion spatio-temporelle [68, 69], cette mémoire spatiale maintient une représentation géométrique tridimensionnelle fiable des obstacles et de la cible, même lors de pertes de détection visuelle. Ce mécanisme garantit la continuité géométrique du signal envoyé au planificateur.

## 2.3 Représentations géométriques pour la collision

Cette section évalue les méthodes de modélisation de l'encombrement spatial du robot et des obstacles à partir desquelles une métrique de distance différentiable peut être formulée.

### 2.3.1 Champs de distance de la scène et limites computationnelles

Imposer une contrainte d'évitement active suppose de disposer d'une métrique de distance différentiable entre le robot et les obstacles de son environnement. Une première famille de méthodes centre cette métrique sur la scène en discrétisant l'espace de travail. Les champs de distance signés euclidiens (*Euclidean Signed Distance Fields*, ESDF) encodent, en chaque point de l'espace, la distance à la surface de l'obstacle le plus proche. Formellement, le champ associe à un point  $\mathbf{p} \in \mathbb{R}^3$  sa distance signée à la surface d'un obstacle  $\mathcal{O} \subset \mathbb{R}^3$  :

$$d(\mathbf{p}) = s_{\mathcal{O}}(\mathbf{p}) \min_{\mathbf{q} \in \partial\mathcal{O}} \|\mathbf{p} - \mathbf{q}\|_2, \quad s_{\mathcal{O}}(\mathbf{p}) = \begin{cases} -1, & \mathbf{p} \in \mathcal{O}, \\ +1, & \text{sinon,} \end{cases} \quad (2.6)$$

où la distance est ainsi comptée négativement à l'intérieur du volume de l'obstacle et positivement à l'extérieur, la frontière  $\partial\mathcal{O}$  correspondant au niveau zéro  $d(\mathbf{p}) = 0$ . Un tel champ de distance théorique vérifie, presque partout, l'équation eikonale :

$$\|\nabla d(\mathbf{p})\|_2 = 1, \quad (2.7)$$

indiquant que le gradient du champ est unitaire et pointe dans la direction d'éloignement la plus rapide de l'obstacle, ce qui fournit directement la direction d'évitement ; l'égalité vaut presque partout, le gradient n'étant pas défini sur l'axe médian, lieu des points équidistants de plusieurs faces.

Pour évaluer numériquement ce champ de distance de la scène en robotique, trois grands paradigmes coexistent :

1. **Le pré-calcul hors ligne (Offline) :** Pour des environnements statiques et connus, l'ESDF est échantillonné à l'avance sur une grille de voxels à partir du modèle CAO de la scène. Cette méthode, utilisée historiquement dans des planificateurs comme CHOMP [4], permet des requêtes de distance en temps constant  $\mathcal{O}(1)$  lors de la planification, mais s'avère incapable de s'adapter au moindre changement d'environnement ou à l'apparition d'obstacles imprévus.
2. **La reconstruction en ligne par grilles volumétriques incrémentales (Online Grids) :** Les bibliothèques de cartographie en direct, à l'instar d'OctoMap [67], Vox-

blox [70] ou nvblox [71], maintiennent et mettent à jour incrémentalement l’ESDF à partir des flux de données de capteurs RGB-D. Pour accroître la rapidité ou la précision de ces grilles, des architectures optimisées comme FIESTA [72] (optimisant la vitesse de calcul pour la planification de drones) ou Voxfield [73] (qui exploite des TSDF non projectifs pour améliorer la fidélité de la distance) ont été développées. Bien que ces grilles s’adaptent aux scènes dynamiques et exploitent le parallélisme des GPU [74], leur coût de mise à jour tridimensionnelle demeure élevé et dépend fortement de la résolution choisie, ce qui devient prohibitif à la fréquence d’une boucle de commande réactive.

3. **Les champs de distance neuronaux continus en ligne (Online Continual Neural SDF) :** Afin de s’affranchir de la discrétisation par grilles volumétriques, des approches de reconstruction continue en ligne, comme iSDF d’Ortiz et al. [75], entraînent en temps réel un réseau de neurones de coordonnées sur le flux sensoriel actif pour apprendre une représentation continue et différentiable de la scène. Ce concept s’inscrit dans la lignée des frameworks de SLAM implicite en temps réel tels qu’iMAP [76] et sa variante hiérarchique NICE-SLAM [77]. Cependant, ces modèles neuronaux souffrent d’une latence de convergence lors de changements rapides de la scène et de risques d’oubli catastrophique, ce qui complique leur intégration directe dans une boucle de contrôle sécuritaire à haute fréquence.

Ces limitations computationnelles partagées motivent le développement de représentations alternatives centrées sur le robot plutôt que sur la scène.

### 2.3.2 Primitives conservatives et modèles géométriques simplifiés

Pour contourner la lourdeur du calcul des champs de distance globaux de la scène, une seconde famille de méthodes choisit plutôt de simplifier la géométrie du robot lui-même. L’approche classique consiste à encapsuler les différents liens de la chaîne cinématique dans des volumes englobants simples, tels que des sphères ou des capsules, dont les tests de collision analytiques se parallélisent très efficacement sur GPU. La bibliothèque cuRobo de Sundaralingam et al. [78] exploite cette idée pour générer des mouvements sûrs en quelques dizaines de millisecondes, surpassant de plus d’un ordre de grandeur les solveurs classiques de l’état de l’art. Récemment, ces enveloppes ont été adaptées pour prendre en compte l’outil monté à l’effecteur en temps réel [79]. Le revers de cette approximation géométrique conservative est qu’elle tend à surestimer grandement l’encombrement réel du robot. Cette perte d’espace libre exploitable restreint la capacité du robot à évoluer dans des zones confinées, ce qui est problématique pour des tâches d’assistance physique comme l’alimentation.

### 2.3.3 Champs de distance signée du robot

Afin de concilier la précision géométrique et la rapidité de calcul, un changement de paradigme récent propose de déplacer le champ de distance de la scène vers le robot en modélisant directement la géométrie du bras articulé. Les champs de distance du robot (*Robot Distance Fields*, RDF), introduits par Li et al. [80], approchent la SDF de chaque lien du robot par des polynômes de Bernstein. Pour une coordonnée spatiale normalisée  $\mathbf{u} = (u_1, u_2, u_3) \in [0, 1]^3$ , la distance signée associée à un lien  $l$  s'écrit comme un produit tensoriel 3D de la base polynomiale de Bernstein de degré  $n$  :

$$\bar{d}_l(\mathbf{u}) = \sum_{i=0}^n \sum_{j=0}^n \sum_{k=0}^n c_{ijk}^l B_{i,n}(u_1) B_{j,n}(u_2) B_{k,n}(u_3), \quad (2.8)$$

où les coefficients  $c_{ijk}^l$ , appris hors ligne à partir du modèle CAO, encodent précisément la forme géométrique tridimensionnelle du lien. Les polynômes de base de Bernstein de degré  $n$  sont définis analytiquement par :

$$B_{i,n}(u) = \binom{n}{i} u^i (1-u)^{n-i}. \quad (2.9)$$

Puisque la distance approchée  $\bar{d}_l(\mathbf{u})$  est une fonction polynomiale, son gradient par rapport aux coordonnées spatiales s'obtient analytiquement sans nécessiter de coûteuses approximations par différences finies. En particulier, la dérivée d'un polynôme de base par rapport à la coordonnée normalisée  $u$  s'écrit simplement sous la forme :

$$\frac{d}{du} B_{i,n}(u) = n (B_{i-1,n-1}(u) - B_{i,n-1}(u)), \quad (2.10)$$

avec la convention que  $B_{-1,n-1}(u) = 0$  et  $B_{n,n-1}(u) = 0$ . Cette propriété permet un calcul exact et instantané du gradient spatial sur GPU, pour une empreinte mémoire négligeable.

Des représentations neuronales, tels les *Neural SDF* [81], approchent également la SDF du robot ; leur coût d'inférence, initialement élevé, a été sensiblement réduit par des décompositions suivant la cinématique directe en réseaux légers évalués en parallèle [82]. Ces modèles demeurent toutefois dépourvus de la régularité mathématique et des gradients analytiques exacts qu'offre l'approximation polynomiale de Bernstein, qui fournit une structure différentiable et continue. Ses gradients pouvant être interrogés directement par rapport aux points de l'environnement, Govoni et al. [83] les ont exploités pour l'évitement réactif dans un cadre de commande de tâches ; dans une veine voisine, la politique SAMP [84] encode conjointement les SDF du robot et de l'environnement sur des ancres spatiales partagées afin de conditionner

une politique de mouvement apprise sensible aux collisions. Le même courant de recherche a par ailleurs poussé la logique centrée sur le robot jusque dans l'espace des configurations : les champs de distance de configuration (*Configuration space Distance Fields*, CDF) de Li et al. [85] expriment la distance aux obstacles directement en coordonnées articulaires, de sorte que le gradient livre, sans chaînage cinématique supplémentaire, la direction articulaire d'éloignement. L'usage d'opérateurs de composition non lisses (comme le minimum discret sur les distances des différents liens ou le *soft-min* pour regrouper les contributions de l'environnement) implique en revanche que la fonction de distance globale du robot n'est pas strictement de classe  $\mathcal{C}^2$ , une limitation théorique dont il faut tenir compte lorsque cette distance est exploitée comme fonction barrière dans une loi de commande.

## 2.4 Sécurité réactive et fonctions de contrôle barrières

Cette section analyse les lois de commande réactives issues de la théorie du contrôle qui imposent une contrainte de sécurité en vitesse, ainsi que les hypothèses sous lesquelles leurs garanties valent.

### 2.4.1 Champs de potentiel et méthodes réactives classiques

La sécurité réactive consiste à modifier localement et en temps réel le mouvement du robot directement lors de son exécution sur la plateforme physique. Historiquement, la première formulation repose sur des forces virtuelles à travers la méthode des champs de potentiel artificiels (APF) introduite par Khatib [86]. Le principe est d'associer à la cible une force attractive et aux obstacles des forces répulsives. Bien que cette approche soit très légère en calcul et s'implémente facilement en ligne, elle souffre de limites théoriques majeures : l'absence de garanties de stabilité pour des dynamiques complexes, des oscillations à proximité des obstacles et un blocage fréquent dans des minima locaux induits par des géométries concaves. Le concept reste malgré tout exploité dans des architectures génératives récentes pour guider des politiques d'imitation vers des trajectoires sûres [87].

### 2.4.2 Fonctions barrières de contrôle et invariance avant

Pour pallier l'absence de garanties formelles de sécurité des champs de potentiel, les fonctions barrières de contrôle (*Control Barrier Functions*, CBF), formalisées par Ames et al. [88] sur la base des certificats de barrière de Wieland et Allgöwer [89], offrent un cadre d'invariance

rigoureux. Considérons un système dynamique non linéaire affine en contrôle de la forme :

$$\dot{\mathbf{x}} = f(\mathbf{x}) + g(\mathbf{x})\mathbf{u}, \quad (2.11)$$

où  $\mathbf{x} \in \mathbb{R}^d$  désigne l'état et  $\mathbf{u} \in \mathbb{R}^m$  l'action de commande. On définit un ensemble de sécurité  $\mathcal{C}$  comme le sous-ensemble de l'espace d'état où une fonction scalaire continûment différentiable  $h : \mathbb{R}^d \rightarrow \mathbb{R}$  reste positive :

$$\mathcal{C} = \{\mathbf{x} \in \mathbb{R}^d : h(\mathbf{x}) \geq 0\}, \quad (2.12)$$

la frontière  $h(\mathbf{x}) = 0$  marquant le contact ou la limite de sécurité avec l'obstacle. L'ensemble  $\mathcal{C}$  est dit invariant vers l'avant si toute trajectoire débutant dans  $\mathcal{C}$  y demeure indéfiniment. En exploitant les dérivées de Lie  $L_f h(\mathbf{x}) = \nabla h(\mathbf{x})^\top f(\mathbf{x})$  et  $L_g h(\mathbf{x}) = \nabla h(\mathbf{x})^\top g(\mathbf{x})$ , la fonction  $h$  est une fonction barrière de contrôle s'il existe une fonction  $\alpha$  de classe  $\mathcal{K}$ , c'est-à-dire strictement croissante et nulle à l'origine, telle que, dans le cas général :

$$\sup_{\mathbf{u} \in \mathbb{R}^m} [L_f h(\mathbf{x}) + L_g h(\mathbf{x})\mathbf{u}] \geq -\alpha(h(\mathbf{x})). \quad (2.13)$$

Cette condition autorise la fonction barrière à décroître à l'approche de la frontière, mais à un taux borné par  $\alpha$ , qui s'annule exactement sur celle-ci : la trajectoire ne peut donc jamais quitter l'ensemble sûr  $\mathcal{C}$ . En pratique, la fonction  $\alpha$  est le plus souvent choisie linéaire,  $\alpha(h) = \kappa h$  avec un gain  $\kappa > 0$  réglant l'agressivité du freinage près de la frontière, ce qui conduit à la condition d'invariance affine en la commande :

$$L_f h(\mathbf{x}) + L_g h(\mathbf{x})\mathbf{u} \geq -\kappa h(\mathbf{x}). \quad (2.14)$$

Lorsque le contrôle du robot s'effectue directement en vitesse articulaire (degré relatif un, où l'état correspond à la configuration articulaire  $\mathbf{x} = \mathbf{q}$  et l'action à la vitesse  $\mathbf{u} = \dot{\mathbf{q}}$ ), la dynamique se réduit à  $\dot{\mathbf{q}} = \mathbf{u}$ . La condition barrière (2.14) se simplifie alors sous la forme linéaire suivante :

$$\nabla h(\mathbf{q})^\top \dot{\mathbf{q}} \geq -\kappa h(\mathbf{q}). \quad (2.15)$$

Dans le cadre de l'évitement de collision articulé, la fonction barrière  $h_i(\mathbf{q})$  associée à un point critique  $i$  du robot s'exprime directement à partir de la distance physique aux obstacles  $d_{\text{robot},i}(\mathbf{q})$  minorée d'une marge de sécurité minimale  $d_{\text{safe}}$  :

$$h_i(\mathbf{q}) = d_{\text{robot},i}(\mathbf{q}) - d_{\text{safe}}. \quad (2.16)$$

La condition (2.15) étant affine par rapport aux vitesses articulaires  $\dot{\mathbf{q}}$ , elle peut s'intégrer sous forme de contraintes linéaires au sein d'un programme quadratique (QP-CBF) résolu à chaque cycle de commande [90] :

$$\dot{\mathbf{q}}^* = \arg \min_{\dot{\mathbf{q}}} \frac{1}{2} \|\dot{\mathbf{q}} - \dot{\mathbf{q}}_{\text{nom}}\|_2^2 \quad \text{s.c.} \quad \nabla h_i(\mathbf{q})^\top \dot{\mathbf{q}} \geq -\kappa h_i(\mathbf{q}), \quad \forall i \in \{1, \dots, N_c\}, \quad (2.17)$$

où  $\dot{\mathbf{q}}_{\text{nom}}$  représente la consigne nominale et  $N_c$  désigne le nombre de contraintes actives (les points critiques les plus proches de l'environnement). Le programme quadratique (2.17) agit ainsi comme un filtre de sécurité réactif : géométriquement, il projette la vitesse nominale sur l'intersection des demi-espaces admissibles définis par les contraintes actives, un polyèdre convexe. Des extensions étendent cette formulation aux systèmes mécaniques de degré relatif deux (commande en couples ou accélérations) [91].

La résolution numérique de (2.17) peut être confiée à un solveur quadratique générique, tel qu'OSQP [92], qui retourne la solution exacte au prix d'une structure de calcul à nombre d'itérations variable. Une alternative classique existe lorsque les contraintes forment une intersection de demi-espaces linéaires découplés : la méthode des projections sur ensembles convexes (*Projections Onto Convex Sets*, POCS), issue des projections alternées de von Neumann et généralisée par Bregman [93]. Les contraintes y sont balayées cycliquement, chaque contrainte violée étant traitée par une projection en forme close sur son demi-espace. La suite de points ainsi produite converge vers un point de l'intersection [94], et chaque balayage a une forme de calcul fixe, sans branchement, qui se prête à une exécution massivement parallèle.

Le point limite de POCS est admissible, mais n'est en général pas la projection du point de départ. Retrouver exactement la solution de (2.17) requiert la variante de Dykstra [95,96], qui adjoint un terme de correction mémorisé par contrainte et qui, pour des demi-espaces, coïncide avec la méthode duale de Hildreth [97]. Cette distinction sépare les deux propriétés du filtre. L'admissibilité de la commande (la sécurité) relève de l'appartenance à l'intersection, que les projections cycliques atteignent asymptotiquement ; l'optimalité (la déviation minimale vis-à-vis de l'intention nominale) relève de la projection exacte, que seuls le solveur générique ou la variante de Dykstra garantissent.

### 2.4.3 Filtres de sécurité prédictifs et méthodes d'atteignabilité

La projection QP-CBF de l'équation (2.17) n'est toutefois qu'une instance d'un concept plus large : le filtre de sécurité, dont Hsu et al. [98] proposent une vue unifiée. Un tel filtre est un module de supervision minimalement invasif : il surveille la commande nominale, quelle qu'en soit l'origine, et ne la modifie que lorsque la sécurité l'exige. Au-delà des CBF, deux

autres familles dominant ce paradigme.

Les filtres de sécurité prédictifs de Wabersich et Zeilinger [99] transposent la philosophie de la commande prédictive : à chaque cycle, un problème de commande optimale vérifie sur un horizon fini qu’il existe encore une trajectoire de repli ramenant le système vers un ensemble sûr terminal. Cette anticipation corrige la principale faiblesse structurelle des CBF, leur myopie. Là où la contrainte barrière ne raisonne que sur la dérivée instantanée de  $h$ , le filtre prédictif détecte à l’avance les situations où les saturations d’actionneurs rendront la contrainte infaisable. Ce gain se paie toutefois en calcul : il faut résoudre en ligne un programme non convexe sur tout l’horizon, et disposer d’un modèle dynamique fiable.

Les méthodes fondées sur l’atteignabilité de Hamilton-Jacobi calculent pour leur part l’ensemble des états à partir desquels la collision devient inévitable ; elles offrent les garanties les plus robustes face aux perturbations bornées [100]. Elles souffrent en revanche de la malédiction de la dimension et restent difficilement applicables au-delà de quelques dimensions d’état sans approximation [100]. Des mises en œuvre récentes atteignent certes la fréquence de commande, mais dans un cadre restreint, où la vérification se réduit à freiner le long d’une trajectoire d’intention fixée, face à des objets suivis à erreur bornée [101].

Le cas traité dans ce mémoire ne relève pas de ce cadre : un bras à sept degrés de liberté y est contraint par des milliers de points de nuage renouvelés à chaque cycle de perception, et la correction géométrique doit être recalculée contre ce nuage brut à chaque pas de commande. De ces trois familles, la projection QP-CBF demeure la mieux adaptée à la fréquence de commande visée, au prix d’un raisonnement purement instantané.

#### 2.4.4 Limites des CBF sous perception bruitée et contrôle discret

Les garanties d’invariance établies en temps continu et sous l’hypothèse d’un modèle géométrique parfait se dégradent cependant fortement lors de leur déploiement sur une plateforme physique réelle. Trois facteurs limitent l’invariance absolue :

1. **La discrétisation temporelle :** La boucle de commande s’exécute à fréquence finie, ce qui peut provoquer des franchissements de la barrière entre deux pas de calcul. Les formulations en temps discret (DT-CBF) [102] répondent à cette limite par la condition exacte  $h(\mathbf{x}_{k+1}) \geq (1 - \gamma) h(\mathbf{x}_k)$ . Cette condition porte cependant sur l’état *suivant* : pour une barrière composée (cinématique directe et champ de distance évalués sur de nombreux points), elle est non convexe en la commande et exigerait la résolution d’un programme non linéaire à chaque cycle. Sa linéarisation au premier ordre, seule forme compatible avec le temps réel, restitue exactement la condition en temps continu avec  $\kappa = \gamma/T$  ; le reste de linéarisation doit alors être couvert par



une marge. Ce traitement échantillonné-robuste est donc la seule voie praticable à la fréquence de commande pour une barrière composée. La formulation DT-CBF ne supprime pas la marge, qui demeure nécessaire au comportement inter-échantillons et aux erreurs de perception : elle en déplace un terme vers un solveur non linéaire en ligne. Les formulations échantillonnées de Breeden et al. [103] bornent quant à elles ce comportement inter-échantillons par un terme de robustesse explicite, dérivé de constantes de Lipschitz du système plutôt que d'une marge dimensionnée globalement.

2. **Les saturations matérielles :** Les limites de vitesse et de couple des actionneurs restreignent le domaine d'admissibilité du QP. Si la contrainte CBF exige une accélération ou une vitesse supérieure aux capacités physiques du robot, le programme quadratique devient infaisable [104], compromettant la sécurité.
3. **Le bruit de perception et l'erreur d'interface :** Le calcul du gradient sur des nuages de points réels et bruités introduit des fluctuations dans les contraintes. De plus, la réalisation de la consigne  $\dot{\mathbf{q}}^*$  par le contrôleur bas niveau du robot n'est jamais instantanée ni parfaite : retards et erreurs de poursuite séparent la vitesse certifiée de la vitesse exécutée.

Ces dégradations n'abaissent toutefois pas la CBF au rang d'heuristique répulsive, et la distinction avec les champs de potentiel de la Section 2.4.1 demeure structurelle. Le potentiel ajoute une force corrective dont l'amplitude qui, lorsqu'elle est réglée, s'équilibre contre l'attraction de la tâche, sans certificat de sécurité à aucun niveau. En revanche, la CBF impose une contrainte dure sur la commande, qu'aucune intensité de la consigne nominale ne peut franchir, et son invariance est un théorème dont les hypothèses sont explicites. Le traitement rigoureux des trois écarts précédents ne consiste donc pas à renoncer à la garantie, mais à en circonscrire la portée : énoncer les hypothèses sous lesquelles le certificat vaut, quantifier les écarts de l'implémentation et les couvrir par une marge dimensionnée en conséquence, puis valider empiriquement la part qui échappe au calcul.

## 2.5 Couplage entre modèles génératifs et filtres de sécurité

Reste à allier l'expressivité et la multimodalité des modèles génératifs avec la rigueur des garanties de non-collision. La littérature répartit les réponses entre deux stratégies, examinées ici pour positionner le découplage retenu : intégrer les contraintes directement au cœur du processus de génération, ou appliquer un filtre de sécurité en aval, lors de l'exécution de la trajectoire.

### 2.5.1 Contraintes intégrées à la génération

La première stratégie injecte la contrainte de sécurité au sein même du processus d'intégration de l'équation différentielle (guidage intra-ODE). Par exemple, SafeFlow de Dai et al. [105] formule des fonctions barrières adaptées au *Flow Matching* afin de confiner le flux de trajectoires dans l'espace libre sans nécessiter de réentraînement du réseau. De son côté, CASF (*Constrained Action Streaming Flows*) de Long et al. [106] déforme localement la métrique riemannienne de l'espace de commande pour rejeter les trajectoires s'approchant des frontières d'obstacles. OmniGuide de Song et al. [107] exprime quant à lui le guidage sous forme de champs d'énergie différentiables, composés d'attracteurs et de répulseurs ancrés dans l'espace tridimensionnel, qui orientent l'échantillonnage de politiques généralistes sans réentraînement. D'autres variantes récentes adaptent ces concepts sous forme de contraintes de Lyapunov ou de barrières stochastiques intégrées aux processus de diffusion [108], ou s'appuient sur des optimisations unifiées pour garantir l'admissibilité dynamique des mouvements [109–111]. À la frontière de cette famille, SafeFlowMatcher [112] sépare l'intégration en une phase de prédiction libre, sans intervention, suivie d'une phase de correction certifiée par CBF à convergence en temps fini vers la zone sûre : une *réparation certifiée post-génération*, qui corrige le chemin produit avant toute exécution, sans agir sur la boucle de commande.

Bien qu'efficaces en simulation, ces méthodes intra-génération se heurtent à une limite structurelle : la dérive distributionnelle (*distributional drift*). En forçant le processus d'intégration à traverser des états latents qui s'écartent de la distribution des données expertes vue à l'entraînement, ces contraintes perturbent la dynamique interne du modèle, au point de dégrader la qualité globale du mouvement généré et le taux de succès sur la tâche [101]. C'est précisément pour y échapper que SafeFlowMatcher reporte son intervention sur le chemin final plutôt que sur les états latents intermédiaires ; deux limites subsistent néanmoins : la correction demeure appliquée au sein de la génération, et elle suppose une fonction barrière connue analytiquement dans l'espace d'état des points de passage, hypothèse que les scènes perçues par capteur ne satisfont pas directement. Le coût computationnel d'une résolution de contraintes répétée à chaque étape d'intégration, ou à chaque génération dans le cas de la phase de correction, limite de surcroît l'utilisation de ces méthodes pour du contrôle réactif en boucle fermée.

### 2.5.2 Filtres de sécurité appliqués à l'exécution

Pour préserver l'intégrité du modèle génératif, la seconde stratégie maintient la phase de génération nominale totalement inchangée et n'applique les contraintes de sécurité qu'en aval, lors de la commande physique du robot. Cette stratégie prolonge la logique des filtres

de sécurité de la Section 2.4.3 en l’appliquant aux politiques génératives; son illustration récente la plus aboutie est le cadre PACS (*Path-Consistent Safety filtering*) proposé par Römer et al. [101] pour les politiques de diffusion. Plutôt que de déformer géométriquement les actions générées, PACS convertit chaque bloc d’actions en une trajectoire d’intention, puis applique un freinage cohérent avec le chemin (*path-consistent braking*) : le robot est ralenti ou arrêté le long de cette trajectoire, dont la sécurité est certifiée en temps réel par une analyse d’atteignabilité ensembliste. Ce filtrage externe maintient l’exécution dans la distribution d’entraînement de la politique et fournit des garanties formelles de sécurité en environnement dynamique, en supposant toutefois des mesures de position des objets à erreur bornée. Les auteurs montrent qu’à l’inverse, un filtre réactif naïf de type CBF appliqué à l’effecteur pousse la politique hors de sa distribution d’entraînement, avec des taux de réussite inférieurs allant jusqu’à 68 % en simulation et 37 % sur plateforme réelle. Cette cohérence au chemin restreint cependant la correction au seul freinage : face à un obstacle qui bloque durablement la trajectoire nominale, le robot s’immobilise sans pouvoir le contourner, et les auteurs identifient eux-mêmes la gestion des obstacles statiques ou quasi statiques comme une perspective future. Ce constat motive une correction géométrique locale qui, lorsqu’elle doit s’écarter de la trajectoire nominale, cherche à en rester le plus proche possible afin de limiter la dérive distributionnelle.

D’autres approches confient la correction à un opérateur humain plutôt qu’à un certificat : FlowCorrect [113] adapte ainsi la politique au déploiement à partir de corrections de pose éparses fournies interactivement, rattrapant des cas d’échec proches de la collision sans réentraîner le réseau de base. Des filtres temporels spécifiques permettent également de compenser les délais d’inférence inhérents aux grands modèles génératifs [114, 115].

### 2.5.3 Découplage entre intention nominale et sécurité réactive

L’analyse croisée de ces deux familles d’approches justifie l’architecture découplée mise en œuvre dans ce travail. Plutôt que d’entrelacer la sécurité et la génération nominale, nous optons pour une isolation fonctionnelle complète entre :

1. **Un planificateur nominal en horizon glissant** basé sur le *Flow Matching*, chargé de générer l’intention nominale à long terme sans subir de dérive distributionnelle : rétroactif vis-à-vis de la cible, qu’il réobserve à chaque replanification, mais aveugle aux obstacles.
2. **Un filtre de sécurité réactif local** basé sur une projection QP-CBF, s’exécutant à haute fréquence en ligne pour intercepter les collisions physiques sur le robot.

Cette architecture partage la structure en boucle fermée de PACS (politique générative gelée,

filtrage à l'exécution, replanification en horizon glissant), mais s'en écarte sur le siège de la correction : là où PACS la confine au temps (freinage le long du chemin, sans exposition hors distribution mais sans contournement possible), le filtre proposé l'autorise dans l'espace, ce qui permet de contourner un obstacle bloquant au prix d'une exposition hors distribution non nulle. Cette exposition est toutefois minimisée par construction puisque la projection QP-CBF retourne la commande sûre la plus proche de l'intention nominale (Chapitre 4)

Ce découplage fonctionnel concilie efficacement la richesse et la multimodalité des trajectoires apprises par le modèle génératif avec la réactivité et la rigueur théorique indispensables pour améliorer la sécurité physique sur une plateforme réelle. Ce principe de séparation constitue le fil conducteur de la méthodologie proposée dans ce mémoire.

## 2.6 Synthèse critique et positionnement

Cette dernière section résume les lacunes de l'état de l'art, situe l'architecture proposée face aux cadres de référence au moyen d'un tableau comparatif, et prépare la formulation des objectifs de recherche.

### 2.6.1 Lacunes identifiées dans la littérature

L'analyse croisée de la littérature scientifique contemporaine met en évidence trois lacunes majeures :

1. **L'hypothèse d'une perception parfaite** : La plupart des cadres de travail actuels validant l'alliance du *Flow Matching* et des CBF supposent des obstacles analytiques parfaits connus a priori, se montrant ainsi inadaptés face aux nuages de points 3D bruts et bruités.
2. **L'absence de persistance temporelle des modèles génératifs** : Lors d'une occlusion temporaire de la cible, les planificateurs génératifs perdent leur signal de conditionnement géométrique et produisent des trajectoires aberrantes ou figées, faute d'une couche cartographique persistante capable de stabiliser les points de conditionnement.
3. **L'intégration temps réel des représentations géométriques** : La mise à jour en ligne des champs de distance de la scène demeure trop lourde pour la fréquence de commande ; les champs de distance centrés sur le robot lèvent ce verrou, mais leur exploitation comme fonction barrière contre des nuages de points bruts, en aval d'une politique générative, reste inexplorée.

### 2.6.2 Positionnement de l'architecture proposée

Pour répondre à ces limites, ce mémoire soutient la pertinence d'une *architecture découplée entre génération nominale et filtrage réactif de sécurité*. La trajectoire nominale est générée en horizon glissant, sans rétroaction vis-à-vis des obstacles, par un modèle *Flow Matching* conditionné par une observation géométrique persistante issue de la perception RGB-D et du suivi SAM2, tandis que la sécurité immédiate est déléguée à un filtre de projection CBF articulaire bas niveau s'appuyant sur une approximation polynomiale de Bernstein du SDF local du robot.

### 2.6.3 Vers les objectifs de recherche

Ces lacunes, mises en regard du constat de dégradation de la tâche par les filtres réactifs naïfs [101], circonscrivent une question précise : celle de la valeur ajoutée d'un filtre de sécurité indépendant, alimenté par une perception persistante et un champ de distance du corps complet du robot, appliqué en aval d'une politique de *Flow Matching* gelée, sans sacrifier ni la réussite de la tâche ni le budget temps réel. Les objectifs de recherche qui y répondent et les critères mesurables qui leur sont associés sont énoncés au Chapitre 3, à la lumière du positionnement qui suit.

Le Tableau 2.1 résume le positionnement de l'approche proposée vis-à-vis des cadres de référence de la littérature. Les critères de comparaison s'y lisent comme suit : le modèle nominal désigne la politique générative produisant l'intention de mouvement ; la sécurité qualifie la nature du mécanisme d'évitement (implicite lorsqu'elle est seulement apprise des démonstrations, souple lorsqu'elle guide la génération sans imposer de contrainte, explicite lorsqu'une contrainte est imposée à l'exécution, formelle lorsqu'elle est certifiée mathématiquement) ; la représentation précise la forme sous laquelle les obstacles sont fournis au mécanisme de sécurité ; la correction décrit comment la sortie nominale est modifiée ; la colonne occlusions indique si le conditionnement perceptif résiste à une perte temporaire de visibilité ; enfin, la réactivité évalue la capacité à réagir à un changement de scène à la fréquence de commande ; ses niveaux (limitée, moyenne, élevée) sont qualitatifs et ordonnés par la fréquence à laquelle une correction peut suivre un tel changement, telle que rapportée par les auteurs respectifs.

L'examen critique de la littérature met en évidence un compromis entre l'expressivité des politiques génératives et la rigueur des algorithmes de préservation de la sécurité. Le *Flow Matching*, lissant et déterministe à échantillon initial fixé, capture la multimodalité des mouvements experts en réduisant la latence propre aux processus de diffusion stochastique ;

Tableau 2.1 Tableau comparatif des frameworks de planification et de sécurité de l'état de l'art.

| <b>Cadre / Méthode</b> | <b>Modèle nominal</b> | <b>Sécurité</b>                         | <b>Représentation</b>                      | <b>Correction</b>                                            | <b>Occlusions</b>           | <b>Réactivité</b>                                                 |
|------------------------|-----------------------|-----------------------------------------|--------------------------------------------|--------------------------------------------------------------|-----------------------------|-------------------------------------------------------------------|
| Diffusion Policy       | Diffusion             | Implicite                               | Espace latent                              | Replanification en horizon glissant, sans filtre de sécurité | Non                         | Limitée par le débruitage                                         |
| OmniGuide              | Flow Matching         | Souple                                  | Champ d'énergie 3D                         | Guidage intra-ODE                                            | Non                         | Moyenne                                                           |
| CASF                   | Streaming Flow        | Souple                                  | Neural SDF                                 | Déformation intra-ODE                                        | Partielle                   | Élevée mais coûteuse                                              |
| SafeFlow               | Flow Matching         | Explicite : CBF adaptées au flux (FMBF) | Obstacles analytiques                      | Guidage à chaque pas d'intégration + filtre terminal         | Non                         | Limitée par la génération                                         |
| SafeFlow-Matcher       | Flow Matching         | Explicite : CBF à temps fini            | Obstacles analytiques                      | Correction certifiée du chemin final (prédiction-correction) | Non                         | Élevée en simulation                                              |
| PACS                   | Diffusion             | Formelle : atteignabilité ensembliste   | Objets suivis, erreur bornée               | Freinage sur la trajectoire nominale, sans contournement     | Non                         | Très élevée (1 kHz)                                               |
| <b>Notre approche</b>  | <b>Flow Matching</b>  | <b>Explicite : projection SDF-CBF</b>   | <b>Nuage filtré persistant + SDF robot</b> | <b>Correction vitesse articulaire à l'exécution</b>          | <b>Oui, via persistance</b> | <b>Élevée, à la fréquence de commande (validée au Chapitre 5)</b> |

son exécution aveugle aux obstacles au sein d'environnements réels et changeants demeure néanmoins sujette aux incertitudes perceptives. Quant aux méthodes qui incorporent des contraintes au cours de la génération, elles alourdissent le temps de calcul ou provoquent une dérive distributionnelle nuisible au succès de la tâche. Le Chapitre 4 détaille l'architecture découplée proposée en réponse à ces verrous : une perception persistante qui s'articule de manière synchrone avec un filtre réactif fondé sur une approximation par polynômes de Bernstein, pour améliorer en temps réel, sous des contraintes vérifiables localement, l'exécution sécurisée au cœur des objectifs de recherche.

## CHAPITRE 3 MOTIVATION DE RECHERCHE ET OBJECTIFS

Ce chapitre présente la motivation de la recherche proposée, en dérive le but général et les objectifs spécifiques, puis délimite la portée et les exclusions qui ont encadré le développement du système.

### 3.1 Positionnement de recherche et motivation

Les politiques génératives, portées par les modèles de diffusion puis par le Flow Matching, ont démontré ces dernières années leur capacité à apprendre des tâches de manipulation à partir d'un nombre restreint de démonstrations. Elles capturent la multimodalité des gestes experts et, dans le cas du Flow Matching, s'échantillonnent en quelques itérations d'intégration déterministes à échantillon initial fixé, compatibles avec une replanification en boucle fermée.

Malgré ces progrès, ces politiques restent le reflet de leurs données : comme elles sont entraînées sur des démonstrations sans obstacle, elles n'offrent aucune garantie face à un environnement qui n'a pas été vu à l'entraînement. La revue de littérature montre que les réponses existantes à cette limite se répartissent en trois familles, dont les deux premières portent un coût rédhibitoire pour l'application visée. La première contraint le processus de génération lui-même, au prix d'une dérive distributionnelle et d'un surcoût de calcul à chaque pas d'échantillonnage. La deuxième enseigne l'évitement au modèle : entraîner une politique générative à éviter les collisions est possible, mais exige un effort considérable, qu'il s'agisse de collecter des démonstrations enrichies d'obstacles, à recommencer pour chaque famille de scènes, ou de faire corriger les trajectoires défaillantes par un opérateur au déploiement, comme le propose FlowCorrect [113]. La troisième famille filtre l'exécution plutôt que la génération : les fonctions barrières de contrôle y apportent des garanties formelles d'invariance, mais leur emploi aux côtés de politiques génératives demeure le plus souvent restreint à l'effecteur seul ou à des obstacles décrits par des primitives géométriques connues, car leur déploiement bute en pratique sur une question préalable : disposer d'une représentation de distance du corps complet du robot, interrogeable avec son gradient à la fréquence de commande.

Ce mémoire se positionne à cette intersection : découpler entièrement la génération de la sécurité, en plaçant celle-ci en aval du modèle plutôt qu'en son sein. Une politique de Flow Matching pré-entraînée, jamais réentraînée, fournit l'intention ; un filtre de sécurité réactif,



fondé sur des fonctions barrières évaluées par un champ de distance signée appris du corps complet du robot et alimenté par une carte persistante de la scène, corrige à la fréquence de commande la composante de vitesse qui menace la sécurité. L’objectif est de conserver simultanément les deux propriétés que les approches existantes échangent l’une contre l’autre : l’intention de la trajectoire nominale et l’absence de collision dans des environnements non vus.

### 3.2 Objectifs de recherche

Le but général de cette recherche est de rendre exécutables, en toute sécurité et sans réentraînement, les trajectoires produites par une politique générative dans des environnements quasi statiques encombrés que celle-ci n’a jamais observés. La revue de littérature ayant montré qu’un filtre réactif mal conçu peut dégrader la réussite de la tâche en poussant l’exécution hors de la distribution d’entraînement [101], l’évaluation porte sur le compromis complet entre sécurité, réussite de la tâche et budget de calcul, et non sur la seule réduction des collisions. Les objectifs suivants soutiennent ce but.

#### 3.2.1 Objectif principal

Concevoir, implémenter et valider en simulation une architecture découplée qui exécute en temps réel et sans collision les trajectoires produites par une politique générative de *Flow Matching* gelée, dans des environnements quasi statiques encombrés non vus à l’entraînement, sans réentraînement du modèle et sans dégradation rédhibitoire de la réussite de la tâche.

#### 3.2.2 Objectifs spécifiques

- **SO1** : Implémenter une chaîne de perception qui détecte, segmente et suit la cible dans le flux RGB-D, et qui maintient une carte persistante des obstacles robuste aux occlusions et aux pertes de détection.
- **SO2** : Constituer un jeu de démonstrations réelles et simulées pour deux tâches de manipulation, et le prétraiter en données d’entraînement conditionnées sur la perception.
- **SO3** : Entraîner par *Flow Matching* un modèle de génération de trajectoires conditionné sur ces données, et convertir ses sorties en trajectoires articulaires exécutables par le robot.
- **SO4** : Intégrer un bouclier de sécurité fondé sur des fonctions barrières de contrôle et un champ de distance signée du corps complet du robot, corrigeant la commande en

temps réel pour éviter les collisions avec l’environnement.

- **SO5** : Valider le système complet en simulation sur les deux tâches de manipulation retenues, dans différents environnements quasi statiques non vus à l’entraînement, en comparaison appariée avec et sans filtre de sécurité, selon les critères de validation du Tableau 3.1.
- **SO6** : Évaluer le transfert de l’architecture à un second manipulateur à sept degrés de liberté, du Franka Emika Panda au UFactory xArm7, seuls la cinématique et les champs de distance par lien étant adaptés.

S’y ajoute une exigence transversale de temps réel : la chaîne complète doit tenir la fréquence de commande visée, le filtre de sécurité résolvant sa projection à la cadence de la boucle de commande, complété d’une couche de freinage à plus haute cadence qui re-certifie la commande maintenue entre deux résolutions, pendant que la génération se poursuit en parallèle. Le Tableau 3.1 rend ces objectifs mesurables en associant à l’évaluation de SO5 ses métriques, sa condition de comparaison et son critère d’acceptation ; les valeurs mesurées sont rapportées en regard de ces critères au Chapitre 5.

Tableau 3.1 Critères de validation associés aux objectifs de recherche.

| Critère                  | Métriques                                                                                                      | Comparaison                                     | Critère d’acceptation                                                                                                                                                 |
|--------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sécurité                 | Collisions (contact au maillage exact), distance minimale de séparation, durée cumulée des violations de marge | Filtre actif contre inactif, scénarios appariés | Aucune collision avec filtre dans les scénarios où le nominal en produit ; réduction de la durée de violation                                                         |
| Préservation de la tâche | Taux de réussite ; norme des interventions ; déviation de la trajectoire exécutée vis-à-vis de la nominale     | Filtre actif contre inactif, scénarios appariés | Dégradation du taux de réussite d’au plus 10 points de pourcentage en l’absence d’obstacle interférent ; intervention strictement nulle hors de la bande d’activation |
| Persistance              | Rappel des points d’obstacle sous occlusion ; issue de sécurité de l’épisode                                   | Carte persistante contre perception instantanée | Contrainte d’évitement maintenue pendant toute l’occlusion, sans perte de l’obstacle                                                                                  |
| Temps réel               | Temps de calcul par sous-système (médiane, p95) ; fraction de cycles au-delà de la période                     | Budget de période par module                    | Médiane sous la période de commande ; dépassements de queue quantifiés et couverts par l’analyse de marge du Chapitre 4                                               |

### 3.3 Portée et exclusions de la recherche

Compte tenu des contraintes d'un mémoire de maîtrise, la portée du projet a été délibérément limitée. Les éléments suivants sont exclus de la conception et de l'évaluation du système :

- **Déploiement sur robot réel** : le système complet est validé en simulation. Les démonstrations d'entraînement proviennent en partie du robot réel, mais l'exécution en boucle fermée sur le matériel n'est pas évaluée.
- **Couverture des tâches** : la validation porte sur deux scénarios, une tâche d'alimentation assistée et une tâche de saisie et dépôt ; la généralisation à d'autres tâches n'est pas étudiée.
- **Configuration des capteurs** : le nombre et le placement des caméras sont fixés a priori pour chaque tâche (une caméra RGB-D montée au poignet pour l'alimentation assistée, une caméra RGB-D statique pour la saisie et dépôt) ; déterminer la configuration de capteurs minimale pour la tâche n'est pas un objectif.
- **Dimensionnement du jeu de données** : le nombre de démonstrations nécessaires à l'entraînement du modèle génératif n'est pas étudié ; le jeu collecté est celui jugé suffisant pour les tâches retenues.
- **Obstacles dynamiques** : la garantie de sécurité visée est celle de l'environnement quasi statique, au sens suivant : les obstacles peuvent se déplacer, mais lentement devant le cycle de mise à jour de leur représentation, de sorte que la scène peut être traitée comme fixe entre deux mises à jour et que le déplacement d'un obstacle sur un cycle demeure petit devant la marge de sécurité ; les cadences et la marge qui quantifient ce régime sont établies au Chapitre 4. Les mécanismes de robustification face à des obstacles lentement mobiles sont évalués empiriquement, mais aucune garantie formelle n'est revendiquée à leur égard.
- **Interaction humaine** : bien que la tâche d'alimentation soit motivée par l'assistance à la personne, aucune étude utilisateur ni modélisation de l'humain (visage, bouche, préférences alimentaires) n'est réalisée.
- **Type de robot traité** : l'étude porte sur les manipulateurs sériels à base fixe et à sept degrés de liberté, instanciés ici par le Franka Emika Panda et le UFactory xArm7, dont la redondance cinématique est exploitée par la cinématique inverse et par le filtre de sécurité. Les bases mobiles et les autres morphologies ne sont pas considérées.
- **Propriétés des objets** : la cible et les obstacles sont supposés rigides et opaques. Les objets déformables, ainsi que les matériaux transparents ou fortement réfléchissants qui mettent en défaut le capteur de profondeur, sortent du cadre étudié.

## CHAPITRE 4 DESIGN DU SYSTÈME

Ce chapitre présente la conception et la mise en œuvre du système de génération de trajectoires sans collision. Le travail suit une approche expérimentale guidée par l’implémentation, caractérisée par un développement itératif, une architecture modulaire et une évaluation empirique. Le système est décrit à travers cinq blocs de conception directement alignés sur les objectifs spécifiques de la Section 3.2 :

- la plateforme expérimentale et l’architecture d’ensemble qui accueillent tous les modules (fondation du déploiement de SO5) ;
- la chaîne de perception sémantique et la carte persistante de la scène (SO1) ;
- l’acquisition des démonstrations et leur prétraitement en données d’entraînement (SO2) ;
- le modèle génératif de trajectoires par Flow Matching et la chaîne de conversion et d’exécution de ses sorties (SO3, SO6) ;
- le filtre de sécurité réactif SDF-CBF (SO4).

Cet ordre suit le flux des données, de la perception à la commande : chaque bloc consomme la sortie du précédent, et le chapitre se conclut par la portée des garanties du filtre. La validation expérimentale du système, phase d’évaluation de SO5 et SO6, fait l’objet du Chapitre 5.

### 4.1 Plateforme Expérimentale Et Architecture Globale

Cette section porte sur la fondation du système : la plateforme expérimentale et l’architecture d’ensemble sur lesquelles tous les modules (SO1 à SO4) sont implantés et testés. Elle couvre les données d’entrée et de sortie de chacun des modules utilisés pour réaliser l’ensemble des objectifs listés au Chapitre 3.

#### 4.1.1 Plateforme matérielle et logicielle

Le système est développé et validé sur un manipulateur Franka Emika Panda à sept degrés de liberté, dont la redondance cinématique est exploitée tant par la cinématique inverse que par le filtre de sécurité ; la méthode se transpose à tout manipulateur redondant décrit par un modèle URDF, moyennant l’adaptation des éléments propres à la plateforme : le modèle URDF lui-même, les SDF par lien à réentraîner sur ses maillages, et l’interface du contrôleur de vitesse. Pour la tâche d’alimentation, le robot porte une fourchette fixée à sa pince, intégrée par composition de son modèle URDF, et une caméra RGB-D Intel RealSense D415 ( $640 \times 480$  pixels) est montée au poignet, répliquant le montage utilisé pour la collecte

des démonstrations (Section 4.4.1).

L’ensemble logiciel repose sur ROS 1 Noetic sous Ubuntu 20.04, la scène étant simulée dans Gazebo 11. Bien que ROS Noetic soit distribué avec Python 3.8, les modèles de fondation de vision employés, SAM 3 et SAM 2 (Section 4.2.1), exigent une version plus récente : l’ensemble du système s’exécute donc dans un unique environnement virtuel Python 3.10.13, activé avant le lancement des nœuds ROS. Tous les modules y sont implémentés en Python, à l’exception de la lecture d’état du robot et de la cinématique inverse, écrites en C++ avec la bibliothèque *Pinocchio* [116] afin d’en minimiser la latence. Les composantes d’apprentissage (modèle de Flow Matching, SDF de Bernstein, bouclier CBF) s’appuient sur *PyTorch* 2.10 avec CUDA 13.0 et s’exécutent sur un unique GPU NVIDIA RTX 3090 (24 Go), partagé par l’ensemble des modules ; le serveur de segmentation SAM 3, qui charge un modèle de fondation volumineux, demeure un nœud dédié exposé au reste du système comme un service ROS et invoqué à la demande (Section 4.2.2). Les démonstrations simulées sont collectées avec la librairie *ManiSkill 3* (Section 4.4.1) et la visualisation s’effectue dans RViz.

#### 4.1.2 Architecture modulaire du système

La Figure 4.1 illustre l’architecture modulaire du système et le trajet des données, des images RGB-D jusqu’à la commande de vitesse articulaire. Le flux se sépare en deux chemins dès la perception. Le premier conditionne la génération : la cible, détectée par SAM 3 puis suivie par SAM 2, est projetée en un nuage de points 3D, fusionnée au maillage de l’outil, et le nuage résultant conditionne le modèle de *Flow Matching*, qui replanifie la trajectoire nominale en boucle fermée. Le second garantit la sécurité : les pixels restants forment le nuage d’obstacles qui, nettoyé puis accumulé dans la carte persistante, alimente le bouclier CBF ; celui-ci corrige à la fréquence de commande la trajectoire nominale interpolée par l’exécuteur, avant de l’envoyer au contrôleur de vitesse du robot. Deux liaisons secondaires ferment le système : le nœud de saisie soude la cible à l’outil au moment du contact et publie le nuage saisi, que le bouclier protège dès lors comme un lien supplémentaire du robot, et l’état articulaire mesuré revient à tous les modules par le bus de retour de droite, la boucle physique se refermant à travers la simulation.

Le système comportant de nombreux mécanismes secondaires, le Tableau 4.1 en clarifie d’emblée la hiérarchie : il distingue les mécanismes qui portent directement les objectifs de recherche, l’infrastructure qui les rend exécutables, et les heuristiques de robustesse, désactivables sans invalider le certificat de sécurité. Cette hiérarchie guide la lecture du chapitre : chaque section précise, pour les mécanismes qu’elle introduit, s’ils relèvent du cœur de la contribution ou de son enrobage.

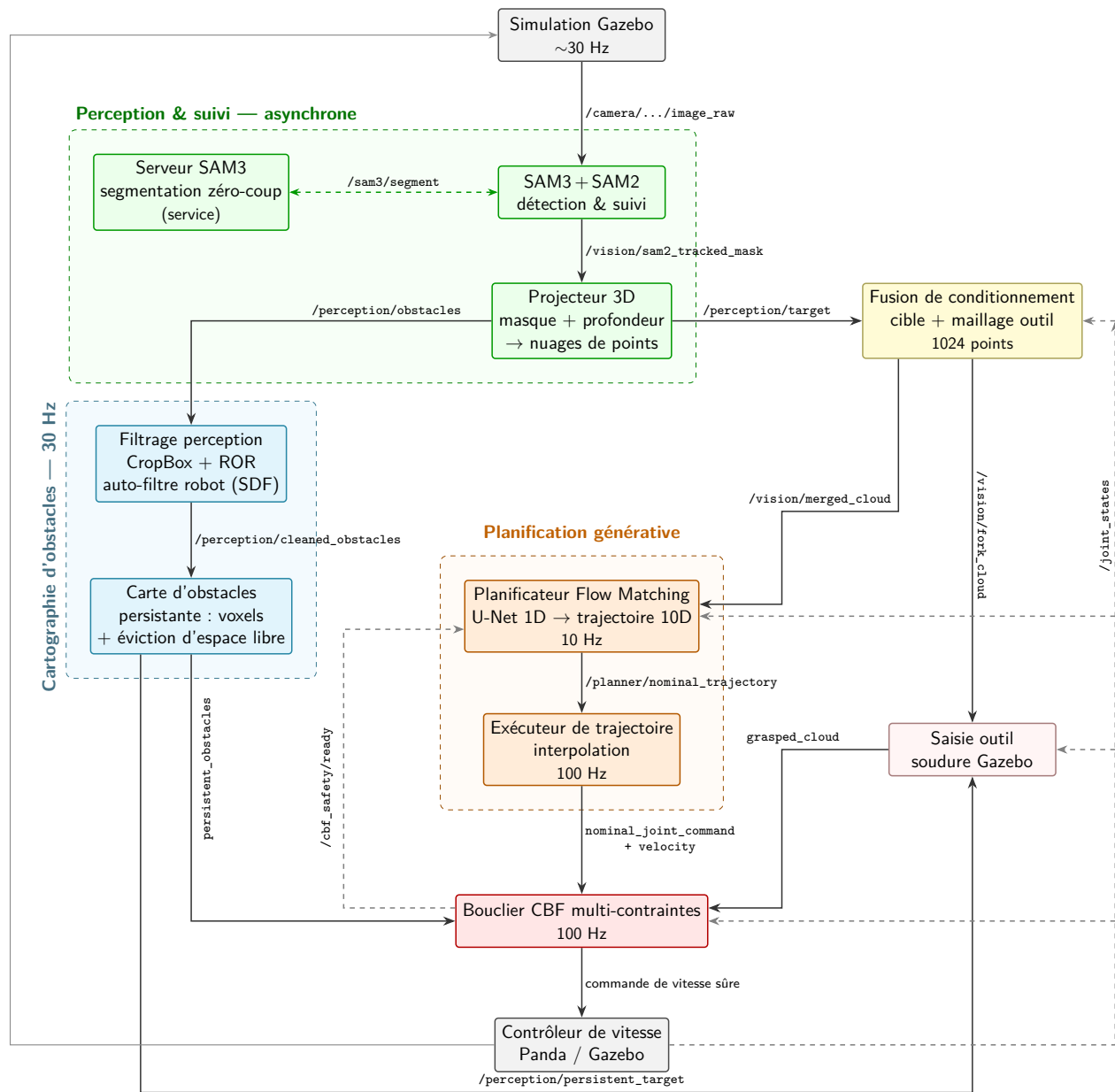


Figure 4.1 Architecture globale du système asynchrone.

Tableau 4.1 Hiérarchie des mécanismes du système selon leur rôle vis-à-vis des objectifs de recherche.

| Rôle                                                      | Mécanismes                                                                                                                                                                                                                                                                                   | Critère servi                |
|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| Essentiels à la contribution                              | Politique Flow Matching gelée ; carte persistante d’obstacles ; SDF de Bernstein corps-complet raffiné et son gradient analytique ; barrières par lien soft-min (paire valeur-gradient cohérente) et projection multi-contraintes ; métrique de tâche $\mathbf{W}$ ; marge $d_{\text{safe}}$ | Sécurité, tâche, persistance |
| Infrastructure habilitante                                | Chaîne SAM 3/SAM 2 et rétroprojection ; cinématique inverse et réajustement temporel ; exécuter à horloge de progression ; graphe CUDA et flux séparés ; interface de vitesse articulaire                                                                                                    | Temps réel                   |
| Heuristiques de robustesse (sans effet sur le certificat) | Dégagement postural ; resserrement temporel $\eta$ ; atténuation de réparation $g_{\min}$ ; réglages de filtres                                                                                                                                                                              | Progression de la tâche      |

Cette architecture est délibérément asynchrone : chaque groupe de la figure tourne à sa propre cadence et communique par messages, sans jamais bloquer les groupes en aval. La perception et la planification sont ainsi découplées de la boucle de commande : le bouclier CBF n’attend jamais le planificateur, et un nuage de conditionnement périmé dégrade la trajectoire nominale sans jamais suspendre la commande à 100 Hz. La hiérarchie des fréquences est un choix de conception qui reflète la dynamique de chaque information : le filtre de sécurité et la commande, seuls garants de la réactivité, tournent à 100 Hz, doublés d’un frein réflexe à  $\sim 1$  kHz qui re-certifie la commande maintenue entre deux résolutions (Section 4.7.5) ; la cartographie d’obstacles et la sélection des points critiques à 30 Hz ; la replanification générative à 10 Hz ; la perception sémantique, la plus coûteuse, s’exécute de manière asynchrone à la cadence que lui laisse le GPU partagé. La vérification expérimentale de ce budget temporel fait l’objet du Chapitre 5.

## 4.2 Perception Sémantique et Reconstruction 3D

Cette section porte sur le premier volet de SO1 : la chaîne de perception qui, à partir du flux RGB-D de la caméra au poignet, produit en temps réel les deux nuages de points consommés par le système, celui de la cible, qui conditionne le modèle génératif (SO3), et celui des obstacles, qui alimente le filtre de sécurité (SO4).

### 4.2.1 Extraction Sémantique

La détection des objets d'intérêt repose sur le modèle de fondation *Segment Anything Model 3* (SAM 3) de Meta [56], qui produit, à partir d'une image RGB et d'une requête textuelle, la boîte englobante, la segmentation au niveau du pixel (le masque) et un score de confiance de détection. Le masque étant disponible directement en sortie du modèle, un modèle plus léger peut ensuite assurer le suivi des objets détectés, ce qui réduit fortement le temps d'inférence du suivi temporel (Section 4.2.2). Entraîné sur plus de quatre millions de concepts textuels et 38 millions de phrases synthétiques, SAM 3 détecte une variété quasi illimitée d'objets. La Figure 4.2 présente un exemple de masque obtenu pour la requête *Pink Block*.



Figure 4.2 Exemple de masque obtenu par le modèle de segmentation SAM3 pour le prompt *Pink Block*

### 4.2.2 Suivi temporel de la cible

Une évaluation de SAM 3 à chaque image serait incompatible avec le budget temps réel, le modèle de fondation exigeant plusieurs centaines de millisecondes par inférence. Le suivi est donc délégué à *Segment Anything Model 2* (SAM 2) [55], un modèle de segmentation vidéo plus léger doté d'une mémoire d'attention : initialisé avec le masque produit par SAM 3, il propage ce masque d'image en image à une cadence de 5 Hz, sur des images réduites à 512 pixels de côté au maximum, la fraîcheur du masque important davantage que sa finesse.



Lorsque le suivi se dégrade (masque vide ou dégénéré), une nouvelle détection SAM 3 est demandée pour réinitialiser le suiveur ; ces requêtes sont espacées d’une période de repli qui s’allonge après des échecs répétés (de 8 à 30 s), afin qu’une cible durablement absente ne monopolise pas le budget GPU partagé avec le reste du système.

### 4.2.3 Projection RGB-D vers l’espace 3D

Le masque suivi est combiné à la carte de profondeur alignée pour produire les nuages de points, par la rétroprojection sténopé déjà employée au prétraitement (Section 4.4.2) : chaque pixel de profondeur valide est projeté dans le repère de la caméra à l’aide des paramètres intrinsèques, puis exprimé dans le repère de base par la transformation extrinsèque, obtenue de la cinématique directe du robot puisque la caméra est montée au poignet. Deux nuages sont produits à chaque image : les pixels du masque forment le nuage de la cible, les autres celui des obstacles. Le coût des traitements en aval est borné par sous-échantillonnage aléatoire, à 2000 points pour la cible et 5000 points pour les obstacles.

### 4.2.4 Séparation cible, obstacles et outil visible

La caméra étant montée au poignet, l’outil occupe une part importante du champ de vision : sans traitement, ses pixels seraient projetés dans le nuage d’obstacles et le filtre de sécurité freinerait le robot contre son propre outil. L’outil est donc retiré par deux mécanismes complémentaires. En 2D, un masque est obtenu de SAM 3, dilaté de 5 pixels puis soustrait de l’image avant projection. En 3D, un filtre fondé sur le SDF du maillage de l’outil supprime les fuites résiduelles que le masque 2D ne capture pas : tout point à moins de 5 mm de la surface du maillage, positionné par la cinématique directe, est écarté. Cette combinaison élimine les points de l’outil sans effacer les obstacles réels situés juste au-delà de cette coquille ; les auto-projections du corps du robot lui-même sont écartées séparément, au nettoyage spatial de la Section 4.2.5.

### 4.2.5 Nettoyage spatial du nuage d’obstacles

Le nuage d’obstacles brut hérite des artefacts typiques des capteurs de profondeur, points volants aux discontinuités de profondeur et retours situés hors de la zone d’intérêt, auxquels s’ajoutent les projections du corps du robot lui-même dans le champ de la caméra montée au poignet. Trois opérations le nettoient. Une boîte de recadrage (*CropBox*) restreint d’abord le nuage au volume de travail du robot ; un filtre de voisinage (*Radius Outlier Removal*) rejette ensuite tout point n’ayant pas un nombre minimal de voisins dans un rayon donné,

signature des points isolés non associés à une surface réelle ; un auto-filtre écarte enfin les points appartenant au robot. Ce dernier réutilise le modèle de distance signée du corps complet du robot appris hors ligne (Section 4.7.1), celui-là même qui alimente le filtre de sécurité : chaque point est évalué par ce SDF à la configuration articulaire courante, et tout point dont la distance à la surface d'un lien est négative ou inférieure à 5 mm est retiré. Sans ce retrait, les projections du bras seraient accumulées dans la carte persistante et le bouclier CBF, traitant le robot comme un obstacle, le freinerait contre son propre corps. Le nuage nettoyé constitue l'entrée de la carte persistante de la section suivante.

### 4.3 Cartes Persistantes de la scène

Cette section porte sur le second volet de SO1 : doter la perception d'une mémoire spatiale, la carte persistante, qui rend la représentation des obstacles robuste aux occlusions et aux pertes de détection.

#### 4.3.1 Motivation : robustesse aux pertes de détection et occlusion

La chaîne de perception précédente est instantanée : elle ne décrit que ce que la caméra voit à l'image courante. Or le champ de vision de la caméra au poignet est étroit et se déplace avec le bras : un obstacle contourné sort du champ précisément au moment où le robot passe à côté, et l'approche finale plonge la caméra vers la cible, masquant le reste de la scène. Une perception sans mémoire ferait ainsi disparaître les contraintes de sécurité au moment où elles importent le plus, et les pertes de détection transitoires (occlusion par le bras, échec ponctuel du suivi) auraient le même effet. La carte persistante conserve donc en mémoire les surfaces observées, et c'est elle, plutôt que la vue instantanée, qui alimente le filtre de sécurité.

La même politique de mémoire s'applique à la cible : en cas de perte du suivi, le dernier nuage de cible valide est conservé et le conditionnement du modèle génératif est gelé plutôt qu'annulé, la trajectoire nominale demeurant inchangée pendant l'occlusion au lieu de provoquer un arrêt du système ; une temporisation invalide cette mémoire si la perte se prolonge au-delà de quelques secondes.

#### 4.3.2 Génération des cartes persistantes

La carte est une grille de voxels creuse de pas 1 cm, résolution dictée par la précision requise à la pointe de l'outil. À chaque image, le nuage d'obstacles nettoyé y est voxelisé, la voxelisation fusionnant les doublons et bornant le coût des traitements en aval, puis chaque voxel observé

est inscrit dans la carte, plafonnée à  $10^5$  voxels. Les points appartenant à la cible, identifiés par le masque suivi, en sont exclus : la cible est accumulée dans une carte distincte, maintenue compacte par éviction des voxels s'éloignant trop de son centroïde courant, de sorte qu'elle n'apparaisse jamais comme un obstacle à sa propre saisie. La carte d'obstacles est publiée à 30 Hz vers le filtre de sécurité.

### 4.3.3 Mise à jour des cartes persistantes

Deux règles gouvernent la vie d'un voxel. Hors du champ de vision courant, il persiste indéfiniment : aucune éviction par vieillissement n'est appliquée, et c'est précisément cette persistance qui protège le robot des obstacles qu'il ne voit plus. Dans le champ de vision, une éviction par espace libre est calculée sur GPU à chaque image : chaque voxel est reprojeté dans l'image de profondeur courante et, si la profondeur mesurée dépasse la sienne d'une marge  $\delta_{\text{libre}}$  (fixée à 2 cm, soit deux voxels), la caméra voit *au-delà* de l'emplacement du voxel, qui ne peut donc plus être occupé et est évincé immédiatement. Un obstacle déplacé ou retiré disparaît ainsi de la carte dès qu'il repasse dans le champ de vision, sans délai de péremption ; lorsque la capacité maximale est atteinte, les voxels les plus anciens sont écartés en priorité.

## 4.4 Acquisition des données et représentation des démonstrations

Cette section présente le protocole utilisé pour collecter les démonstrations qui servent à entraîner le modèle génératif, puis les étapes de prétraitement qui transforment ces enregistrements bruts en données d'entraînement.

### 4.4.1 Protocole de collecte des démonstrations

Les démonstrations proviennent de deux sources complémentaires : le robot et les capteurs réels d'une part, la simulation d'autre part, cette dernière reposant sur la librairie *ManiSkill 3*, retenue pour sa facilité d'utilisation et son moteur physique à l'état de l'art fondé sur SAPIEN [117]. Les deux sources reproduisent des environnements quasi identiques, de sorte que les données y sont récoltées de la même manière.

## Échantillonnage des configurations

Pour garantir un protocole reproductible, l'échantillonnage des configurations est défini dans deux repères imbriqués plutôt que de façon qualitative : le repère robot  $\mathcal{F}_b$  (base du manipu-

lateur, fixe) et un repère assiette  $\mathcal{F}_p$  centré sur l'assiette. L'assiette est placée successivement aux 9 positions d'une grille  $3 \times 3$  exprimée directement dans  $\mathcal{F}_b$  : centrée à  $(0,40, 0)$  m, de pas 0,08 m selon  $x_b$  et 0,20 m selon  $y_b$ . Pour chacune, la cible occupe 5 positions disposées en quinconce dans  $\mathcal{F}_p$  (demi-écart de 0,06 m), chacune assortie d'une orientation aléatoire. Cette grille produit  $9 \times 5 = 45$  configurations, chacune réalisée une fois. La Figure 4.3 illustre cet échantillonnage. À chaque configuration, l'outil descend verticalement vers la cible (axe  $-z$  de  $\mathcal{F}_b$ ) en maintenant les dents verticales et alignées sur l'axe principal de la cible.

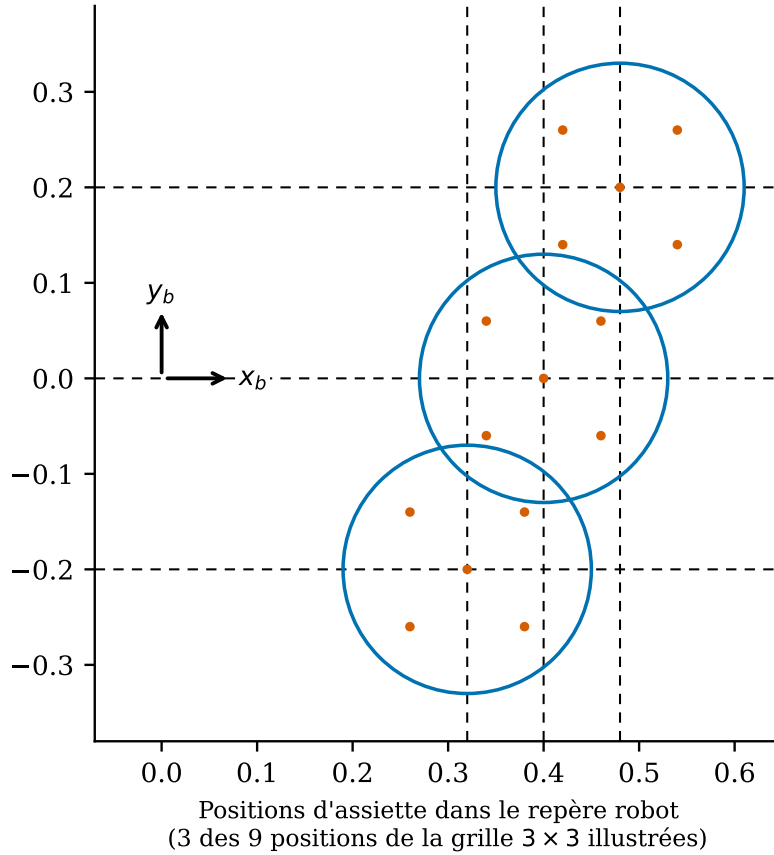


Figure 4.3 Protocole d'échantillonnage des cibles, vu de dessus dans le repère de base du robot  $\mathcal{F}_b$ . Les assiettes (26 cm de diamètre) se chevauchant, seules trois des neuf positions de la grille  $3 \times 3$  sont illustrées (supérieur droit, centre, inférieur gauche), le centre de chacune étant marqué par une croix noire. Autour de chaque centre, les 5 points indiquent les positions de cible en quinconce.

### Démonstrations réelles par guidage kinesthésique

Chaque démonstration réelle est fournie par guidage kinesthésique : le robot, amené dans la configuration articulaire initiale  $\mathbf{q}_0 = [0, -0,126, 0, -2,193, 0, 2,065, 0,786]$  rad puis placé

en mode de guidage manuel, est conduit par l’opérateur depuis cette posture jusqu’à la cible selon le schéma d’approche décrit ci-dessus. Pendant l’exécution, la chaîne d’acquisition échantillonne à 10 Hz la pose du TCP dans le repère de base (position cartésienne et orientation en quaternion), sa vitesse linéaire et angulaire, ainsi que le flux RGB-D de la caméra montée à l’effecteur, chaque échantillon étant horodaté depuis le début de la trajectoire. La mise en œuvre de cette chaîne (deux nœuds ROS dédiés), le format d’enregistrement et l’organisation des fichiers sont détaillés à l’Annexe A.

## Démonstrations simulées de la tâche d’alimentation

En simulation, l’environnement réplique la scène réelle : même configuration articulaire initiale  $\mathbf{q}_0$ , même montage de la caméra à l’effecteur et même cadence d’enregistrement de 10 Hz, le robot y étant équipé de la fourchette par composition de son modèle URDF. Les démonstrations n’y sont toutefois pas fournies par guidage kinesthésique, mais par une politique experte scriptée qui reproduit le geste de l’opérateur en trois phases : approche alignée sur l’axe principal de la cible en conservant les dents de la fourchette à la verticale, redressement du poignet qui ramène les dents à l’horizontale, puis transport vers une pose de livraison fixe ; une fois piquée, la cible est rigidement attachée à la fourchette. L’échantillonnage des configurations y est continu plutôt que discret : la position de la cible est tirée uniformément dans un rectangle du plan de travail délimité par le champ de vision de la caméra, son orientation uniformément sur  $[-\pi, \pi]$ , et sa géométrie parmi sept aliments maillés retenus pour la variété de leurs formes ; la randomisation étend en outre le protocole réel selon la hauteur, la cible étant surélevée sur un piédestal dont la hauteur est tirée uniformément entre 4 et 10 cm. Quinze démonstrations simulées complètent ainsi les 45 démonstrations réelles de la tâche d’alimentation ; le détail de la politique experte et des plages de randomisation est donné à l’Annexe A. La Figure 4.4 illustre ces trois phases.

## Jeu de données de saisie et dépôt

Le jeu de données de saisie et dépôt (*pick-and-place*), entièrement simulé, suit le même principe avec le Panda standard. Le cube y est un corps dynamique réellement saisi par la pince, sans attachement artificiel, et la politique experte enchaîne descente, saisie, transport le long d’un arc dont les pentes de départ et d’arrivée sont prescrites à  $60^\circ$  et  $-60^\circ$  par rapport à la table, dépôt dans la boîte puis retrait. Les positions du cube et de la boîte de dépôt sont tirées uniformément dans leurs zones respectives à chaque épisode, la randomisation de la boîte évitant que le modèle n’apprenne une position de dépôt fixe ; 40 trajectoires sont ainsi collectées. Le détail des politiques expertes et des plages de randomisation est donné à

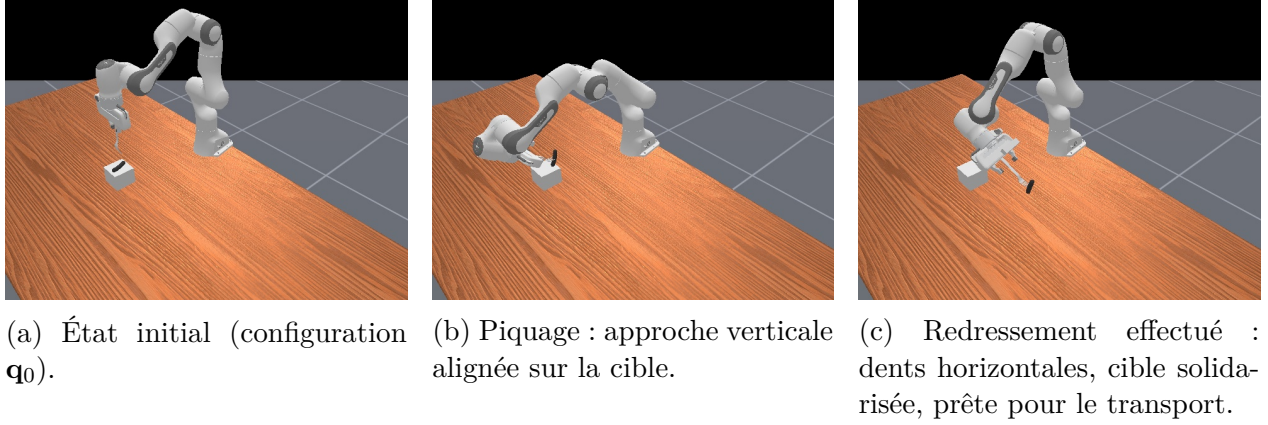


Figure 4.4 Trois phases de la politique experte simulée pour la tâche d'alimentation.

l'Annexe A. La Figure 4.5 illustre trois de ces phases (descente, transport, dépôt).

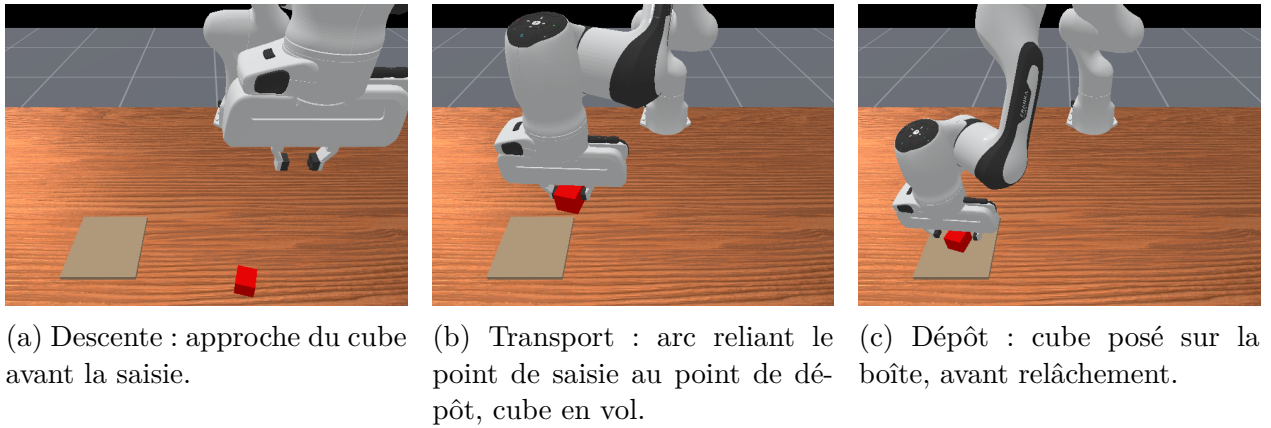


Figure 4.5 Trois phases de la politique experte simulée pour la tâche de saisie et dépôt.

#### 4.4.2 Prétraitement des données

Les enregistrements bruts issus du protocole de la Section 4.4.1 contiennent les poses du TCP et les flux RGB-D, mais pas le nuage de points conditionnel attendu par le modèle génératif de la Section 4.5. Un prétraitement hors ligne les transforme donc en données d'entraînement en trois étapes : la détection de la cible dans la première image de chaque trajectoire, la génération du nuage de points conditionnel à chaque pas, puis la mise à jour du fichier de trajectoire. Réaliser ces étapes hors ligne évite de recalculer la perception à chaque époque d'entraînement et permet de valider visuellement les nuages générés avant celui-ci.

## Détection de la cible par segmentation

La première étape consiste à détecter la cible dans la première image de la caméra à l'effecteur de chaque trajectoire. Cette détection est réalisée avec le modèle SAM 3 présenté à la Section 4.2.1 : l'image, préalablement convertie au format `.jpg` attendu par le modèle, lui est fournie accompagnée d'une requête textuelle décrivant la cible (*Pink block*, *Broccoli*, *Carrot slice*, *Chicken breast*, *Sushi*, *Nigiri*, *Sausage* ou *Potato* pour la tâche d'alimentation ; *Red Cube* et *Brown box* pour la tâche de saisie et dépôt), et le masque au score de confiance le plus élevé est retenu. La détection n'est effectuée que sur la première image en raison de la distance minimale de mesure du capteur de profondeur, de 31 cm selon le fabricant à une résolution de  $640 \times 480$  : l'approche amène rapidement la caméra sous cette distance, de sorte que seules les premières images garantissent une carte de profondeur valide de la cible.

## Génération du nuage de points conditionnel

Le modèle de Flow Matching décrit à la Section 4.5 étant conditionné sur les nuages de points de l'outil et de la cible, un nuage fusionné est généré pour chaque pas de chaque trajectoire. Cette génération s'appuie sur quatre éléments : le masque obtenu à l'étape précédente, la carte de profondeur initiale, la pose du TCP à chaque pas et le modèle CAO de l'outil.

Le nuage de la cible est d'abord reconstruit dans le repère de la caméra. Le masque est superposé à la carte de profondeur initiale afin de ne conserver que les pixels de la cible dotés d'une profondeur valide, puis chaque pixel  $(u, v)$  de profondeur  $Z$  est rétroprojeté par le modèle sténopé,  $X_c = Z(u - c_x)/f_x$ ,  $Y_c = Z(v - c_y)/f_y$ ,  $Z_c = Z$ , où  $f_x$ ,  $f_y$  sont les longueurs focales et  $c_x$ ,  $c_y$  les coordonnées du point principal, paramètres intrinsèques de la caméra D415 obtenus auprès du fabricant. Le nuage résultant est ensuite exprimé dans le repère de la base du robot par la relation (4.1), évaluée à la pose du TCP du premier pas, où  $\mathbf{T}_{cible}^b$  est la matrice de transformation entre la cible et la base du robot. Cette relation se déduit de la Figure 4.6, qui représente l'ensemble des repères du système :  $\mathcal{F}_b$  le repère de la base du robot,  $\mathcal{F}_c$  celui de la caméra à l'effecteur,  $\mathcal{F}_e$  celui du TCP et  $\mathcal{F}_o$  celui de l'outil utilisé.

$$\mathbf{T}_{cible}^b = \mathbf{T}_e^b \mathbf{T}_c^e \mathbf{T}_{cible}^c \quad (4.1)$$

Le nuage de l'outil est quant à lui obtenu à partir de son modèle CAO, dont la surface est échantillonnée avec la librairie `trimesh` (fonction `trimesh.sample.sample_surface`). Ce nuage, défini dans le repère de l'outil  $\mathcal{F}_o$ , est positionné dans le repère de base à chaque pas

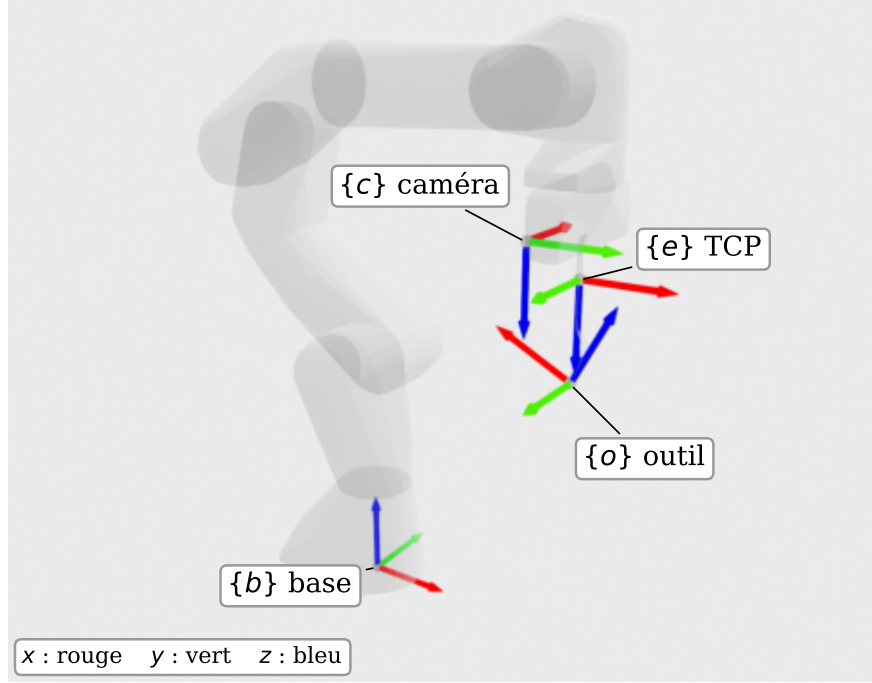


Figure 4.6 Représentation schématique des différents repères du système

par la relation (4.2), où  $\mathbf{T}_o^e$  est la transformation constante entre le TCP et l'outil.

$$\mathbf{T}_o^b = \mathbf{T}_e^b \mathbf{T}_o^e \quad (4.2)$$

À noter que  $\mathbf{T}_o^e = \mathbf{I}$  dans le cas où l'outil est le TCP, comme pour la tâche de saisie et dépôt. La carte de profondeur n'étant exploitable qu'au premier pas, la position de la cible aux pas suivants est reconstruite géométriquement plutôt que mesurée. Avant la saisie, la cible repose sur son support et son nuage demeure à sa position initiale. À partir de l'instant de saisie, elle est considérée rigidement solidaire de l'outil et son nuage suit le mouvement de celui-ci selon la relation (4.3),

$$\tilde{\mathbf{p}}_k = \mathbf{T}_{o,k}^b \left( \mathbf{T}_{o,g}^b \right)^{-1} \tilde{\mathbf{p}}_0, \quad k \geq g \quad (4.3)$$

où  $\tilde{\mathbf{p}}_0$  est un point de la cible en coordonnées homogènes à sa position initiale,  $\tilde{\mathbf{p}}_k$  ce même point au pas  $k$ , et  $g$  l'indice du pas de saisie. Cette reconstruction suppose une saisie ferme et sans glissement, hypothèse cohérente avec le protocole de collecte de la Section 4.4.1.

L'indice de saisie  $g$  est déterminé différemment selon la tâche. Pour la tâche d'alimentation, la saisie s'effectue par piquage : les dents de la fourchette pénètrent la cible, de sorte que la distance euclidienne entre la pointe de l'outil, positionnée par la relation (4.2), et le centroïde de la cible décroît durant l'approche et atteint son minimum à l'insertion maximale. Cette



distance est donc évaluée à chaque pas, lissée par un filtre gaussien d'écart-type de deux pas d'échantillonnage afin d'atténuer le bruit sur la pose et sur le centroïde, et l'indice de son minimum définit  $g$ .

Pour la tâche de saisie et dépôt, la saisie s'effectue par fermeture de la pince. Un état binaire de la pince, ouverte ou fermée, est plutôt dérivé à chaque pas des données enregistrées, à partir de la commande envoyée à la pince ou, à défaut, de l'ouverture moyenne des doigts comparée à un seuil de 3 cm. La première transition vers l'état fermé définit  $g$ . La relation (4.3) solidarise alors le cube au TCP jusqu'à la réouverture de la pince au dépôt, après quoi son nuage demeure à sa position de dépôt. La boîte de dépôt, qui n'est jamais saisie, conserve sa position initiale pendant toute la trajectoire.

Les nuages de l'outil et de la cible sont ensuite fusionnés, puis sous-échantillonnés à 1024 points par la stratégie d'échantillonnage du point le plus éloigné, qui préserve la couverture géométrique des deux objets. Pour la tâche de saisie et dépôt (*pick-and-place*), le cube à saisir et la boîte de dépôt sont détectés puis reconstruits par la même chaîne de segmentation et de rétroprojection ; le nuage fusionné compte également 1024 points, répartis entre la pince (40 %), le cube (20 %) et la boîte de dépôt (40 %), avec la même stratégie de sous-échantillonnage.

Le nuage fusionné est finalement centré, en translation seulement, sur l'origine du repère de l'outil. La position de l'outil est soustraite de chaque point, sans qu'aucune rotation ne soit appliquée, de sorte que les axes du nuage restent alignés sur ceux du repère de base  $\mathcal{F}_b$ . Ce centrage rend la représentation invariante à la position absolue de la scène dans l'espace de travail, ce qui favorise la généralisation aux différentes configurations du protocole, tout en préservant l'information d'orientation. Un changement de repère complet vers  $\mathcal{F}_o$ , rotation comprise, serait au contraire dégénéré : après la saisie, l'outil et la cible sont immobiles dans le repère de l'outil et l'observation deviendrait constante, en plus de masquer la direction de la gravité, nécessaire pour distinguer les différentes phases.

### Mise à jour des données de trajectoire

La dernière étape recopie le fichier JSON de chaque trajectoire dans le jeu de données prétraité et l'augmente de quatre éléments : le nom du fichier du nuage conditionnel associé à chaque pas ; la pose de l'outil dans le repère de base  $\mathcal{F}_b$  à chaque pas (position et orientation en quaternion), calculée par la relation (4.2) à partir de la pose du TCP enregistrée ; l'état binaire de la pince  $s_k \in \{0, 1\}$  à chaque pas, dérivé des données enregistrées comme décrit plus haut pour la tâche de saisie et dépôt, et constant à 1 pour la tâche d'alimentation, la pince y demeurant fermée sur la fourchette pendant toute la trajectoire ; et la transformation

constante  $\mathbf{T}_o^e$ , consignée une seule fois à la racine du fichier. La pose de l’outil et l’état binaire de la pince forment ensemble l’observation proprioceptive à dix dimensions du modèle de la Section 4.5 et définissent l’espace dans lequel les trajectoires sont générées. L’organisation des fichiers du jeu de données prétraité est détaillée à l’Annexe A.

## 4.5 Génération de trajectoire par Flow Matching

Cette section présente le modèle génératif du système : la formulation mathématique du Flow Matching conditionnel, l’architecture qui la réalise, puis la procédure d’entraînement et ses hyperparamètres. Le même modèle, avec les mêmes hyperparamètres, est employé pour les deux tâches ; seul le jeu de données d’entraînement change.

### 4.5.1 Formulation mathématique du Flow Matching conditionnel

Le formalisme du Flow Matching, présenté au Chapitre 2, transporte la gaussienne standard  $p_0$  vers la distribution cible des données  $p_1$  au moyen d’un champ de vitesses  $u_t$  dont le flot  $\psi_t$  obéit à l’équation différentielle ordinaire (4.4), avec la condition initiale  $\psi_0(x) = x$  :

$$\frac{d}{dt}\psi_t(x) = u_t(\psi_t(x)) \quad (4.4)$$

Générer un échantillon revient à intégrer numériquement cette équation de  $t = 0$  à  $t = 1$  depuis un tirage  $X_0 \sim p_0$ . Le chemin probabiliste retenu est l’interpolation linéaire du *Rectified Flow* [36],

$$X_t = (1 - t)X_0 + tX_1 \quad (4.5)$$

dont la vitesse conditionnelle vaut  $\dot{X}_t = X_1 - X_0$  ; l’objectif d’entraînement régresse directement cette vitesse par le réseau  $u_t^\theta$  de poids  $\theta$  :

$$\mathcal{L}(\theta) = \mathbb{E}_{t, X_0, (X_1, c)} \left\| u_t^\theta(X_t | c) - (X_1 - X_0) \right\|^2, \text{ où } t \sim \mathcal{U}[0, 1], X_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), (X_1, c) \sim \mathcal{D} \quad (4.6)$$

où  $\mathcal{D}$  est le jeu de démonstrations et  $c$  l’observation associée à chaque segment, décrite plus bas. Ce simple objectif conditionnel possède les mêmes gradients que la régression du champ marginal exact [33], et les chemins quasi rectilignes qu’induit l’interpolation linéaire autorisent une intégration précise en une dizaine de pas [34], l’argument décisif face aux modèles de diffusion pour la replanification en temps réel (Chapitre 2).

Dans le cadre de ce travail, un échantillon  $X_1$  est un segment de trajectoire de l’outil sur un horizon de  $H = 16$  pas,  $X_1 = [\mathbf{a}_1, \dots, \mathbf{a}_H] \in \mathbb{R}^{H \times 10}$ . Chaque action  $\mathbf{a}_k \in \mathbb{R}^{10}$  regroupe

la position de l’outil (trois composantes), son orientation (six composantes) et l’état binaire de la pince (une composante), tels que définis à la Section 4.4.2. L’orientation adopte la représentation 6D continue de Zhou et al. [118], soit les deux premières colonnes de la matrice de rotation : contrairement au quaternion, cette représentation est exempte de discontinuités, ce qui en facilite la régression par un réseau de neurones ; le quaternion consigné dans les données prétraitées  $y$  est converti au chargement. Le champ de vitesses est enfin conditionné sur l’observation  $c$  de la scène,  $u_t^\theta(X_t | c)$ , de sorte que le modèle apprend la distribution conditionnelle des segments experts  $p_1(X_1 | c)$  plutôt que leur distribution marginale.

#### 4.5.2 Architecture du modèle

Le modèle réalise le champ de vitesses conditionnel  $u_t^\theta(X_t | c)$  : il reçoit l’observation  $c$ , le segment intermédiaire  $X_t$  et le temps d’écoulement  $t$ , et produit une vitesse à dix dimensions pour chacune des  $H$  actions du segment.

#### Encodage de l’observation et du temps d’écoulement

L’observation regroupe le nuage de points conditionnel de la Section 4.4.2 ( $1024 \times 3$ ) et la proprioception des deux derniers pas, soit deux vecteurs à dix dimensions rassemblant la pose de l’outil et l’état binaire de la pince. Conformément au centrage du prétraitement, toutes les positions, celles du nuage comme celles des poses observées et générées, sont exprimées relativement à la position courante de l’outil, les orientations demeurant dans le repère de base. Avant d’entrer dans le réseau, les positions et l’état de la pince sont normalisés linéairement dans  $[-1, 1]$  à partir des minima et maxima du jeu d’entraînement ; la représentation 6D de l’orientation, déjà bornée dans  $[-1, 1]$ , est laissée telle quelle.

Deux encodeurs transforment cette observation en un vecteur de conditionnement  $c \in \mathbb{R}^{128}$  : le nuage de points traverse l’encodeur léger de *3D Diffusion Policy* [29], un perceptron partagé entre les points suivi d’un max-pooling qui rend le descripteur invariant à leur ordre, et la proprioception un perceptron dédié. Le temps d’écoulement  $t$ , dont le champ de vitesses (4.6) dépend explicitement, est quant à lui fourni au réseau par le plongement sinusoïdal multi-échelle introduit avec l’architecture Transformer [119], un scalaire brut constituant un signal trop pauvre pour moduler l’ensemble du réseau ; raffiné par un perceptron, ce plongement est concaténé au vecteur  $c$  pour constituer le conditionnement global  $c_g \in \mathbb{R}^{384}$ , par lequel la dépendance du champ  $u_t^\theta$  au temps et à l’observation entre dans le réseau de vitesse. Ces encodeurs sont repris tels quels de la littérature et ne constituent pas une contribution de ce mémoire ; leur composition exacte (dimensions des couches, activations, fréquences du plongement) est documentée à l’Annexe B.

## Réseau de vitesse U-Net 1D

Le réseau de vitesse est le U-Net 1D conditionnel popularisé par *Diffusion Policy* [25], dont la vue d'ensemble est illustrée à la Figure B.1 de l'Annexe B. Le segment  $X_t \in \mathbb{R}^{H \times 10}$  y est traité comme un signal unidimensionnel de longueur  $H$  à dix canaux, et la sortie  $\mathbf{v}_t$ , de même forme, est l'estimation de la vitesse  $u_t^\theta(X_t | c)$  pour chacune des  $H$  actions. Le réseau comprend trois niveaux de résolution (256, 512 puis 1024 canaux, la longueur temporelle étant divisée par deux entre les niveaux), un goulot central, puis une remontée symétrique dont chaque niveau concatène les cartes du niveau descendant correspondant (connexions de saut). Tous les niveaux sont constitués de blocs résiduels conditionnels, dans lesquels le conditionnement global  $c_g$  module les cartes de caractéristiques par une transformation affine de type FiLM : cette injection du conditionnement à tous les niveaux de résolution permet à l'observation d'influencer autant la forme d'ensemble du segment que ses détails locaux. La composition interne de ces blocs (convolutions, normalisation, activation et modulation) est détaillée à l'Annexe B.

Le U-Net convolutif est préféré à la tête Transformer, l'autre architecture de débruitage proposée par *Diffusion Policy*, pour trois raisons : la variante convolutive s'entraîne de façon stable sans réglage particulier là où la variante Transformer est nettement plus sensible aux hyperparamètres [25], robustesse qui prime avec quelques dizaines de démonstrations par tâche ; les biais inductifs des convolutions temporelles (localité, régularité) correspondent aux trajectoires démontrées, lisses et échantillonnées à 10 Hz, sans les changements d'action à haute fréquence pour lesquels la variante Transformer est recommandée ; et sur un horizon court ( $H = 16$ ), la modélisation des dépendances à longue portée par l'auto-attention n'apporte rien, le goulot du U-Net couvrant déjà l'horizon entier.

## Inférence et conversion en consignes

À l'inférence, la génération d'un segment consiste à intégrer l'équation (4.4) de  $t = 0$  à  $t = 1$  à partir d'un tirage  $X_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ , par le schéma d'Euler explicite :

$$X_{t+\Delta t} = X_t + \Delta t u_t^\theta(X_t | c), \quad \Delta t = 1/N. \quad (4.7)$$

Le segment obtenu est dénormalisé puis converti en consignes exécutables : les positions absolues sont récupérées en réappliquant la position courante de l'outil, inverse du centrage en translation ; la matrice de rotation de chaque action est reconstruite par orthonormalisation de Gram-Schmidt de ses six composantes d'orientation [118] ; et l'état binaire de la pince est obtenu par seuillage de la dixième composante à 0,5. Le résultat est une séquence de

$H = 16$  poses de l’outil accompagnées de la commande de la pince, transmise à la chaîne de conversion et d’exécution de la section suivante.

### 4.5.3 Entraînement du modèle

Les trajectoires prétraitées sont découpées en échantillons par fenêtre glissante : à chaque pas d’une trajectoire correspond un échantillon regroupant le nuage conditionnel courant, la proprioception des deux derniers pas et le segment des  $H = 16$  actions à venir, soit 1,6 s à la cadence d’enregistrement de 10 Hz. Les trajectoires sont réparties entre entraînement (80 %) et validation (20 %).

Chaque échantillon d’entraînement subit une augmentation aléatoire qui préserve la cohérence géométrique entre le nuage et les poses : une rotation de la scène complète autour de la verticale, tirée uniformément dans  $\pm 15^\circ$  et appliquée simultanément au nuage, aux positions et aux composantes d’orientation ; une translation uniforme de  $\pm 1$  cm appliquée au nuage et aux positions ; et un bruit gaussien d’écart-type 5 mm, tronqué à  $\pm 1$  cm, ajouté aux points du nuage pour imiter le bruit du capteur. L’état binaire de la pince n’est pas modifié par l’augmentation.

Le Tableau 4.2 rassemble les hyperparamètres d’entraînement, identiques pour les deux tâches. L’optimisation minimise la perte (4.6) avec AdamW, un pas d’apprentissage suivant un recuit en cosinus après une période d’échauffement et un écrêtage du gradient à une norme unitaire. Une moyenne mobile exponentielle (EMA) des poids du modèle est maintenue tout au long de l’entraînement et sert tant à l’évaluation qu’au déploiement, ce qui lisse les oscillations des derniers pas d’optimisation ; le point de contrôle retenu est celui qui minimise la perte de validation évaluée avec ces poids moyennés.

## 4.6 Conversion et exécution de la trajectoire nominale

La sortie du modèle de la section précédente est une séquence de poses de l’outil dans l’espace cartésien ; le robot, lui, est asservi dans l’espace articulaire. Cette section décrit la chaîne qui transforme chaque segment généré en une trajectoire articulaire exécutable : le passage des poses de l’outil aux poses du TCP, leur conversion en configurations articulaires par cinématique inverse, puis le réajustement temporel qui leur affecte des instants de passage produisant un profil de vitesse régulier.

Tableau 4.2 Hyperparamètres d’entraînement du modèle de Flow Matching.

| Hyperparamètre                                    | Valeur                          |
|---------------------------------------------------|---------------------------------|
| Horizon de prédiction $H$ / horizon d’observation | 16 / 2                          |
| Dimension des actions et de la proprioception     | 10                              |
| Points du nuage conditionnel                      | 1024                            |
| Taille de lot                                     | 128                             |
| Nombre d’époques                                  | 500                             |
| Optimiseur                                        | AdamW                           |
| Pas d’apprentissage initial                       | $1 \times 10^{-4}$              |
| Régularisation ( <i>weight decay</i> )            | $1 \times 10^{-6}$              |
| Ordonnancement du pas d’apprentissage             | cosinus, 500 pas d’échauffement |
| Écrêtage du gradient (norme)                      | 1,0                             |
| Taux de décroissance EMA (maximal)                | 0,9999                          |
| Pas d’intégration à l’inférence $N$               | 10                              |

#### 4.6.1 Conversion outil–TCP

Le modèle de Flow Matching est entraîné et échantillonné dans le repère de l’outil  $\mathcal{F}_o$  : pour la tâche d’alimentation, les poses générées sont celles de la pointe de la fourchette, soit l’élément qui interagit physiquement avec la cible. Le contrôleur du robot n’asservit toutefois pas ce repère, mais celui du TCP  $\mathcal{F}_e$ . La pose du TCP à commander s’obtient en inversant la relation (4.2) utilisée au prétraitement (Section 4.4.2) :

$$\mathbf{T}_{e,k}^b = \mathbf{T}_{o,k}^b \left( \mathbf{T}_o^e \right)^{-1}, \quad k = 1, \dots, H, \quad (4.8)$$

où  $\mathbf{T}_{o,k}^b$  est la pose homogène de l’outil au pas  $k$ , assemblée à partir de la position et de la matrice de rotation issues de l’inférence (Section 4.5), et  $\mathbf{T}_o^e$  la transformation rigide constante entre le TCP et l’outil, illustrée à la Figure 4.6 et consignée dans les fichiers de trajectoire (Section 4.4.2). Cette transformation étant connue et invariante, la conversion est exacte et ne coûte qu’un produit matriciel par pose ; pour la tâche de saisie et dépôt, l’outil est le TCP lui-même ( $\mathbf{T}_o^e = \mathbf{I}$ ). Les poses ainsi obtenues constituent les poses désirées du TCP, notées  $\mathbf{T}_{e,k}^{b\star}$  dans la sous-section suivante.

#### 4.6.2 Cinématique inverse pour obtenir la trajectoire articulaire nominale

La trajectoire générée par *Flow Matching* décrit le mouvement du lien contrôlé dans l’espace cartésien : sur l’horizon  $H$ , le modèle produit une séquence de poses que la conversion outil–TCP de la sous-section précédente ramène en poses désirées du TCP,  $\mathbf{T}_{e,k}^{b\star} \in SE(3)$ ,

$k = 1, \dots, H$ . Le robot étant commandé dans l'espace articulaire, chaque pose doit être convertie en une configuration  $\mathbf{q}_k \in \mathbb{R}^7$  par cinématique inverse. Seules les sept articulations du bras interviennent dans la tâche : les deux articulations de la pince complètent le vecteur de configuration du modèle mais n'influencent pas sur la pose du TCP, et la préhension est commandée séparément. Le manipulateur disposant de sept articulations pour une tâche de pose à six degrés de liberté, le problème est redondant et son issue doit être fixée par un critère explicite.

La résolution s'effectue, pour chaque point de passage, par une descente de Gauss-Newton amortie sur l'erreur de pose, implémentée en C++ avec la bibliothèque `Pinocchio`. À l'itération courante, l'écart entre la pose atteinte  $\mathbf{T}_e^b(\mathbf{q})$  et la pose désirée  $\mathbf{T}_{e,k}^{b*}$  est mesuré dans l'algèbre de Lie de  $SE(3)$ , via le logarithme du groupe,

$$\mathbf{e}(\mathbf{q}) = \log\left((\mathbf{T}_{e,k}^{b*})^{-1} \mathbf{T}_e^b(\mathbf{q})\right) \in \mathbb{R}^6.$$

La Jacobienne géométrique du repère TCP,  $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6 \times 7}$ , donne le torseur cinématique (vitesses linéaire et angulaire) du TCP produit par une vitesse articulaire. La différentielle exacte de l'erreur logarithmique est  $\mathbf{J}_{\log}(\mathbf{e}) \mathbf{J}(\mathbf{q})$ , où  $\mathbf{J}_{\log}$  est la Jacobienne du logarithme de  $SE(3)$ ; celle-ci tendant vers l'identité lorsque  $\mathbf{e} \rightarrow \mathbf{0}$ , le solveur l'omet et utilise la linéarisation  $\mathbf{e}(\mathbf{q} + \Delta\mathbf{q}) \approx \mathbf{e}(\mathbf{q}) + \mathbf{J}(\mathbf{q}) \Delta\mathbf{q}$ , approximation de Gauss-Newton standard qui n'altère que la direction des premiers pas, loin de la convergence, sans changer le point fixe  $\mathbf{e} = \mathbf{0}$ . L'incrément articulaire combine alors une tâche principale, celle de réduire  $\mathbf{e}$ , ainsi qu'une tâche secondaire confinée au noyau de  $\mathbf{J}$ , c'est-à-dire aux mouvements articulaires qui laissent la pose du TCP inchangée :

$$\Delta\mathbf{q} = \underbrace{-\mathbf{J}^+\mathbf{e}}_{\text{suivi de pose}} + \underbrace{(\mathbf{I} - \mathbf{J}^+\mathbf{J})\left(-k_0(\mathbf{q} - \mathbf{q}_{\text{ref}})\right)}_{\text{résolution de la redondance}}, \quad \mathbf{J}^+ = \mathbf{J}^\top (\mathbf{J}\mathbf{J}^\top + \lambda\mathbf{I})^{-1}.$$

Le premier terme,  $-\mathbf{J}^+\mathbf{e}$ , est la pseudo-inverse amortie de  $\mathbf{J}$  [120, 121] appliquée à l'erreur : l'amortissement  $\lambda$  régularise le voisinage des singularités, où la pseudo-inverse non amortie produirait des pas divergents. Le second terme rapproche la configuration d'une posture de référence  $\mathbf{q}_{\text{ref}}$  au moyen de l'objectif  $-k_0(\mathbf{q} - \mathbf{q}_{\text{ref}})$ , que le projecteur  $\mathbf{I} - \mathbf{J}^+\mathbf{J}$  restreint au noyau de  $\mathbf{J}$  [122] : la résolution de la redondance limite ainsi son influence sur la tâche principale, au premier ordre et à l'amortissement près,  $\mathbf{I} - \mathbf{J}^+\mathbf{J}$  n'étant qu'un projecteur approché lorsque  $\lambda > 0$ . L'incrément résultant est borné en norme ( $\|\Delta\mathbf{q}\| \leq \delta_{\text{max}} = 0,2$  rad), puis intégré sur la variété des configurations,  $\mathbf{q} \leftarrow \mathbf{q} \oplus \Delta\mathbf{q}$ , jusqu'à ce que  $\|\mathbf{e}\| < \varepsilon = 10^{-5}$  ou que 50 itérations soient atteintes.

La référence  $\mathbf{q}_{\text{ref}}$  est prise égale à la configuration d’amorçage (*warm-start*), elle-même propagée le long de l’horizon : la première pose est résolue à partir de la configuration mesurée du robot, et chaque pose suivante à partir de la solution de la précédente. Ce chaînage assure simultanément une résolution cohérente de la redondance, les  $\mathbf{q}_k$  restant voisins, et la continuité de la trajectoire articulaire ; des garde-fous numériques (déroulement des écarts successifs dans  $[-\pi, \pi]$ , projection de la rotation cible sur  $SO(3)$ , rejet des configurations dégénérées au profit de la configuration d’amorçage) complètent la robustesse du solveur.

Le solveur produit ainsi la trajectoire articulaire nominale  $\mathbf{q}_{1:H}$ , qui est la seule grandeur retenue en aval : elle est publiée, puis ré-échelonnée temporellement (Section 4.6.3) avant d’alimenter le filtre de sécurité.

#### 4.6.3 Réajustement temporel de la trajectoire articulaire

La cinématique inverse fournit une suite de configurations articulaires  $\mathbf{q}_1, \dots, \mathbf{q}_H$ , mais sans horodatage : le modèle de *Flow Matching* produit un nombre fixe de points de passage dont l’espacement dans l’espace articulaire n’est pas uniforme et varie aussi bien à l’intérieur d’un plan que d’un plan à l’autre. Leur affecter une cadence constante, par exemple un point toutes les 0,1 s, ferait alors varier fortement la vitesse articulaire réelle : élevée entre deux configurations éloignées, faible entre deux configurations voisines, d’où un profil de vitesse irrégulier et saccadé. Le réajustement attribue au contraire à chaque segment une durée proportionnelle à sa longueur articulaire, de sorte que la vitesse reste approximativement constante d’un segment à l’autre.

La longueur d’un segment est mesurée par une norme articulaire pondérée,

$$\ell_i = \left\| \mathbf{U} (\mathbf{q}_{i+1} - \mathbf{q}_i) \right\|, \quad \mathbf{U} = \text{diag}(\mathbf{u}),$$

où les poids  $\mathbf{u}$  autorisent à pénaliser différemment les articulations (tous égaux à l’unité dans la configuration retenue, soit la norme euclidienne articulaire). Pour une vitesse articulaire cible  $v^\star$  (fixée à 0,3 rad/s), la durée de chaque segment et les instants de passage valent

$$\Delta t_i = \frac{\max(\ell_i, \ell_{\min})}{v^\star}, \quad t_1 = 0, \quad t_{i+1} = t_i + \Delta t_i,$$

chaque segment de longueur  $\ell_i \geq \ell_{\min}$  étant alors parcouru à la vitesse articulaire constante  $v^\star$ . Le plancher  $\ell_{\min} = f_{\min} L / (H - 1)$ , fraction  $f_{\min} = 0,02$  de la longueur totale  $L = \sum_i \ell_i$  rapportée au nombre de segments, empêche un segment quasi stationnaire de se voir attribuer une durée nulle, qui ferait diverger la vitesse définie ci-dessous ; les segments plus courts que



ce plancher sont en contrepartie parcourus à une vitesse inférieure à  $v^*$ , d'autant plus faible que le segment est court.

La trajectoire ré-échelonnée  $(\mathbf{q}_i, t_i)$  est publiée sous forme de positions horodatées, sans vitesse explicite. L'exécuteur de trajectoire l'interpole à la fréquence de commande et en publie la vitesse feedforward par différentiation finie :

$$\dot{\mathbf{q}}_{\text{ff}}(t) = \frac{\mathbf{q}_{i+1} - \mathbf{q}_i}{t_{i+1} - t_i}, \quad t \in [t_i, t_{i+1}]. \quad (4.9)$$

Cette vitesse ne provient donc pas d'un second calcul de cinématique inverse, mais de la simple différentiation des positions articulaires aux instants imposés par le réajustement. Par construction de ces instants, sa norme est approximativement constante et proche de  $v^*$  malgré des points de passage non équidistants. Elle représente l'intention instantanée de la trajectoire générée par *Flow Matching* et fournit au filtre de sécurité une référence régulière, plutôt qu'une différentiation bruitée d'une consigne de position échantillonnée à cadence fixe.

L'exécuteur n'avance pas le long de cette trajectoire au rythme de l'horloge murale, mais d'une *horloge de progression* : l'abscisse temporelle courante n'est incrémentée que si le robot suit effectivement la consigne, et son avancement est suspendu dans deux cas. Le premier est un retard de poursuite excessif : si l'écart articulaire entre la consigne interpolée et la configuration mesurée dépasse une limite, la consigne attend le robot au lieu de creuser l'erreur. Le second est signalé par le filtre de sécurité lui-même : tant que la barrière publiée est en violation ( $h < 0$ ), la progression est gelée. Sans ce gel, la consigne nominale continuerait de défiler pendant qu'une déviation de sécurité retient le robot : l'erreur de suivi s'accumulerait, le terme de retour saturerait contre la contrainte et, à la libération, le robot rejoindrait la trajectoire loin en aval, court-circuitant le segment que le filtre venait de lui faire éviter. Avec lui, la trajectoire planifiée reprend exactement là où elle a été interrompue.

#### 4.7 Filtre de sécurité réactif SDF-CBF

Le modèle génératif est entraîné sur des démonstrations sans obstacle et exécuté en boucle ouverte : il n'offre aucune garantie vis-à-vis d'un environnement non vu à l'entraînement. Plutôt que de contraindre le processus de génération (au prix d'une dérive distributionnelle et d'un coût de calcul accru, voir le Chapitre 2), la sécurité est déléguée à un filtre réactif agissant sur la vitesse articulaire à la fréquence de commande. Le robot étant protégé sur l'ensemble de ses liens, chaque lien  $l$  menacé par un obstacle impose sa propre condition de sécurité ; à chaque cycle de commande, le filtre retient la vitesse la plus proche de l'intention

du modèle qui les respecte toutes :

$$\dot{\mathbf{q}}^* = \arg \min_{\dot{\mathbf{q}} \in \mathbb{R}^7} \frac{1}{2} \|\dot{\mathbf{q}} - \dot{\mathbf{q}}_{\text{base}}\|_{\mathbf{W}}^2 \quad \text{s.c.} \quad \nabla h_l(\mathbf{q})^\top \dot{\mathbf{q}} \geq b(h_l(\mathbf{q})), \quad \forall l \in \mathcal{A}. \quad (4.10)$$

où  $\mathbf{q} \in \mathbb{R}^7$  est la configuration articulaire mesurée,  $\dot{\mathbf{q}}_{\text{base}}$  la vitesse nominale (l'intention du modèle génératif),  $\mathcal{A}$  l'ensemble des liens dont la contrainte est active au cycle courant et  $\|\cdot\|_{\mathbf{W}}$  une norme articulaire pondérée par une métrique  $\mathbf{W} \succ 0$ . Chaque contrainte délimite un demi-espace de vitesses admissibles : la commande retenue est la projection, au sens de  $\mathbf{W}$ , de  $\dot{\mathbf{q}}_{\text{base}}$  sur leur intersection. La métrique ne change pas l'ensemble admissible, elle ne modifie donc pas la sécurité obtenue. Elle influence la répartition articulaire de la correction, c'est-à-dire de ce que le filtre sacrifie lorsqu'il doit dévier de l'intention du modèle. Ce problème comporte trois ingrédients, qui structurent la section. Les fonctions barrière  $h_l(\mathbf{q})$ , qui mesurent la distance signée entre chaque lien et les points obstacles  $\mathbf{p}_i \in \mathbb{R}^3$  de la carte persistante, reposent sur un modèle de distance du robot appris hors ligne (Section 4.7.1), à partir duquel elles sont assemblées, tout comme leur gradient  $\nabla h_l$  obtenu en ligne (Sections 4.7.2 et 4.7.3). Le second membre  $b(h_l)$  et la vitesse nominale  $\dot{\mathbf{q}}_{\text{base}}$  sont définis à la Section 4.7.4. Enfin, la Section 4.7.5 résout la projection et définit la métrique  $\mathbf{W}$ , décrit le post-traitement qui transforme cette solution en commande exécutable  $\dot{\mathbf{q}}_{\text{final}}$  ; la Section 4.7.6 traite les minima locaux induits par le filtre, et la Section 4.7.7 délimite la portée des garanties obtenues.

#### 4.7.1 Modèle de distance du robot appris hors ligne

Le filtre doit connaître, en quelques millisecondes et pour des milliers de points simultanément, la distance signée entre tout point de l'espace et la surface du robot ainsi que leur gradient.

#### Représentation par polynômes de Bernstein

Suivant l'approche *Robot Distance Fields* [80], également exploitée pour l'évitement réactif de collisions dans un cadre de commande à priorité de tâches [83], le SDF de chaque lien  $l$  est approximé par une base tensorielle de polynômes de Bernstein dans un repère normalisé propre au lien. La géométrie maillée du lien est d'abord centrée et mise à l'échelle :

$$\bar{\mathbf{x}} = \frac{\mathbf{x} - \mathbf{o}_l}{s_l}, \quad \mathbf{o}_l = \text{centre de la boîte englobante}, \quad s_l = \max_{\mathbf{v} \in \mathcal{V}_l} \|\mathbf{v} - \mathbf{o}_l\|, \quad (4.11)$$

où  $\mathcal{V}_l$  est l'ensemble des sommets du maillage : le lien est ainsi contenu dans la boule unitaire, donc dans le cube  $[-1, 1]^3$  qui sert de domaine d'approximation. Sur ce domaine, le SDF normalisé est une combinaison linéaire de  $n^3$  fonctions de base :

$$\bar{d}_l(\bar{\mathbf{x}}) = \sum_{a,b,c=0}^{n-1} w_{abc}^{(l)} B_a(u_x) B_b(u_y) B_c(u_z) = \Phi(\bar{\mathbf{x}}) \mathbf{w}_l \quad (4.12)$$

$$B_a(u) = \binom{n-1}{a} u^a (1-u)^{n-1-a} \quad (4.13)$$

$$\mathbf{u} = \frac{\bar{\mathbf{x}} + \mathbf{1}}{2} \quad (4.14)$$

et la distance métrique s'en déduit par  $d_l = s_l \bar{d}_l$ . Les polynômes de Bernstein étant définis sur  $[0, 1]$ , le changement de variable affine  $\mathbf{u} = (\bar{\mathbf{x}} + \mathbf{1})/2$  y ramène la coordonnée normalisée : paramétrer la base sur  $[-1, 1]^3$  ou sur  $[0, 1]^3$  est strictement équivalent, et la convention retenue ici est celle de l'implémentation de référence de [80], qui normalise de même chaque lien dans  $[-1, 1]^3$  avant d'évaluer la base sur  $[0, 1]$ . Chacun des neuf liens protégés (liens 2 à 7, la main et les deux doigts de la pince) utilise  $n = 12$  fonctions par axe, soit 1728 coefficients par lien. Trois propriétés motivent ce choix de représentation : le modèle est polynomial, donc  $\mathcal{C}^\infty$  et de gradient analytique exact ; il est linéaire en ses paramètres  $\mathbf{w}_l$ , ce qui rend l'ajustement convexe ; et son évaluation se réduit à des produits tensoriels, massivement parallélisables sur GPU.

## Apprentissage des poids en deux étapes

L'apprentissage des poids procède en deux étapes par lien. La première reproduit le protocole original de [80] : un jeu d'étiquettes de distance signée exacte est généré sur le maillage (points concentrés près de la surface et points uniformes dans le cube normalisé), puis, la sortie du modèle étant linéaire en  $\mathbf{w}_l$ , les poids sont ajustés par *moindres carrés récursifs* avec régularisation de type *ridge*. Cette étape n'est pas une contribution de ce mémoire ; son protocole d'échantillonnage, ses équations de mise à jour et ses hyperparamètres sont reproduits à l'Annexe C. La régression de valeur qui en résulte fournit une distance précise, mais ne contraint pas le gradient : près de la surface, sa direction reste bruitée et sa norme s'écarte de l'unité.

Or le filtre de sécurité consomme précisément ce gradient :  $\nabla h$  fixe la *direction* de la correction de la projection (4.10) et sa *norme* en fixe l'amplitude. Une seconde étape de raffinement géométrique, propre à ce mémoire, est donc appliquée. Un jeu de supervision dédié de 20 000 *rayons normaux vérifiés* est construit : des points de surface sont déplacés le long de la normale de leur facette d'un décalage  $\delta$  tiré aléatoirement, concentré dans la coquille critique autour

de la marge de sécurité  $d_{\text{safe}}$ . Chaque candidat n'est retenu que s'il est vérifié contre le maillage exact : le point doit être extérieur, sa distance exacte recalculée doit coïncider avec le décalage imposé, et la direction exacte du gradient ne doit pas s'écarter sensiblement de la normale de départ, ce qui élimine les rayons corrompus par l'axe médian ou la courbure. Chaque rayon fournit ainsi un triplet exact (point, distance, direction sortante  $\hat{\mathbf{n}}$ ) ; la répartition des décalages et les tolérances de vérification sont détaillées à l'Annexe C. Les poids sont alors affinés par descente de gradient sur la fonction de coût

$$\begin{aligned} \mathcal{L} = & \mathcal{L}_{\text{près}} + \mathcal{L}_{\text{unif}} + \lambda_{\text{ray}} \mathcal{L}_{\text{ray}} + \lambda_{\text{eik}} \mathbb{E}[(\|\nabla \bar{d}\| - 1)^2] \\ & + \mathbb{E}\left[\omega\left(\lambda_{\text{dir}}(1 - \widehat{\nabla \bar{d}} \cdot \hat{\mathbf{n}}) + \lambda_{\text{vec}} \|\nabla \bar{d} - \hat{\mathbf{n}}\|^2\right)\right] \end{aligned} \quad (4.15)$$

où les trois premiers termes sont les erreurs quadratiques de valeur (points près de la surface, uniformes et rayons), le terme eikonal impose  $\|\nabla \bar{d}\| = 1$  partout, et les deux derniers alignent la direction du gradient sur la direction exacte des rayons, avec une pondération  $\omega$  qui décroît avec la distance et concentre l'effort près de la surface ; les poids relatifs  $\lambda$  sont donnés à l'Annexe C. Le gradient  $\nabla \bar{d}$  étant la dérivée analytique d'un polynôme, aucune approximation numérique n'intervient.

Les modèles finaux de chaque lien enchaînent ainsi les deux étapes, la régression de valeur servant d'initialisation au raffinement géométrique ; les hyperparamètres complets d'optimisation sont rassemblés à l'Annexe C. La précision de distance, la direction du gradient et la propriété eikonale qui en résultent sont caractérisées expérimentalement au Chapitre 5.

#### 4.7.2 Fonction barrière

##### Évaluation de la distance et prolongement hors domaine

À l'exécution, chaque point obstacle  $\mathbf{p}_i$  est ramené dans le repère normalisé de chaque lien protégé via la cinématique directe, puis évalué par le modèle appris :

$$\mathbf{x}_{l,i}(\mathbf{q}) = \mathbf{T}_l^b(\mathbf{q})^{-1} \mathbf{p}_i, \quad d_{l,i}(\mathbf{q}) = s_l \Phi\left(\frac{\mathbf{x}_{l,i} - \mathbf{o}_l}{s_l}\right) \mathbf{w}_l.$$

Pour les points situés hors du domaine d'approximation  $[-1, 1]^3$ , la coordonnée normalisée est bornée au cube (point  $\mathbf{c}$ ) et la distance est prolongée par le minorant garanti

$$\hat{d}(\mathbf{p}) = \sqrt{d(\mathbf{c})^2 + \|\mathbf{p} - \mathbf{c}\|^2}.$$

La garantie tient à la géométrie du bornage par axes : tout point de surface  $\mathbf{s}$  du lien appartient au cube et, sur chaque axe borné,  $|p_j - s_j| = |c_j - s_j| + |p_j - c_j|$ , d'où  $\|\mathbf{p} - \mathbf{s}\|^2 \geq \|\mathbf{c} - \mathbf{s}\|^2 + \|\mathbf{p} - \mathbf{c}\|^2$  pour tout  $\mathbf{s}$ , et donc  $d(\mathbf{p})^2 \geq d(\mathbf{c})^2 + \|\mathbf{p} - \mathbf{c}\|^2$ . L'inégalité vaut pour la distance vraie évaluée au point borné ; en pratique,  $d(\mathbf{c})$  est la distance apprise, et le minorant hérite de son erreur d'approximation au même titre que le modèle à l'intérieur du domaine (Section 4.7.7). Sous cette réserve, l'erreur du prolongement est toujours du côté sûr, la distance étant sous-estimée et jamais surestimée. Deux propriétés supplémentaires le distinguent d'autres minorants valides, telle la borne lipschitzienne  $d(\mathbf{c}) - \|\mathbf{p} - \mathbf{c}\|$  : il croît de façon monotone avec l'éloignement, de sorte que son gradient conserve partout l'orientation physique qu'exigent la contrainte de sécurité et le dégagement postural de la Section 4.7.6, alors qu'un minorant décroissant avec l'excursion inverserait la direction de correction ; et il se raccorde au modèle polynomial à la frontière du domaine, continûment en valeur, les composantes du gradient portées par les axes intérieurs s'y raccordant également, tandis que la composante normale aux faces bornées s'annule à la frontière là où celle du polynôme ne s'annule pas : le prolongement est donc continu et différentiable par morceaux, non continûment différentiable. Les points qui déclenchent la contrainte résident pour l'essentiel à l'intérieur du domaine ; le prolongement gouverne en revanche la bande d'activation large du dégagement postural et l'ordonnancement des candidats lointains du préfiltre. Cette formulation fournit une distance différentiable par morceaux par rapport à  $\mathbf{q}$ , interrogeable en parallèle sur GPU pour l'ensemble des points obstacles.

### Sélection des points critiques

Évaluer cette distance sur la totalité de la carte persistante à chaque cycle serait inutilement coûteux : seuls les points les plus proches du robot peuvent activer la contrainte. La carte est donc d'abord élaguée par un préfiltre sphérique autour des liens protégés, puis les  $K$  points de plus faible SDF corps-complet sont conservés :

$$\mathcal{P}_{\text{CBF}} = \text{TopK}_K \left( \min_l d_l(\mathbf{q}, \mathbf{p}_i) \right), \quad K = 100.$$

Cette sélection est mise à jour à une fréquence inférieure à celle de la boucle de commande (30 Hz contre 100 Hz). L'obsolescence maximale de l'ensemble actif qui en résulte ( $\approx 33$  ms) est prise en compte dans le budget de la marge de sécurité  $d_{\text{safe}}$  (Section 4.7.7).

### Agrégation en barrière par lien

Les distances  $d_{l,i}$  des points retenus sont alors agrégées, lien par lien, en une barrière scalaire. Le minimum sur les points obstacles  $y$  est approché par un soft-min de température  $\alpha$ , formulé de manière numériquement stable autour de  $m_l = \min_i d_{l,i}$  :

$$h_l(\mathbf{q}) = m_l - \alpha \log \left[ \sum_{i=1}^N \exp \left( -\frac{d_{l,i} - m_l}{\alpha} \right) \right] - d_{\text{safe}}.$$

Chaque lien protégé possède ainsi sa propre barrière  $h_l$  et son propre ensemble admissible  $\{\mathbf{q} \mid h_l(\mathbf{q}) \geq 0\}$  ; l'ensemble sûr du robot en est l'intersection,

$$\mathcal{C} = \bigcap_l \{\mathbf{q} \mid h_l(\mathbf{q}) \geq 0\}.$$

Réduire cette famille à la seule barrière minimale ne contraindrait que le lien instantanément le plus proche et laisserait les autres liens libres de progresser vers leurs propres obstacles, au prix d'une commutation brutale du lien sélectionné dès qu'un second lien devient critique. Le filtre conserve donc les  $K_a$  liens de plus faible  $h_l$  et traite chacun comme une contrainte distincte ; leur assemblage et leur résolution conjointe sont détaillés à la Section 4.7.5. Le soft-min introduit un biais conservateur borné par  $\alpha \log N$  ; en notant  $h_{\text{soft}}$  la valeur brute issue du soft-min, la valeur compensée  $h_{\text{corr}} = h_{\text{soft}} + \alpha \log N$  est utilisée pour l'interprétation des mesures ; c'est sous ces deux noms que la barrière apparaît au Chapitre 5. Ce terme étant indépendant de  $\mathbf{q}$ , le gradient n'est pas affecté.

Dans le problème (4.10), c'est la valeur de  $h_l$  qui fixe la bande d'activation et le second membre  $b(h_l)$ , et le biais effectif vaut  $\alpha \log M_{\text{eff}}$ , où  $M_{\text{eff}}$  est le nombre de points logés à moins de  $\alpha$  du minimum. Le filtre déployé transmet directement la valeur soft-min au second membre : la valeur et le gradient proviennent alors de la même fonction lisse, la paire est cohérente, et la condition appliquée est la condition CBF standard de  $h_{\text{soft}}$  (Section 4.7.7). Le biais d'encombrement n'y est pas une erreur mais une conservativité certifiée :  $h_{\text{soft}} \leq h_{\min,l} = \min_i d_{l,i} - d_{\text{safe}}$ , de sorte que la marge réelle excède toujours la marge décidée, d'autant plus que le nuage est dense. Cette conservativité a un coût : face à un nuage dense (une surface plane vue de près), le biais atteint l'ordre de la marge  $d_{\text{safe}}$  elle-même et le filtre freine plus tôt que la géométrie ne l'exige, jusqu'à déclencher son régime de récupération. Une variante à valeur exacte est donc également implémentée : elle transmet  $h_{\min,l}$ , disponible sans calcul supplémentaire, tout en conservant le gradient soft-min pour sa continuité lors des commutations du point dominant (Section 4.7.3). Son certificat, indirect, est établi à la Section 4.7.7, et le Chapitre 5 compare les deux modes.

### 4.7.3 Gradient analytique de la contrainte

La projection (4.10) repose entièrement sur le gradient  $\nabla_{\mathbf{q}}h$ , qui en fixe à la fois la direction et l'amplitude. On l'obtient en forme close, par dérivation exacte de chaque maillon de la chaîne, ce qui en garantit le coût borné qu'exige le temps réel. Pour un point obstacle donné, la distance résulte de la composition de trois applications,

$$\mathbf{q} \xrightarrow{\text{cinématique}} \mathbf{x}_{l,i} \xrightarrow{\text{normalisation}} \mathbf{u} \xrightarrow{\text{SDF de Bernstein}} \bar{d}_l,$$

toutes trois dérivables analytiquement. Cette forme close distingue le présent filtre de l'implémentation de référence de [80], qui obtient le gradient articulaire par différences finies sur la cinématique directe : elle en fournit la valeur exacte, sans choix d'un pas de perturbation et à coût indépendant du nombre d'articulations perturbées. Les trois différentielles sont dérivées ci-dessous, puis composées.

**Différentielle cinématique.** L'obstacle est fixe dans le repère de base  $\mathcal{F}_b$  ( $\dot{\mathbf{p}}_i = \mathbf{0}$ ), mais paraît mobile dans le repère  $\mathcal{F}_l$  que les articulations déplacent. Sa position locale  $\mathbf{x}_{l,i} = (\mathbf{R}_l^b)^\top (\mathbf{p}_i - \mathbf{d}_l^b)$  vérifie la relation de fermeture  $\mathbf{p}_i = \mathbf{d}_l^b + \mathbf{R}_l^b \mathbf{x}_{l,i}$ , où  $\mathbf{R}_l^b(\mathbf{q})$  et  $\mathbf{d}_l^b(\mathbf{q})$  sont la rotation et la translation de  $\mathbf{T}_l^b(\mathbf{q})$ . En dérivant cette relation par rapport au temps et en annulant  $\dot{\mathbf{p}}_i$  :

$$\mathbf{0} = \dot{\mathbf{d}}_l^b + \dot{\mathbf{R}}_l^b \mathbf{x}_{l,i} + \mathbf{R}_l^b \dot{\mathbf{x}}_{l,i}.$$

Les blocs linéaire et angulaire de la Jacobienne géométrique du lien,  $\mathbf{J}_{L,l}^b$  et  $\mathbf{J}_{A,l}^b$ , relient ces dérivées à la vitesse articulaire par  $\dot{\mathbf{d}}_l^b = \mathbf{J}_{L,l}^b \dot{\mathbf{q}}$  et  $\dot{\mathbf{R}}_l^b = [\mathbf{J}_{A,l}^b \dot{\mathbf{q}}]_\times \mathbf{R}_l^b$ . En les substituant et en isolant le terme de vitesse locale :

$$\mathbf{R}_l^b \dot{\mathbf{x}}_{l,i} = -\mathbf{J}_{L,l}^b \dot{\mathbf{q}} - [\mathbf{J}_{A,l}^b \dot{\mathbf{q}}]_\times \mathbf{R}_l^b \mathbf{x}_{l,i}.$$

Une multiplication à gauche par  $(\mathbf{R}_l^b)^\top$ , suivie des identités du produit vectoriel  $\mathbf{R}^\top [\mathbf{a}]_\times \mathbf{R} = [\mathbf{R}^\top \mathbf{a}]_\times$  et  $[\mathbf{a}]_\times \mathbf{b} = -[\mathbf{b}]_\times \mathbf{a}$ , isole la Jacobienne recherchée  $\mathbf{J}_{x_{l,i}} = \partial \mathbf{x}_{l,i} / \partial \mathbf{q}$ , telle que  $\dot{\mathbf{x}}_{l,i} = \mathbf{J}_{x_{l,i}} \dot{\mathbf{q}}$  :

$$\mathbf{J}_{x_{l,i}}(\mathbf{q}) = -(\mathbf{R}_l^b)^\top \mathbf{J}_{L,l}^b + [\mathbf{x}_{l,i}]_\times (\mathbf{R}_l^b)^\top \mathbf{J}_{A,l}^b,$$

le premier terme traduisant la translation du repère du lien, le second sa rotation ;  $[\mathbf{x}_{l,i}]_\times$  est la matrice antisymétrique associée au produit vectoriel.

**Jacobienne de la normalisation.** La normalisation qui ramène l'obstacle dans le cube d'apprentissage,  $\mathbf{u} = (\mathbf{x}_{l,i} - \mathbf{o}_l) / (2s_l) + \frac{1}{2}\mathbf{1}$ , est affine ; sa Jacobienne est la matrice diagonale

constante

$$\frac{\partial \mathbf{u}}{\partial \mathbf{x}_{l,i}} = \frac{1}{2s_l} \mathbf{I}_3.$$

**Gradient du SDF de Bernstein.** La dérivée d'un polynôme de Bernstein de degré  $n - 1$  se ramène à la base de degré immédiatement inférieur par la règle d'abaissement

$$\frac{dB_a(u)}{du} = (n - 1) \left( B_{a-1}^{(n-2)}(u) - B_a^{(n-2)}(u) \right),$$

d'où, en dérivant le produit tensoriel selon chacun des trois axes, le gradient  $\nabla_{\mathbf{u}} \bar{d}_l \in \mathbb{R}^3$  :

$$\nabla_{\mathbf{u}} \bar{d}_l = \begin{bmatrix} \sum_{a,b,c} w_{abc}^{(l)} B'_a(u_x) B_b(u_y) B_c(u_z) \\ \sum_{a,b,c} w_{abc}^{(l)} B_a(u_x) B'_b(u_y) B_c(u_z) \\ \sum_{a,b,c} w_{abc}^{(l)} B_a(u_x) B_b(u_y) B'_c(u_z) \end{bmatrix}.$$

**Composition.** La mise à l'échelle métrique  $d_{l,i} = s_l \bar{d}_l$  se simplifie avec le facteur  $1/(2s_l)$  de la normalisation, de sorte que le gradient spatial métrique se réduit à  $\nabla_{\mathbf{x}} d_{l,i} = \frac{1}{2} \nabla_{\mathbf{u}} \bar{d}_l$ . Cette expression vaut pour les points dont la coordonnée normalisée reste dans le cube  $[-1, 1]^3$  ; pour le prolongement minorant  $\hat{d} = \sqrt{d(\mathbf{c})^2 + \|\mathbf{p} - \mathbf{c}\|^2}$  appliqué hors de ce domaine (Section 4.7.2), la dérivation directe de la composition donne

$$\nabla \hat{d} = \frac{d(\mathbf{c}) \nabla d(\mathbf{c})|_{\text{int}} + (\mathbf{p} - \mathbf{c})}{\hat{d}},$$

soit la combinaison, pondérée par  $1/\hat{d}$ , de la composante polynomiale restreinte aux axes intérieurs au cube et du vecteur d'excursion : toutes les composantes sont orientées vers l'extérieur, et le raccord à la frontière du domaine possède la régularité par morceaux déjà discutée à la Section 4.7.2. Enfin, plutôt que d'assembler explicitement  $\mathbf{J}_{x_{l,i}}$  pour chacun des points, ce qui matérialiserait un tenseur coûteux, les mêmes identités du produit vectoriel réécrivent  $\nabla_{\mathbf{q}} d_{l,i} = \mathbf{J}_{x_{l,i}}^\top \nabla_{\mathbf{x}} d_{l,i}$  directement dans le repère de base. En notant  $\mathbf{g}_{l,i} = \mathbf{R}_l^b \nabla_{\mathbf{x}} d_{l,i}$  le gradient spatial du SDF exprimé dans  $\mathcal{F}_b$  et  $\mathbf{r}_{l,i} = \mathbf{p}_i - \mathbf{d}_l^b$  le vecteur reliant l'origine du repère du lien à l'obstacle :

$$\boxed{\nabla_{\mathbf{q}} d_{l,i} = -(\mathbf{J}_{L,l}^b)^\top \mathbf{g}_{l,i} - (\mathbf{J}_{A,l}^b)^\top (\mathbf{r}_{l,i} \times \mathbf{g}_{l,i}),}$$

forme retenue dans l'implémentation, qui ne requiert que deux produits matrice-vecteur et un produit vectoriel par lien. Il reste à propager ce résultat à travers l'agrégation soft-min, conduite indépendamment pour chaque lien : la dérivation du Log-Sum-Exp fait apparaître



les poids  $\omega_i = \text{softmax}_i(-d_{l,i}/\alpha)$ , sélecteur lisse du point obstacle dominant, et le gradient de la barrière du lien  $l$  en est la combinaison convexe :

$$\nabla_{\mathbf{q}} h_l = \sum_{i=1}^N \omega_i \nabla_{\mathbf{q}} d_{l,i}.$$

Chaque facteur étant la dérivée exacte d'un polynôme ou d'une transformation rigide, ce gradient est obtenu sans aucune approximation numérique ; une seule évaluation vectorisée produit conjointement les barrières  $h_l$  et les gradients  $\nabla_{\mathbf{q}} h_l$  de tous les liens protégés, qui alimentent directement les lignes de la contrainte de sécurité résolue à la Section 4.7.5. Le soft-min lisse la direction de chaque gradient lors des commutations du point dominant, au prix du biais déjà mentionné. Le Tableau 4.3 compare le coût de cette forme close à celui de la différentiation automatique (`torch.autograd` de PyTorch) pour l'évaluation conjointe des barrières et de la Jacobienne de contrainte complète.

Tableau 4.3 Temps d'évaluation conjointe des barrières  $h_l$  et de la Jacobienne de contrainte complète ( $L \times 7$ ) pour 100 points obstacles, en fonction du nombre  $L$  de liens protégés inclus : gradient analytique en forme close contre différentiation automatique (moyenne  $\pm$  écart-type en ms, 100 répétitions), avec l'erreur absolue maximale entre les deux Jacobiennes.

| Liens $L$ | Analytique [ms]   | Diff. automatique [ms] | Erreur max. gradient   |
|-----------|-------------------|------------------------|------------------------|
| 1         | $5.596 \pm 0.413$ | $7.180 \pm 0.113$      | $5.960 \times 10^{-8}$ |
| 2         | $5.549 \pm 0.345$ | $8.010 \pm 0.389$      | $2.235 \times 10^{-7}$ |
| 3         | $5.432 \pm 0.149$ | $8.836 \pm 0.457$      | $2.373 \times 10^{-7}$ |
| 4         | $5.444 \pm 0.106$ | $9.789 \pm 0.127$      | $2.373 \times 10^{-7}$ |
| 5         | $5.538 \pm 0.398$ | $10.349 \pm 0.321$     | $2.980 \times 10^{-7}$ |
| 6         | $7.644 \pm 0.156$ | $12.257 \pm 0.517$     | $2.980 \times 10^{-7}$ |
| 7         | $7.792 \pm 0.315$ | $12.899 \pm 0.288$     | $2.980 \times 10^{-7}$ |
| 8         | $7.698 \pm 0.172$ | $13.479 \pm 0.233$     | $2.980 \times 10^{-7}$ |

#### 4.7.4 Contrainte de sécurité et vitesse nominale

Le problème (4.10) requiert encore ses deux entrées : le second membre  $b(h)$ , qui délimite les vitesses admissibles, puis la vitesse nominale  $\dot{\mathbf{q}}_{\text{base}}$ , centre de l'objectif.

##### Second membre saturé de la contrainte

Pour la première, la condition CBF classique impose  $\nabla h^\top \dot{\mathbf{q}} \geq -\kappa h$ . Avec une barrière issue de la perception, le terme linéaire  $\kappa h$  présente deux inconvénients pratiques : loin de l'obstacle

il autorise des vitesses d'approche arbitrairement grandes, et en violation il exige des vitesses de récupération proportionnelles à la pénétration, amplifiant le bruit de  $h$ . La contrainte implémentée sature ces deux régimes :

$$\nabla h(\mathbf{q})^\top \dot{\mathbf{q}} \geq b(h), \quad b(h) = m_{\text{CBF}} + \begin{cases} -v_{\text{in}} \min(h/h_{\text{act}}, 1), & h > 0, \\ v_{\text{rec}} \min(-h/d_{\text{rec}}, 1), & h \leq 0, \end{cases}$$

la contrainte n'étant évaluée que dans la bande d'activation  $h \leq h_{\text{act}}$ . Loin de la frontière, la vitesse d'approche normale est ainsi bornée par  $v_{\text{in}}$  ; à la frontière ( $h = 0$ ), toute vitesse normale entrante est interdite ; en violation, une vitesse de sortie proactive, croissant avec la pénétration jusqu'à  $v_{\text{rec}}$  à la profondeur de saturation  $d_{\text{rec}}$ , est exigée. Cette récupération proactive est complétée par l'étape de réparation décrite à la Section 4.7.5, qui réévalue cette même contrainte saturée sur la commande filtrée et corrige la violation résiduelle que la saturation et le filtrage réintroduisent. Cette formulation appartient à la famille  $\nabla h^\top \dot{\mathbf{q}} \geq -\gamma(h)$ , où le statut de  $\gamma$  mérite d'être énoncé avec précision : les saturations font que  $\gamma$  n'est *pas* globalement une fonction de classe  $\mathcal{K}$  étendue (la croissance stricte y est perdue sur les plateaux), mais restreinte à la bande d'exploitation  $[-d_{\text{rec}}, h_{\text{act}}]$ , elle en est exactement une : continue, strictement croissante, nulle en zéro, linéaire par morceaux de pentes  $v_{\text{in}}/h_{\text{act}}$  et  $v_{\text{rec}}/d_{\text{rec}}$ . Cela suffit au certificat : le théorème d'invariance [88] ne requiert la condition que sur un voisinage de la frontière, dont la bande d'activation est un ; au-dessus de  $h_{\text{act}}$  la contrainte n'est simplement pas évaluée, et sous  $-d_{\text{rec}}$  l'exigence constante  $v_{\text{rec}}$  substitue à la récupération de classe  $\mathcal{K}$ , non bornée et donc incompatible tant avec les limites articulaires qu'avec le bruit de  $h$  en profondeur, un retour *en temps fini* dans la bande à vitesse de sortie garantie. Le resserrement numérique renforce enfin strictement la condition : puisque  $b(0) = m_{\text{CBF}} > 0$ , la condition implémentée exige une vitesse normale strictement sortante à la frontière, soit une condition strictement plus forte que la condition standard ( $\gamma(0) = 0$ ), jamais plus faible.

### Resserrement temporel face aux obstacles mobiles

Telle quelle, cette contrainte suppose un environnement immobile. La condition d'invariance porte en effet sur la dérivée totale de la barrière,  $\dot{h}_l = \nabla h_l^\top \dot{\mathbf{q}} + \partial h_l / \partial t$ , dont le second terme est la contribution propre du mouvement des obstacles ; le second membre  $b(h_l)$  ne borne que le premier. Un obstacle qui approche consomme donc la bande de freinage à un rythme que la commande ne contrôle pas. Ce terme temporel est estimé en ligne, sans modèle de mouvement, comme le résidu entre la variation mesurée de chaque barrière et la

part imputable au déplacement du robot lui-même :

$$\hat{d}_{l,k} = \frac{h_l(t_k) - h_l(t_{k-1})}{\Delta t} - \nabla h_l^\top \frac{\mathbf{q}_k - \mathbf{q}_{k-1}}{\Delta t},$$

puis lissé par un filtre exponentiel de constante  $\tau_{\text{env}}$  en une estimée  $\bar{d}_l$ . L'estimateur est remis à zéro dès qu'un échantillon est invalide, soit qu'une variation instantanée invraisemblable trahisse une commutation de la sélection des points critiques, soit que le lien se trouve loin de tout obstacle : un resserrement périmé ne doit pas survivre à sa cause. Chaque demi-espace actif est alors resserré du terme

$$\eta_l = \text{clip}\left(-\bar{d}_l - \varepsilon_{\text{env}}, 0, \eta_{\text{max}}\right), \quad \nabla h_l^\top \dot{\mathbf{q}} \geq b(h_l) + \eta_l,$$

qui exige du robot qu'il cède le passage proportionnellement à la vitesse d'approche estimée pendant qu'il se trouve encore à l'extérieur de la marge, au lieu de ne réagir qu'à la décroissance constatée de  $h_l$  ; la réévaluation finale de la Section 4.7.5 applique le même second membre resserré. L'estimation est conduite par lien, et non sur le minimum global : un obstacle approchant du coude est détecté même lorsque l'outil, au contact de la cible, maintient le minimum global près de zéro, et le resserrement ne s'applique qu'à la contrainte du lien concerné. Dans une scène immobile,  $\bar{d}_l \approx 0$  et la zone morte  $\varepsilon_{\text{env}}$  annule exactement le terme : la contrainte quasi statique originale est retrouvée et le mécanisme est inerte.

### Vitesse nominale filtrée

La seconde entrée du problème (4.10) est la vitesse que le système *souhaite* exécuter. Celle-ci combine le feedforward retimé  $\dot{\mathbf{q}}_{\text{ff}}$  (Section 4.6.3) et une correction de suivi proportionnelle saturée :

$$\dot{\mathbf{q}}_{\text{fb}}^0 = \text{sat}\left(K_p(\mathbf{q}_{\text{nom}} - \mathbf{q}_{\text{cur}}), \dot{q}_{\text{fb,max}}\right).$$

À proximité de la frontière, ce terme de suivi est cependant restreint à sa composante tangentielle d'avancement. En effet, lorsque le filtre dévie le robot de la trajectoire nominale pour des raisons de sécurité, un retour proportionnel complet pointe directement vers la trajectoire (c'est-à-dire contre la correction de sécurité) et les deux termes s'annulent en régime permanent, immobilisant le robot contre la frontière. Avec  $\hat{\mathbf{t}} = \dot{\mathbf{q}}_{\text{ff}} / \|\dot{\mathbf{q}}_{\text{ff}}\|$  la direction d'avancement nominale :

$$\dot{\mathbf{q}}_{\text{fb}} = \begin{cases} \max\left(0, \hat{\mathbf{t}}^\top \dot{\mathbf{q}}_{\text{fb}}^0\right) \hat{\mathbf{t}}, & h \leq h_{\text{act}}, \\ \dot{\mathbf{q}}_{\text{fb}}^0, & \text{sinon.} \end{cases}$$

La vitesse nominale brute  $\dot{\mathbf{q}}_{\text{raw}} = \dot{\mathbf{q}}_{\text{ff}} + \dot{\mathbf{q}}_{\text{fb}}$  est ensuite lissée par un filtre passe-bas de constante  $\tau_{\text{nom}}$  :

$$\gamma_{\text{nom}} = \frac{\Delta t}{\Delta t + \tau_{\text{nom}}}, \quad \dot{\mathbf{q}}_{\text{base},k} = (1 - \gamma_{\text{nom}}) \dot{\mathbf{q}}_{\text{base},k-1} + \gamma_{\text{nom}} \dot{\mathbf{q}}_{\text{raw},k},$$

puis saturée aux limites de vitesse articulaire et annulée sous un seuil de maintien (évitant toute dérive en station). Le résultat  $\dot{\mathbf{q}}_{\text{base}}$  est la *vitesse nominale filtrée* : c'est l'entrée du filtre de sécurité, et la référence par rapport à laquelle toutes les interventions de celui-ci sont mesurées au Chapitre 5.

#### 4.7.5 De la vitesse nominale à la commande sécurisée

Les entrées du problème (4.10) sont désormais toutes définies : les barrières et leurs gradients (Sections 4.7.2 et 4.7.3), le second membre resserré et la vitesse nominale filtrée  $\dot{\mathbf{q}}_{\text{base}}$  (Section 4.7.4). Il reste à le résoudre, puis à transformer sa solution en commande exécutable.

#### Projection sur les demi-espaces actifs

La projection CBF recherche la vitesse admissible la plus proche de  $\dot{\mathbf{q}}_{\text{base}}$  au sens de la métrique  $\mathbf{W}$ , c'est-à-dire sa projection sur l'intersection des demi-espaces actifs (4.10) :

$$\dot{\mathbf{q}}_{\text{proj}} = \arg \min_{\dot{\mathbf{q}}} \frac{1}{2} \|\dot{\mathbf{q}} - \dot{\mathbf{q}}_{\text{base}}\|_{\mathbf{W}}^2 \quad \text{s.c.} \quad \nabla h_l^\top \dot{\mathbf{q}} \geq b(h_l) + \eta_l, \quad \forall l \in \mathcal{A}.$$

La correction de sécurité  $\Delta \dot{\mathbf{q}}_{\text{CBF}} \equiv \dot{\mathbf{q}}_{\text{proj}} - \dot{\mathbf{q}}_{\text{base}}$  n'est pas une entrée du calcul mais la quantité *dérivée* qui mesure l'intervention du filtre. Cette dernière est nulle dès que toutes les contraintes sont satisfaites et de norme égale à l'intensité de la correction.

La brique élémentaire de la résolution est la projection sur un seul demi-espace, qui admet une forme close pour toute métrique  $\mathbf{W}$ . Pour le lien  $l$  :

$$\dot{\mathbf{q}} \leftarrow \dot{\mathbf{q}} + \omega \frac{\max(0, b(h_l) + \eta_l - \nabla h_l^\top \dot{\mathbf{q}})}{\max(\nabla h_l^\top \mathbf{W}^{-1} \nabla h_l, \varepsilon_0)} \mathbf{W}^{-1} \nabla h_l.$$

Si la contrainte du lien est déjà respectée, le terme  $\max(0, \cdot)$  s'annule et la vitesse reste inchangée ; sinon, elle est déplacée le long de  $\mathbf{W}^{-1} \nabla h_l$  pour ramener le produit  $\nabla h_l^\top \dot{\mathbf{q}}$  sur sa borne, le facteur de relaxation  $\omega$  pondérant ce pas. Quelle que soit la métrique, le demi-espace est satisfait exactement dès que  $\nabla h_l^\top \mathbf{W}^{-1} \nabla h_l > \varepsilon_0$  : le plancher numérique  $\varepsilon_0 = 10^{-8}$  ne mord que pour des gradients dégénérés, par ailleurs écartés par le seuil de fiabilité  $g_{\text{min}}$  de la réparation, et  $\mathbf{W}$  ne modifie que la direction articulaire dans laquelle la correction est réalisée. Lorsqu'un *seul* lien est actif, une application de cette mise à jour avec  $\omega = 1$  résout

donc exactement (4.10). Une variante antérieure régularisait le dénominateur additivement ( $+\varepsilon$ ,  $\varepsilon = 10^{-4}$ ), laissant subsister un déficit de contrainte de l'ordre d'une fraction  $\varepsilon$  de la correction demandée, couvert par la marge numérique  $m_{\text{CBF}}$  ; elle est abandonnée au profit du plancher, la résolution multi-contraintes de la section suivante exigeant des projections élémentaires exactes.

## Métrie de tâche

Le choix de  $\mathbf{W}$  répond à une question que la formulation euclidienne tranche mal : quel mouvement sacrifier quand la sécurité exige de dévier de l'intention du modèle. Avec  $\mathbf{W} = \mathbf{I}$ , la correction est colinéaire à  $\nabla h_l$  et se dépose sur les articulations offrant le plus grand bras de levier sur la barrière, sans égard à leur effet sur la tâche : lorsqu'un lien du corps (typiquement le coude) rencontre un obstacle, c'est l'épaule entière qui recule et, avec elle, l'effecteur : la trajectoire exécutée du lien contrôlé s'écarte fortement de la trajectoire nominale alors même que la contrainte ne portait pas sur lui. La métrie de tâche retenue mesure au contraire le coût d'une vitesse articulaire par le déplacement qu'elle inflige au lien contrôlé :

$$\mathbf{W} = \mathbf{J}_p^\top \mathbf{J}_p + w_{\text{rot}} \mathbf{J}_o^\top \mathbf{J}_o + \lambda \mathbf{I}, \quad \|\dot{\mathbf{q}}\|_{\mathbf{W}}^2 = \|\mathbf{J}_p \dot{\mathbf{q}}\|^2 + w_{\text{rot}} \|\mathbf{J}_o \dot{\mathbf{q}}\|^2 + \lambda \|\dot{\mathbf{q}}\|^2,$$

où  $\mathbf{J}_p$  et  $\mathbf{J}_o$  sont les blocs linéaire et angulaire de la Jacobienne géométrique du lien contrôlé. Parmi toutes les vitesses satisfaisant le même demi-espace, la projection retient alors celle qui déplace le moins le lien contrôlé : la correction se résout, autant que la cinématique le permet, dans le noyau de la Jacobienne de tâche, le même levier de redondance que la cinématique inverse exploite pour sa tâche secondaire. Le poids  $w_{\text{rot}} = 0,01$  règle l'arbitrage position–orientation (un radian de rotation du repère de tâche y coûte autant que dix centimètres de translation) et la régularisation  $\lambda = 0,01$  maintient  $\mathbf{W}$  inversible en attribuant un coût résiduel aux mouvements du noyau, faible devant le coût de tâche. La métrie est un préconditionneur lentement variable : elle est rafraîchie à 30 Hz dans le fil de perception, par différences finies sur la cinématique directe, et la boucle de commande à 100 Hz la lit sans synchronisation, l'obsolescence maximale ( $\approx 33$  ms, soit moins de 0,025 rad de mouvement articulaire) étant sans effet mesurable sur la projection.

## Résolution multi-contraintes par projections de Dykstra

Avec plusieurs liens actifs, l'intersection des demi-espaces n'admet plus de forme fermée. Le problème (4.10) demeure un programme quadratique qu'un solveur générique résoudrait exactement [92], mais au prix d'une structure de calcul à itérations et branchements variables,

incompatible avec la capture en graphe CUDA décrite ci-dessous. La méthode des projections cycliques (POCS, Section 2.4.2) offre cette forme fixe : la projection sur le demi-espace d'un lien pouvant re-violer celui d'un lien déjà traité, l'ensemble  $\mathcal{A}$  est balayé à plusieurs reprises à la manière de Gauss–Seidel et, les gradients étant figés à la configuration courante, la suite des projections converge vers un point de l'intersection [94]. Ce point n'est cependant pas, en général, la projection de  $\dot{\mathbf{q}}_{\text{base}}$  au sens de  $\mathbf{W}$  : l'excès de correction déposé par une projection précoce n'est jamais restitué lorsque les contraintes suivantes le rendent superflu, et le point atteint dépend de l'ordre de balayage. Une vérification numérique hors ligne sur des instances aléatoires à contraintes multiples simultanément actives (solution exacte obtenue par énumération des ensembles actifs des conditions KKT) chiffre cet excès : le coût de déviation des projections cycliques dépasse celui de la projection exacte de 6,2 % en médiane et de 23 % au 90<sup>e</sup> centile, écart permanent qu'aucun balayage supplémentaire ne résorbe. Le filtre déployé emploie donc la variante de Dykstra [95,96], équivalente pour des demi-espaces à la méthode duale de Hildreth [97] : chaque contrainte  $l$  conserve un vecteur de correction mémorisé  $\mathbf{z}_l$ , réinjecté avant chaque nouvelle projection,

$$\dot{\mathbf{q}}' = \dot{\mathbf{q}} + \mathbf{z}_l, \quad \dot{\mathbf{q}} \leftarrow P_l(\dot{\mathbf{q}}'), \quad \mathbf{z}_l \leftarrow \dot{\mathbf{q}}' - P_l(\dot{\mathbf{q}}'),$$

où  $P_l$  est la projection élémentaire en forme close ci-dessus. La suite converge alors vers la projection *exacte* de  $\dot{\mathbf{q}}_{\text{base}}$  sur l'intersection au sens de  $\mathbf{W}$ , indépendante de l'ordre de balayage, et coïncide avec les projections cycliques simples dès qu'au plus une contrainte est active (les  $\mathbf{z}_l$  demeurent nuls), le cas le plus fréquent en pratique. Deux conditions d'implémentation, non stylistiques, s'y attachent. D'une part, les projections élémentaires doivent être exactes : avec le dénominateur régularisé additivement, chaque projection sous-corrige d'une fraction fixe que les balayages cycliques résorbent d'eux-mêmes en re-projetant le résidu, mais que la mémoire de Dykstra (qui restitue la correction avant de re-projeter) reproduit à l'identique à chaque balayage, figeant l'itéré en deçà du demi-espace ; d'où le dénominateur exact à plancher retenu plus haut. D'autre part, contrairement à ce qu'une lecture rapide suggère, cette variante ne sacrifie pas la forme fixe : la mémoire se loge dans un tenseur de forme constante  $(K_a+1) \times 7$ , remis à zéro en tête de chaque résolution, et un lien déjà satisfait laissant  $\dot{\mathbf{q}}$  et  $\mathbf{z}_l$  inchangés, compléter  $\mathcal{A}$  à un effectif fixe demeure sans effet sur le point de convergence. La sélection *top- $K_a$*  et la boucle de  $N_{\text{it}}$  balayages conservent donc leur structure sans branchement ni masquage dynamique, capturée une seule fois dans un graphe CUDA puis exécutée intégralement sur le GPU à chaque cycle de commande, pour un surcoût d'environ deux opérations vectorielles par projection. La boucle demeure tronquée à  $N_{\text{it}}$  balayages : l'admissibilité en sortie de boucle n'est qu'approchée, et la sécurité de la commande émise

continue de reposer non sur cette convergence, mais sur la réévaluation finale décrite plus bas, qui réimpose la contrainte sur la vitesse réellement publiée.

### Saturation, lissage et réparation de la commande

La vitesse projetée est ensuite saturée aux limites articulaires puis lissée par un filtre passe-bas exponentiel de constante  $\tau_{\text{safe}}$ , la saturation précédant le filtrage afin que celui-ci ne considère jamais une vitesse physiquement irréalisable. La troncature des balayages, la saturation et le lissage pouvant laisser subsister ou réintroduire une légère violation de la contrainte, celle-ci est réévaluée sur la vitesse filtrée  $\dot{\mathbf{q}}_f$ , avec le même second membre saturé et resserré  $b(h) + \eta$  que la projection principale (Section 4.7.4) ; toute violation résiduelle est corrigée par une réparation  $\Delta\dot{\mathbf{q}}_{\text{rep}}$ , projection corrective sur la même contrainte linéarisée et dans la même métrique  $\mathbf{W}$ , qui répartit donc sa correction de la même manière que le solveur au lieu de défaire, par un pas euclidien, la préférence pour le noyau de tâche. Deux garde-fous la bornent : sa norme est plafonnée ( $\Delta v_{\text{max}}$  tant que la barrière est positive, plafond distinct  $\Delta v_{\text{max}}^{\text{rec}}$ , plus permissif, lorsque  $h < 0$  et qu’une sortie rapide de la zone de violation devient prioritaire), et son amplitude est atténuée lorsque la norme du gradient passe sous le seuil de fiabilité  $g_{\text{min}}$  : le cas de l’obstacle mince, où les gradients des points situés de part et d’autre du lien se compensent dans le soft-min et où la direction  $\nabla h / \|\nabla h\|$  perd son sens géométrique, la violation résiduelle étant alors résorbée aux cycles suivants. Hors de ces deux régimes dégradés, l’admissibilité de la commande émise est ainsi une propriété *vérifiée en bout de chaîne*, sur la vitesse réellement publiée, et non une conséquence supposée de la convergence des balayages. Les formes closes du lissage et de la réparation, ainsi que le détail de ces garde-fous, sont donnés à l’Annexe D. La vitesse finale envoyée à la commande est donc

$$\dot{\mathbf{q}}_{\text{final}} = \text{sat}(\dot{\mathbf{q}}_f + \Delta\dot{\mathbf{q}}_{\text{rep}}).$$

Cette saturation finale, nécessaire au respect des limites articulaires, peut à son tour écrêter la réparation ; c’est pourquoi le résidu de contrainte est réévalué une dernière fois sur la commande effectivement publiée, *après* saturation, et exporté à chaque cycle comme diagnostic (le signal `constr_final` du Chapitre 5) : un écrêtage résiduel est ainsi mesuré plutôt que supposé absent, et sa résorption s’opère aux cycles suivants. Intégrer les bornes articulaires comme contraintes du problème (4.10) en supprimerait la cause, au prix de balayages supplémentaires ; la voie retenue est cette vérification en sortie.

## Interface avec le contrôleur de vitesse

Le contrôleur bas niveau du robot étant un contrôleur de *vitesse* articulaire, cette vitesse certifiée constitue directement la commande envoyée à l'actionneur, sans étape d'intégration en une consigne de position : aucune étape logicielle de conversion ne s'interpose entre la projection et l'actionneur, et la garantie en temps discret se réduit au seul maintien de cette vitesse pendant une période de commande (bloqueur d'ordre zéro), comptabilisé dans la marge  $d_{\text{safe}}$  (Section 4.7.7) et surveillé activement entre les résolutions par le frein réflexe décrit ci-dessous (Section 4.7.5). La dynamique interne de la boucle de vitesse bas niveau (bande passante, retard de poursuite) demeure quant à elle hors du certificat : elle relève de l'hypothèse de suivi explicitée à la Section 4.7.7. Une référence de position intégrée est maintenue en parallèle à des fins de supervision, mais elle n'est pas l'entrée du contrôleur ; lorsque la contrainte est inactive et que la projection ne modifie pas la vitesse nominale,  $\dot{\mathbf{q}}_{\text{final}}$  se réduit à la vitesse nominale lissée, transmise telle quelle. La fidélité de cette interface, c'est-à-dire le fait que le taux de variation de barrière commandé  $\nabla h^\top \dot{\mathbf{q}}_{\text{final}}$  est effectivement réalisé par le contrôleur, est vérifiée expérimentalement au Chapitre 5.

## Frein réflexe entre les instants de résolution

Le certificat du filtre est instantané : la projection garantit  $\nabla h^\top \dot{\mathbf{q}} \geq b(h) + \eta$  à l'instant de résolution, puis la commande est maintenue par le bloqueur d'ordre zéro pendant toute la période suivante, durant laquelle  $h$  évolue sans surveillance. Loin de la frontière, la marge  $d_{\text{safe}}$  absorbe cette érosion ; en revanche, les phases où la tâche impose de *longer* la frontière ( $h$  de quelques millimètres pendant plusieurs secondes, régime typique du transport d'objet près d'une paroi) consomment le certificat à chaque maintien, d'autant plus que la cadence effective de résolution fléchit sous la charge du GPU partagé. Plutôt que d'immobiliser en permanence cette réserve dans la marge, une seconde couche, le *frein réflexe*, re-certifie la commande maintenue *entre* les résolutions : une architecture multi-cadence au sens de [123], où la décision lourde (sélection des points critiques, gradients, projection) demeure à la cadence du solveur tandis qu'un frein léger, quelques opérations vectorielles par ligne, tourne à  $\sim 1$  kHz entre le filtre et le contrôleur.

À chaque tique, le frein met à l'échelle la commande maintenue par un unique scalaire  $\alpha \in [0, 1]$ ,  $\dot{\mathbf{q}}_{\text{exec}} = \alpha \dot{\mathbf{q}}_{\text{final}}$  : un frein pur, qui préserve le chemin articulaire et n'en module que la cadence de parcours (l'analogie articulaire de la surveillance de vitesse et de séparation normalisée en robotique collaborative [124]), sans jamais amplifier ni dévier la commande. Le scalaire retenu est le plus grand qui demeure certifié sûr sur l'horizon de maintien  $T_{\text{rfx}}$ , dans l'esprit des conditions CBF échantillonnées de Breeden et al. [103] : chaque ligne de gradient



étant supposée  $L$ -lipschitzienne en  $\mathbf{q}$ , la borne de Taylor

$$h(\mathbf{q}_0 + \mathbf{d}) \geq h(\mathbf{q}_0) + \nabla h^\top \mathbf{d} - \frac{L}{2} \|\mathbf{d}\|^2$$

minore la barrière le long du segment rectiligne que parcourt, en commande de vitesse, l'état sous bloqueur d'ordre zéro ; cette minorante étant concave le long du segment, sa vérification à l'extrémité certifie l'intervalle entier. Il en résulte, par ligne active, une inéquation scalaire du second degré en  $\alpha$  dont la plus grande racine admissible est en forme close et restitue l'extrapolation linéaire  $\alpha_l = (h_l - h_{\text{stop}})/(-\dot{h}_l T_{\text{rfx}})$  lorsque  $L \rightarrow 0$  ;  $\alpha$  est le minimum sur les lignes, la dérive mesurée depuis l'instant de résolution étant comptée dans la borne. Le frein s'engage instantanément ( $\alpha$  adopte sans délai toute valeur inférieure) et se relâche par filtre exponentiel de constante  $\tau_{\text{rel}}$ , l'asymétrie reflétant celle des coûts : freiner tard est une erreur de sécurité, relâcher tard un simple coût de performance. Les lignes déjà en violation et commandées en sortie ne sont jamais freinées, un frein pur ne devant pas annuler une vitesse de récupération. Le certificat vaut sous quatre hypothèses explicites : suivi exact de la vitesse entre les tiques, validité locale de la constante  $L$  (surveillée en ligne par un estimateur empirique publié à chaque mise à jour, dont les dépassements transitoires signent les commutations de point critique), exactitude de la valeur de barrière transmise, et majoration du terme d'environnement. Chaque écart dégrade la garantie en estimation conservatrice plutôt qu'en certificat. Cette couche a une conséquence assumée sur la lecture des résultats : lorsque le certificat expire en cours de maintien, la vitesse exécutée n'est plus  $\dot{\mathbf{q}}_{\text{final}}$  mais  $\alpha \dot{\mathbf{q}}_{\text{final}}$ , et l'écart mesuré entre les deux n'est pas une erreur de suivi mais l'intervention du frein, exportée cycle par cycle par le signal  $\alpha$  ; la fidélité du contrôleur s'évalue donc contre la commande réellement émise.

#### 4.7.6 Évitement des minima locaux du filtre

La projection est minimale par construction : elle annule exactement la composante de vitesse que la borne interdit, et ne recherche jamais davantage de dégagement que la contrainte n'en exige. Cependant, le modèle génératif étant aveugle aux obstacles, une trajectoire nominale générée alors que le robot est pris dans un minimum local réémet le même ordre à chaque replanification. L'équilibre entre cette intention et la correction qui l'annule immobilise alors le robot sur la frontière  $h \approx 0$  : la sécurité est préservée, mais la tâche ne progresse plus. Deux situations de blocage sont distinguées, selon que la contrainte active porte sur un lien du corps ou sur le lien contrôlé lui-même. Seule la première est traitée activement, par le dégagement postural ci-dessous, ajouté à la vitesse nominale en amont de la projection : chaque demi-espace actif le filtre au même titre que l'intention du modèle, de sorte qu'il

ne peut pas dégrader la sécurité. La seconde est délibérément laissée sans mécanisme de déblocage : l'arrêt sûr qui en résulte est assumé comme limite du filtre (Section 4.7.7) et documenté au Chapitre 5.

### Dégagement postural

Le dégagement postural traite les contraintes portées par le corps du bras en exploitant la même redondance que la métrique de tâche : le manipulateur disposant d'un noyau de dimension un autour de la Jacobienne 6D du lien contrôlé, la posture peut se déformer sans que la tâche n'en souffre. La vitesse injectée pousse le corps dans la direction du noyau qui accroît la barrière critique,

$$\dot{\mathbf{q}}_{\text{clear}} = k_{\text{clear}} \rho(h) a \frac{\mathbf{N} \nabla h}{\|\mathbf{N} \nabla h\|}, \quad \mathbf{N} = \mathbf{I} - \mathbf{J}^+ \mathbf{J},$$

où  $\mathbf{N}$  est le projecteur amorti sur le noyau de la Jacobienne complète du lien contrôlé,  $\rho(h) = \text{clip}((h_{\text{ns}} - h)/h_{\text{ns}}, 0, 1)$  une rampe de proximité active dans la bande  $h < h_{\text{ns}}$  et  $a = \min(\|\mathbf{N} \nabla h\|/(0,3 \|\nabla h\|), 1)$  une rampe d'autorité proportionnelle à la fraction du gradient contenue dans le noyau. Le bras s'écarte ainsi de l'obstacle par pivot du coude, à pose du lien contrôlé inchangée au premier ordre, *tout en continuant de suivre* la trajectoire de tâche au lieu de se garer contre la frontière ; la rampe  $a$  éteint le terme précisément lorsque la contrainte porte sur le lien contrôlé et qu'aucune déformation posturale ne peut la desserrer.

### 4.7.7 Portée des garanties

#### Hypothèses et écarts d'implémentation

Au-delà du solveur lui-même, il reste à délimiter ce que le filtre garantit réellement. L'invariance avant théorique de l'ensemble  $\mathcal{C}$  suppose une dynamique en temps continu, un SDF exact, des obstacles correctement observés et immobiles, et un suivi parfait de la commande par le contrôleur bas niveau. L'implémentation s'écarte de ces hypothèses sur trois points quantifiables : la discrétisation de la commande (100 Hz), l'obsolescence de l'ensemble des points critiques (mise à jour à 30 Hz) et l'erreur d'approximation du SDF de Bernstein (caractérisée au Chapitre 5). S'y ajoute le cas dégénéré du gradient effondré face aux obstacles minces, où la réparation finale est volontairement atténuée (seuil  $g_{\text{min}}$ , Section 4.7.5) et la résorption de la violation reportée aux cycles suivants. Quant à l'hypothèse d'immobilité, le resserrement temporel  $\eta_l$  (Section 4.7.4) la tempère sans la lever : il fait céder le robot proportionnellement à la vitesse d'approche estimée de l'environnement, mais repose sur une

estimation filtrée, retardée de la constante  $\tau_{\text{env}}$  et plafonnée à  $\eta_{\text{max}}$ , et ne constitue pas une garantie formelle vis-à-vis d'obstacles mobiles ; la garantie énoncée demeure celle de l'environnement quasi statique, l'apport du terme face à un obstacle lentement mobile étant évalué empiriquement au Chapitre 5.

### Certificat des deux modes de barrière

Le mode déployé n'appelle aucun raisonnement particulier : la valeur et le gradient provenant tous deux de  $h_{\text{soft}}$ , fonction lisse de  $\mathbf{q}$ , la condition appliquée est la condition CBF standard et l'invariance avant de  $\mathcal{C}_{\text{soft}} = \{h_{\text{soft}} \geq 0\}$  suit du théorème usuel. Puisque  $h_{\text{soft}} \leq h_{\text{min},l}$ , tout point de cet ensemble vérifie de surcroît  $\min_i d_{l,i} \geq d_{\text{safe}}$  : le certificat couvre la distance exacte, assorti d'une réserve supplémentaire  $\alpha \log M_{\text{eff}}$  qui croît précisément là où le nuage s'encombre. La variante à valeur exacte (Section 4.7.2) pourrait en revanche sembler sacrifier ce raisonnement : le vecteur employé comme gradient n'y est pas la dérivée du minimum exact transmis au second membre. Elle admet pourtant un certificat, qu'il suffit d'attribuer à la barrière compensée  $h_{\text{corr}} = h_{\text{soft}} + \alpha \log N$  déjà introduite. D'une part, le terme de compensation étant indépendant de  $\mathbf{q}$ ,  $\nabla h_{\text{corr}} = \nabla h_{\text{soft}}$  : le gradient implémenté est exactement celui de  $h_{\text{corr}}$ , qui est lisse. D'autre part, la valeur décisionnelle vérifie  $h_{\text{min},l} = \min_i d_{l,i} - d_{\text{safe}} \leq h_{\text{corr}}$  et le second membre  $b(\cdot)$  est monotone non croissant, de sorte que la condition appliquée,

$$\nabla h_{\text{soft}}^\top \dot{\mathbf{q}} \geq b(h_{\text{min},l}) \implies \nabla h_{\text{corr}}^\top \dot{\mathbf{q}} \geq b(h_{\text{corr}}),$$

implique la condition CBF standard, cohérente en valeur et en gradient, de la barrière  $h_{\text{corr}}$  ; la bande d'activation est couverte de même, puisque  $h_{\text{min},l} \leq h_{\text{corr}}$  garantit que la contrainte est évaluée dès que  $h_{\text{corr}} \leq h_{\text{act}}$ . L'invariance avant certifiée de cette variante porte donc sur l'ensemble  $\mathcal{C}_{\text{corr}} = \{h_{\text{corr}} \geq 0\}$ , dont tout point vérifie  $\min_i d_{l,i} \geq d_{\text{safe}} - \alpha \log N$ . Le prix de l'élimination de la conservativité n'est ainsi pas la perte du certificat, mais une réduction bornée de la marge certifiée, exactement égale au biais par lien :  $h_{\text{min},l} - h_{\text{soft},l} = \alpha \log M_{\text{eff}}$  avec  $M_{\text{eff}} = \sum_i \exp\left(-(d_{l,i} - m_l)/\alpha\right) \in [1, N]$ , mesure de l'encombrement effectif à moins de  $\alpha$  du minimum, bornée au pire par  $\alpha \log N \approx 9,2 \text{ mm}$  ( $\alpha = 0,002$ ,  $N = 100$ ).

### Nature de la garantie et dimensionnement de la marge

Ce filtrage réactif n'est pour autant pas un champ de potentiel répulsif : l'intention du modèle n'entre que dans l'objectif du problème (4.10), jamais dans ses contraintes, de sorte qu'aucune vitesse violant un demi-espace de sécurité n'est atteignable, quelle que soit l'intensité de cette intention. La garantie tient donc de la structure du filtre et non du réglage

d'une marge : c'est l'invariance avant de l'ensemble certifié ( $\mathcal{C}_{\text{soft}}$  pour le mode déployé,  $\mathcal{C}_{\text{corr}}$  pour la variante à valeur exacte), exacte en temps continu pour un ensemble fixe et fini de points obstacles observés, quelle que soit leur disposition géométrique, sous des hypothèses qu'il convient d'explicitier : (i) sélection de points et effectif  $N$  constants entre deux mises à jour ; (ii) intersection non vide des demi-espaces actifs avec les limites de vitesse articulaire (faisabilité) ; (iii) suivi exact de la vitesse commandée par la boucle bas niveau ; (iv) état initial dans l'ensemble certifié ; (v) environnement immobile. La mise à jour de la sélection à 30 Hz peut en particulier réintroduire des points déjà à l'intérieur de la marge : ce cas sort du certificat, est pris en charge par le régime de récupération et la réparation plafonnée (Annexe D), et son issue est évaluée empiriquement. La marge  $d_{\text{safe}}$  ne produit pas l'évitement, elle le robuste : elle éloigne la frontière certifiée de la surface de collision réelle, de sorte que les écarts d'implémentation énumérés ci-dessus se consomment dans cette réserve plutôt qu'en pénétration. Son dimensionnement (15 mm) est propre au cadre d'évaluation retenu : la validation se déroulant en simulation sans bruit de capteur, avec des nuages d'obstacles synthétiques échantillonnés sur la surface exacte des objets et des transformations de repères connues, les termes d'erreur perceptive (bruit de profondeur, calibration, quantification de carte) y sont nuls, et la réserve n'a à couvrir que la discrétisation de la commande (au plus  $v_{\text{in}} T = 3$  mm par période à 100 Hz, plus le reste de linéarisation dû à la courbure de  $h$  sur un pas), l'obsolescence de la sélection des points critiques ( $\approx 33$  ms) et l'erreur du SDF appris, auxquelles s'ajoute, pour la variante à valeur exacte seulement, la réduction  $\alpha \log M_{\text{eff}}$  du certificat (le mode déployé convertit ce même terme en réserve supplémentaire). Ce traitement de la discrétisation est celui, échantillonné-robuste, discuté au Chapitre 2 : la contrainte implémentée coïncide avec la linéarisation de la condition en temps discret de [102], dont le reste est transféré à la marge plutôt qu'à un programme non linéaire exact, incompatible avec la fréquence de commande pour une barrière apprise évaluée sur des milliers de points. Le frein réflexe (Section 4.7.5) complète ce traitement dans l'esprit échantillonné de [103] : la part du comportement inter-échantillons qui excéderait la provision statique est interceptée en ligne, par re-certification à  $\sim 1$  kHz de la commande maintenue, au lieu d'être dimensionnée au pire cas dans la marge.

Ce certificat porte enfin sur la barrière *apprise*, non sur la géométrie réelle : la distance physique hérite de l'erreur d'approximation du SDF. En régime établi, la contrainte maintient  $h_{\text{min}} \geq 0$ , soit une distance apprise d'au moins 15 mm et, au 99<sup>e</sup> centile de l'erreur du SDF (6,8 mm, Chapitre 5), une distance réelle d'au moins 8,2 mm. Dans le pire transitoire certifié de la variante à valeur exacte ( $h_{\text{corr}} = 0$ ), le plancher de 5,8 mm ne domine en revanche plus cette erreur de queue : la conjonction des deux pires cas ne peut être exclue par le seul calcul. La distinction est donc assumée : l'invariance est exacte vis-à-vis de la barrière apprise,

tandis que l'évitement physique est validé empiriquement, par la marge mesurée au maillage exact rapportée au Chapitre 5 ; un déploiement hors simulation exigerait d'augmenter  $d_{\text{safe}}$  du budget d'erreur perceptive correspondant.

### Sécurité et progression de la tâche

La portée de ces garanties est par ailleurs celle de la sécurité, non de la progression : l'invariance de  $\mathcal{C}$  vaut pour une géométrie d'obstacle arbitraire, tandis que le dégagement postural de la Section 4.7.6 n'étend la vivacité qu'aux blocages portés par le corps du bras, résolubles dans le noyau de la tâche. Lorsque l'obstacle barre le chemin du lien contrôlé lui-même, cas typiques de la paroi frontale et de la cavité concave faisant face à la trajectoire nominale, la projection annule la composante entrante de la vitesse, le modèle génératif, aveugle aux obstacles, réémet la même intention à chaque replanification, et le système demeure indéfiniment en arrêt sûr :  $h \geq 0$  est maintenu, mais la tâche ne progresse plus. Cette limite est assumée comme frontière du filtre plutôt que masquée par une heuristique d'exploration locale : sa résolution exige une information globale sur la topologie de la scène, qui relève de la couche de planification et non du filtrage réactif, et le comportement d'arrêt sûr correspondant est documenté comme cas d'échec au Chapitre 5.

Tableau 4.4 Paramètres du filtre de sécurité SDF-CBF.

| Symbole                                                          | Description                                           | Valeur                         |
|------------------------------------------------------------------|-------------------------------------------------------|--------------------------------|
| $d_{\text{safe}}$                                                | marge de sécurité du SDF                              | 15 mm                          |
| $\alpha$                                                         | température du soft-min                               | 0,002                          |
| $K$                                                              | points critiques sélectionnés                         | 100                            |
| $h_{\text{act}}$                                                 | bande d'activation de la contrainte                   | 40 mm                          |
| $v_{\text{in}}$                                                  | vitesse d'approche normale maximale ( $\dot{h}$ )     | 0,30 m/s                       |
| $v_{\text{rec}}$                                                 | vitesse de sortie exigée en violation ( $\dot{h}$ )   | 0,30 m/s                       |
| $d_{\text{rec}}$                                                 | profondeur de saturation en récupération              | 15 mm                          |
| $\Delta v_{\text{max}}$                                          | correction de projection et de réparation maximale    | 0,30 rad/s                     |
| $\Delta v_{\text{max}}^{\text{rec}}$                             | réparation maximale en violation ( $h < 0$ )          | 0,85 rad/s                     |
| $g_{\text{min}}$                                                 | seuil de fiabilité du gradient de réparation          | 0,03                           |
| $m_{\text{CBF}}$                                                 | marge numérique de la contrainte                      | 0,001 m/s                      |
| $K_p, \dot{q}_{\text{fb,max}}$                                   | suivi proportionnel et saturation                     | 5, 0,5 rad/s                   |
| $\tau_{\text{nom}}, \tau_{\text{safe}}$                          | constantes des filtres passe-bas                      | 0,08 s, 0,08 s                 |
| $\Delta q_{\text{lead}}$                                         | avance maximale de la référence                       | 0,05 rad                       |
| $K_a, N_{\text{it}}, \omega$                                     | liens constraints, balayages de Dykstra, relaxation   | 9, 6, 1,0                      |
| $f_{\text{rfx}}, T_{\text{rfx}}, h_{\text{stop}}$                | frein réflexe : cadence, horizon, seuil d'arrêt       | 1 kHz, 20 ms, 0                |
| $L, \tau_{\text{rel}}$                                           | frein réflexe : constante de Lipschitz, relâche       | 2,0 m/rad <sup>2</sup> , 50 ms |
| $\lambda, w_{\text{rot}}$                                        | métrique de tâche : régularisation, poids de rotation | 0,01, 0,01                     |
| $k_{\text{clear}}, h_{\text{ns}}$                                | dégagement postural : gain, bande d'activation        | 0,3 rad/s, 150 mm              |
| $\tau_{\text{env}}, \varepsilon_{\text{env}}, \eta_{\text{max}}$ | resserrement temporel : filtre, zone morte, plafond   | 0,1 s, 0,05 m/s, 0,5 m/s       |

## CHAPITRE 5 RÉSULTATS ET DISCUSSION

Ce chapitre présente la validation expérimentale du système, phase d'évaluation de SO5 et SO6. Plutôt que de dérouler les résultats tâche par tâche, il est organisé par question : chaque section apporte la réponse expérimentale à l'un des critères de validation du Chapitre 3, en s'appuyant sur le banc d'essai qui isole le mieux le mécanisme évalué. La Section 5.1 définit une seule fois les bancs d'essai, les scénarios et les métriques. La Section 5.2 valide les composants pris isolément : le modèle géométrique de sécurité, puis le planificateur nominal. La Section 5.3 répond à la question centrale, l'apport du filtre SDF-CBF, par une comparaison avec et sans filtre puis par deux études détaillées. La Section 5.4 isole la contribution de chaque mécanisme de conception par des ablations dédiées, et la Section 5.5 évalue le transfert de l'architecture à un bras différent. Les Sections 5.6 à 5.8 closent le chapitre par le budget temporel, les cas d'échec et la discussion.

### 5.1 Protocole expérimental

#### 5.1.1 Bancs d'essai

Trois bancs d'essai partagent la même instance du système : la chaîne de perception, le planificateur *Flow Matching*, l'exécuteur et le bouclier CBF du Chapitre 4 y sont déployés sans modification, seuls la tâche, le robot et la source du nuage d'obstacles changent (Tableau 5.1). Le banc d'alimentation exerce le système dans sa configuration complète : la caméra au poignet alimente la chaîne SAM 3/SAM 2 et la carte persistante, qui fournit au filtre ses points obstacles. Les deux bancs de saisie et dépôt remplacent cette perception embarquée par un nuage d'obstacles synthétique publié directement au filtre : la géométrie des obstacles y est contrôlée exactement, ce qui permet de composer les scénarios géométriques (prisme, croissant, sphère mobile) et d'attribuer chaque comportement au mécanisme évalué plutôt qu'au bruit de perception. Le troisième banc reprend la tâche de saisie et dépôt sur un bras xArm7 afin d'évaluer le transfert d'embodiment (Section 5.5). Conformément à la portée établie au Chapitre 3, tous les essais sont conduits en simulation Gazebo.

#### 5.1.2 Scénarios

Cinq scénarios sollicitent chacun un mécanisme précis du système ; le Tableau 5.2 indique quels bancs exécutent chacun d'eux. Le scénario *sans obstacle critique* constitue la référence : aucun obstacle n'approche la trajectoire nominale, et il vérifie la transparence du filtre (in-

Tableau 5.1 Bancs d’essai utilisés pour la validation expérimentale.

|                    | Alimentation      | Saisie et dépôt   | Saisie et dépôt          |
|--------------------|-------------------|-------------------|--------------------------|
| Robot              | Panda 7 DDL       | Panda 7 DDL       | xArm7                    |
| Outil / effecteur  | fourchette        | pince Franka      | pince Franka             |
| Caméra (D415)      | poignet           | statique          | statique                 |
| Source d’obstacles | carte persistante | nuage synthétique | nuage synthétique        |
| Politique FM       | alimentation      | saisie-dépôt      | saisie-dépôt (identique) |

terventions et déviation attendues nulles). Le scénario d’*obstacles statiques* place un obstacle convexe sur le chemin nominal de l’outil et force un contournement. L’*obstacle concave* (croissant) ouvre une poche autour du chemin nominal : il matérialise la limite de vivacité assumée du filtre (Section 4.7.7), dont le comportement d’arrêt sûr attendu est documenté parmi les cas d’échec. La *sphère mobile* traverse l’espace de travail à vitesse constante et sollicite le resserrement dynamique (Section 4.7.4) ; conformément à la portée des garanties, ce scénario évalue une robustification, sans revendication de garantie formelle. L’*occlusion* masque temporairement l’obstacle au champ de vision de la caméra au poignet pendant que le robot opère à proximité : il éprouve la mémoire spatiale de la carte persistante face à la perte d’information instantanée.

Tableau 5.2 Matrice des scénarios évalués par banc d’essai ( $N$  essais par cellule).

| Scénario                     | Alimentation | Saisie-dépôt (Panda) | Saisie-dépôt (xArm7) |
|------------------------------|--------------|----------------------|----------------------|
| Sans obstacle critique       | $N = \dots$  | $N = \dots$          | $N = \dots$          |
| Obstacle(s) statique(s)      | $N = \dots$  | $N = \dots$          | $N = \dots$          |
| Obstacle concave (croissant) | —            | $N = \dots$          | —                    |
| Obstacle mobile (sphère)     | $N = \dots$  | —                    | —                    |
| Occlusion / perte de suivi   | $N = \dots$  | —                    | —                    |

### 5.1.3 Métriques

Les métriques suivantes sont utilisées dans tout le chapitre et ne sont définies qu’ici :

- **Taux de succès** : fraction des essais où la tâche est complétée dans un délai maximal  $T_{\max}$ , la complétion étant le piquage effectif de la cible pour la tâche d’alimentation et le dépôt du cube dans la boîte pour la saisie et dépôt ;
- **Collisions** : nombre d’essais avec contact estimé, c’est-à-dire une distance minimale entre les corps protégés du robot et le nuage de points instantané de la caméra inférieure à 0,5 mm. Ce juge est indépendant de la carte persistante sur laquelle agit



le filtre : le flux brut de la caméra est utilisé directement, de sorte que le critère ne dépend pas de la représentation dont le filtre se sert pour éviter ;

- **Marge minimale**  $h_{\min}$  : plus petite valeur de la barrière corrigée atteinte au cours de l’essai ;
- **Durée de violation** : temps cumulé passé sous la marge de sécurité ( $h < 0$ ) ;
- **Temps de complétion** : durée entre le début de l’essai et le critère de succès, calculée sur les essais réussis seulement ;
- **Temps de blocage** : temps cumulé pendant lequel la progression de la tâche est gelée ;
- **Déviations de trajectoire** : distance (DTW) entre la trajectoire de l’outil exécutée et la trajectoire nominale, mesurant le coût du filtrage sur l’intention ;
- **Temps de calcul** : durée de chaque étape du pipeline (Section 5.6).

Sauf mention contraire, les taux de succès et les collisions sont rapportés en proportion des  $N$  essais de la cellule, et les métriques continues par leur médiane et leur écart interquartile ; la marge minimale  $h_{\min}$  est également rapportée en pire cas, la sécurité se jugeant sur l’essai le plus défavorable.

## 5.2 Validation des composants

Cette section valide isolément les deux composants appris du système avant leur évaluation conjointe : le modèle géométrique au cœur du filtre de sécurité, la SDF du robot approximée par polynômes de Bernstein (Section 4.7.1), puis le planificateur nominal par *Flow Matching* (Section 4.5).

### 5.2.1 Précision du SDF du robot

Trois propriétés de la SDF sont évaluées par rapport à la vérité terrain calculée sur les maillages du robot : la précision de la distance, la qualité de la direction du gradient et le temps de calcul.

Le protocole est le suivant : pour chacun des neuf corps protégés, des rayons sont lancés depuis la surface le long de la normale sortante (40 rayons par corps), et la SDF est évaluée en 25 échantillons par rayon, entre 1 mm et 50 mm de la surface. Seuls les rayons *propres* sont conservés, c’est-à-dire ceux dont la distance exacte à l’union des corps coïncide avec l’avancée le long de la normale, ce qui garantit que l’axe des abscisses des figures correspond bien à la distance réelle à la surface du robot. Les échantillons situés hors du domaine d’entraînement du modèle d’un corps (7,7 % des points, principalement au-delà de 35 mm) sont exclus, car

le modèle y retourne une borne géométrique plutôt que la SDF apprise. Au total, 5541 points extérieurs vérifiés sont retenus. La Figure 5.1 donne un aperçu qualitatif du modèle évalué.

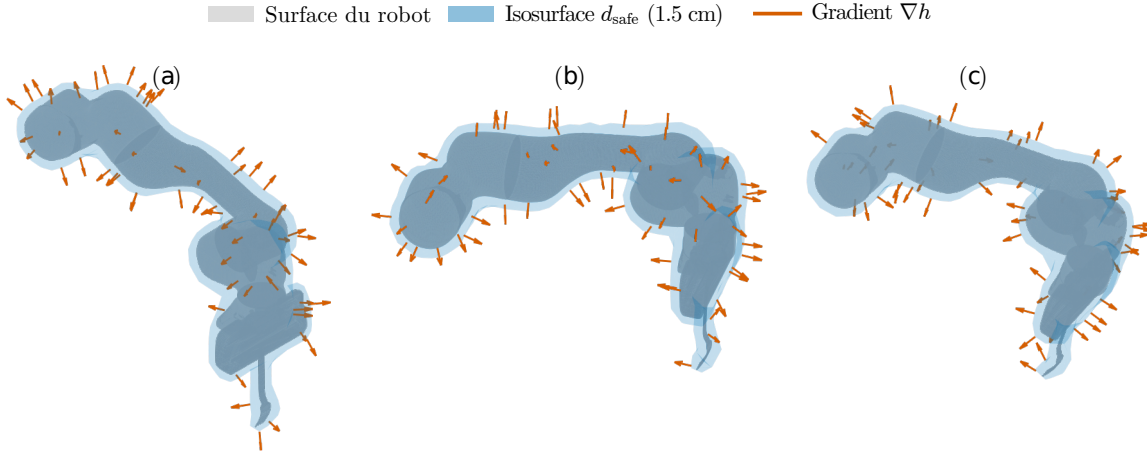


Figure 5.1 Visualisation qualitative du SDF de Bernstein pour trois configurations articulaires : gradients  $\nabla d$  évalués le long des normales à la surface du robot.

La figure 5.2 compare la distance prédite à la distance exacte. L'erreur absolue moyenne est de 1,96 mm avec un biais négligeable ( $-0,02$  mm) et une corrélation  $R = 0,9869$ . La distribution des erreurs signées est centrée et symétrique, avec un 99<sup>e</sup> centile de l'erreur absolue de 6,8 mm.

La figure 5.3(a) présente l'erreur angulaire entre le gradient du modèle et la direction exacte (vecteur vers le point de surface le plus proche). L'erreur médiane globale est de  $5,4^\circ$ , et reste sous  $10^\circ$  pour la grande majorité des points dès 5 mm de la surface. La figure 5.3(b) montre la norme du gradient : une SDF exacte satisfait la propriété eikonale  $\|\nabla d\| = 1$ , et le modèle s'en approche au-delà de 10 mm, avec un affaissement vers 0,7 tout près de la surface, où le polynôme lisse le passage par zéro.

Ces résultats justifient la marge de sécurité  $d_{\text{safe}} = 15$  mm retenue au chapitre précédent, selon la distinction établie à la Section 4.7.7 : en régime établi ( $h_{\text{min}} \geq 0$ ), la distance apprise est d'au moins 15 mm et l'erreur au 99<sup>e</sup> centile (6,8 mm) laisse une distance réelle d'au moins 8,2 mm ; dans le pire transitoire certifié de la variante à valeur exacte ( $h_{\text{corr}} = 0$ , plancher de 5,8 mm), cette erreur de queue n'est plus dominée par le calcul seul, et c'est la marge mesurée au maillage exact, rapportée plus loin, qui établit l'évitement physique. De plus, à  $d_{\text{safe}}$ , la direction du gradient (erreur médiane  $\approx 6^\circ$ ) et sa norme ( $\approx 1$ ) sont déjà bien conditionnées, ce qui assure que la contrainte CBF pousse le robot dans la bonne direction au moment où le filtre s'active.

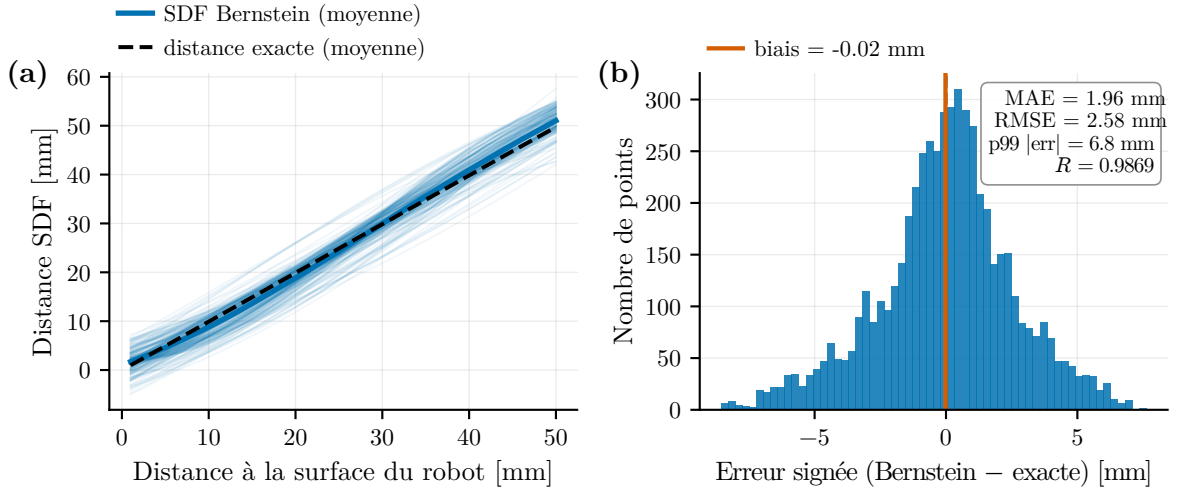


Figure 5.2 Précision de la distance du SDF de Bernstein. (a) Profil de distance le long des normales à la surface du robot ; (b) distribution de l'erreur signée par rapport à la distance exacte calculée sur les maillages.

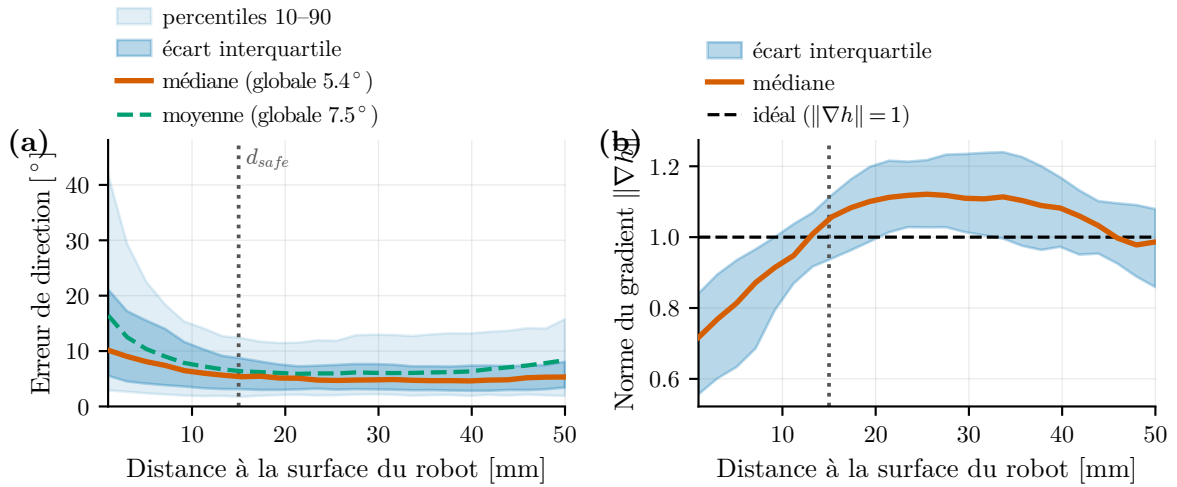


Figure 5.3 Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).

### 5.2.2 Temps d'évaluation du SDF et du gradient

La figure 5.4 compare le temps d'évaluation conjointe de la distance et du gradient entre le modèle de Bernstein (analytique, sur GPU) et la vérité terrain par requête de proximité sur les maillages (`trimesh`, sur CPU). Le temps du modèle forme un plateau d'environ 5 ms jusqu'à quelques milliers de points : à ces tailles, le coût est dominé par des frais fixes par appel (cinématique directe,  $\approx 2,5$  ms, lancement des noyaux GPU, évaluation du gradient et transferts mémoire), le coût par point étant négligeable tant que le GPU n'est pas saturé. La croissance ne devient visible qu'au-delà d'environ 4096 points, soit bien après le point d'opération du filtre ( $K = 100$  points critiques). Au point d'opération, l'évaluation reste donc largement en deçà du budget de 10 ms imposé par la boucle de commande à 100 Hz, tandis que la requête sur maillage croît linéairement avec le nombre de points et dépasse ce budget d'un à deux ordres de grandeur, ce qui exclut son utilisation en ligne.

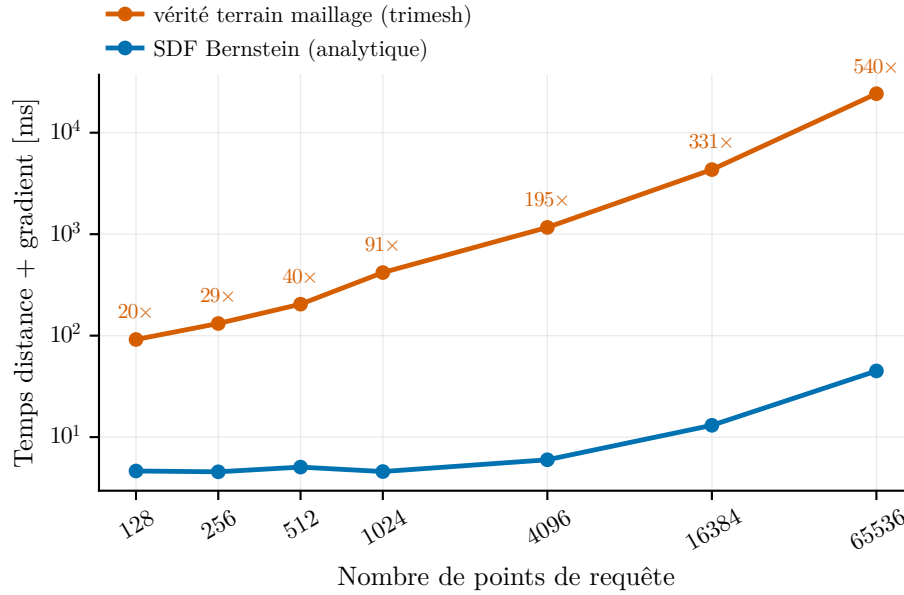


Figure 5.4 Temps de calcul de la distance et du gradient : modèle de Bernstein (GPU) contre vérité terrain sur maillages (CPU), en fonction du nombre de points de requête.

### 5.2.3 Sélection des points critiques

La Figure 5.5 illustre en situation le mécanisme de sélection de la Section 4.7.2 : parmi les voxels de la carte persistante, le préfiltre sphérique autour des liens protégés puis la conservation des  $K = 100$  points de plus faible SDF corps-complet retiennent les points rouges, seuls transmis au filtre comme contraintes potentielles. La sélection épouse la géométrie du

bras plutôt qu’une simple boule autour de l’outil : des points sont retenus le long des liens intermédiaires dès qu’un obstacle s’en approche, conformément à la protection corps-complet du Chapitre 4. L’isosurface  $d_{\text{robot}} = 1,5$  cm tracée autour de l’outil matérialise la frontière certifiée  $h = 0$  que le filtre défend.

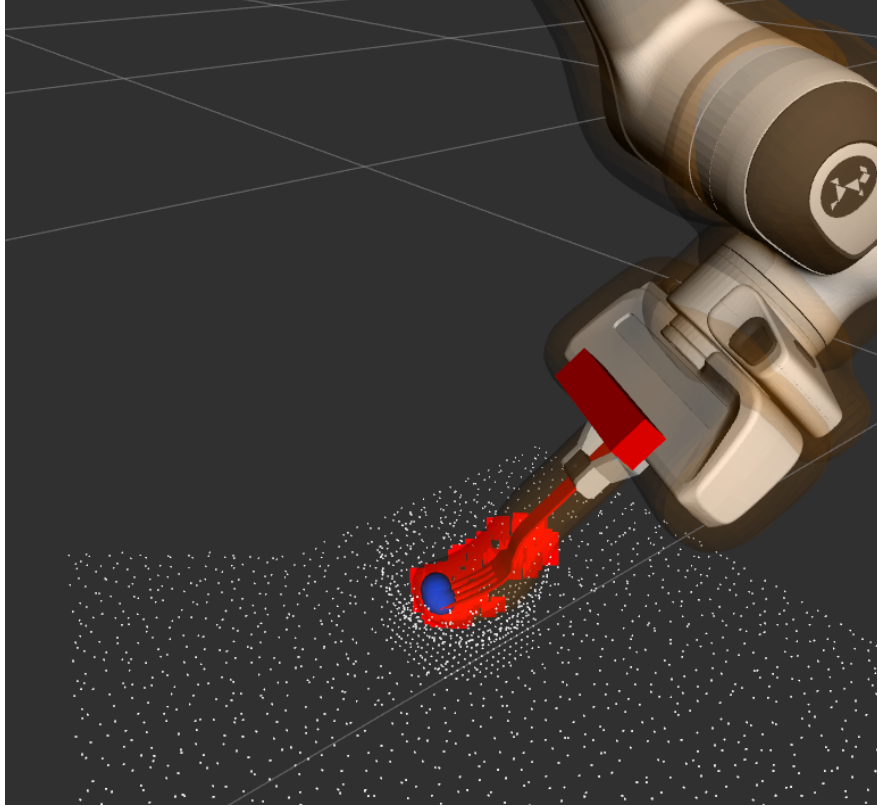


Figure 5.5 Sélection des  $K$  points de la carte persistante les plus proches du robot (en rouge) qui contraignent le filtre CBF ; l’isosurface  $d_{\text{robot}} = 1,5$  cm forme l’enveloppe de sécurité autour de l’outil (en bleu).

#### 5.2.4 Validation du planificateur nominal

Cette sous-section évalue le planificateur *Flow Matching* seul, hors de la boucle de sécurité : fidélité aux démonstrations expertes, comportement le long de l’horizon de prédiction et multimodalité des générations.

La Figure 5.6 présente la distribution des erreurs de reproduction sur le jeu de validation : la déviation de position (DTW) affiche une médiane de 0,67 cm et l’erreur de rotation finale une médiane de  $3,21^\circ$ . Ces erreurs sont du même ordre que la variabilité des démonstrations expertes elles-mêmes : le planificateur restitue l’intention démontrée sans dégradation notable.

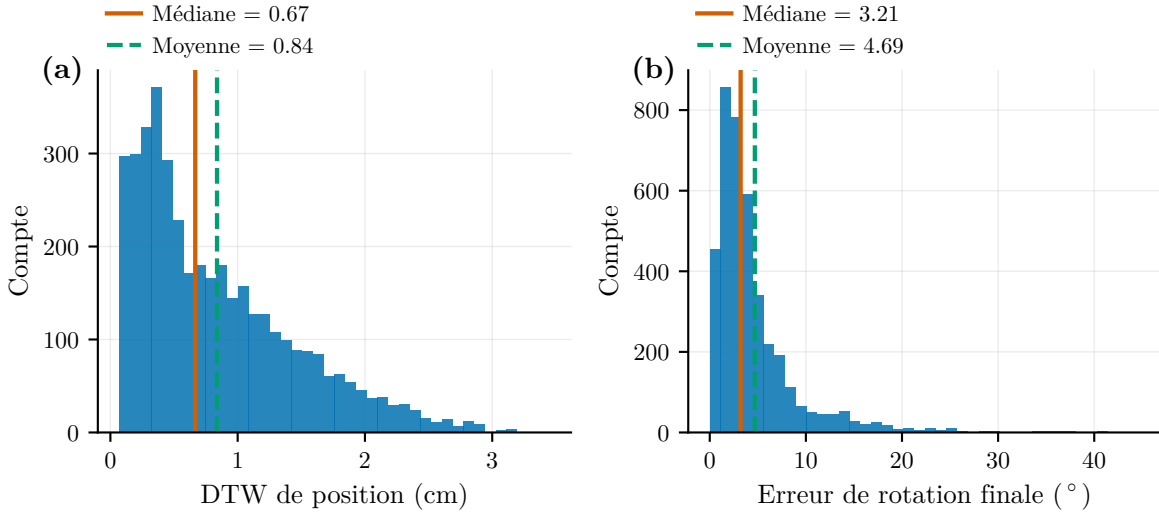


Figure 5.6 Fidélité du planificateur nominal aux démonstrations expertes sur le jeu de validation. (a) Distribution de la déviation de position (DTW) ; (b) distribution de l’erreur de rotation finale.

La Figure 5.7 montre la croissance de l’erreur le long de l’horizon de prédiction : l’incertitude s’accumule avec le point de l’horizon  $k$ . Ce profil justifie le fonctionnement en horizon glissant retenu au Chapitre 4 : la régénération à 10 Hz fait que seuls les premiers points de chaque segment sont exécutés, là où l’erreur est la plus faible, la queue de l’horizon n’étant jamais atteinte.

La Figure 5.8 superpose plusieurs générations issues de la même observation, illustrant la capacité du modèle à représenter la multimodalité des démonstrations, et la Figure 5.9 illustre l’intégration du flot, du bruit initial ( $t = 0$ ) au segment généré ( $t = 1$ ). Des scènes supplémentaires sont présentées à l’Annexe E.

### 5.3 Sécurité et évitement d’obstacles

Cette section répond à la question centrale de ce mémoire : le filtre SDF-CBF permet-il d’exécuter sans collision, dans des environnements non vus à l’entraînement, les trajectoires produites par le planificateur génératif, sans réentraînement de celui-ci ? La comparaison principale oppose le système complet au même système en mode observation (*monitor only*), où la barrière et les diagnostics sont calculés mais aucune correction n’est appliquée.

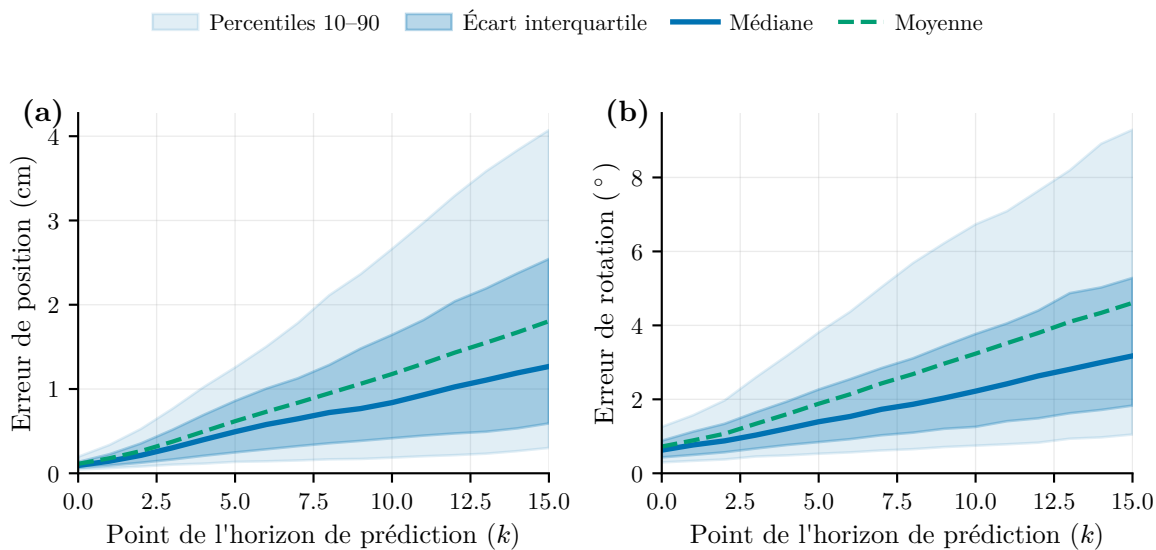


Figure 5.7 Erreur de prédiction en fonction du point  $k$  de l'horizon. (a) Erreur de position ; (b) erreur de rotation. Bandes : percentiles 10–90 et écart interquartile.

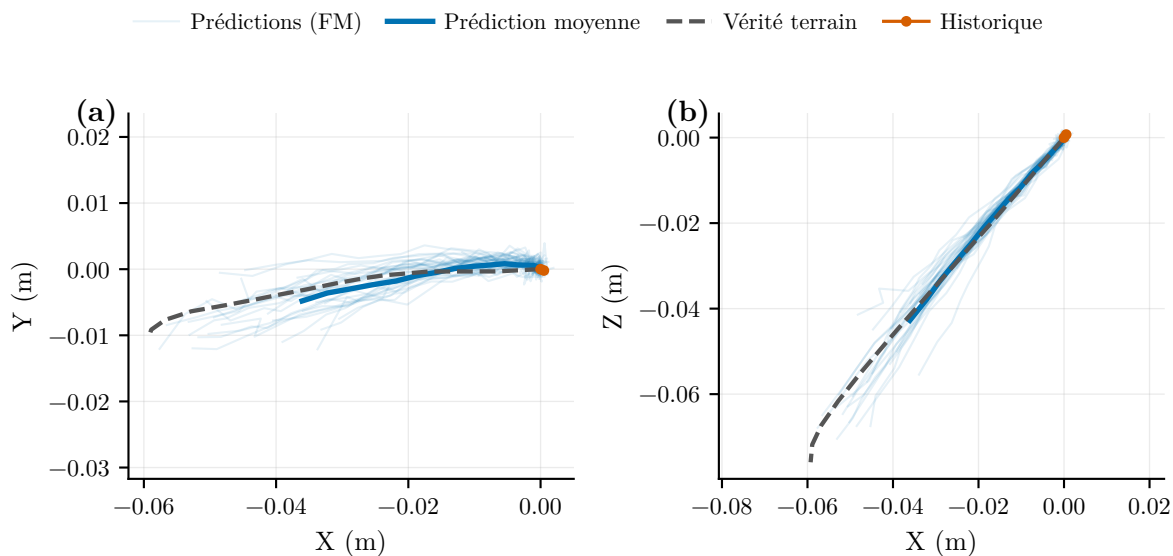


Figure 5.8 Générations multiples du planificateur pour une même observation, superposées à la vérité terrain. (a) Plan  $XY$  ; (b) plan  $XZ$ .

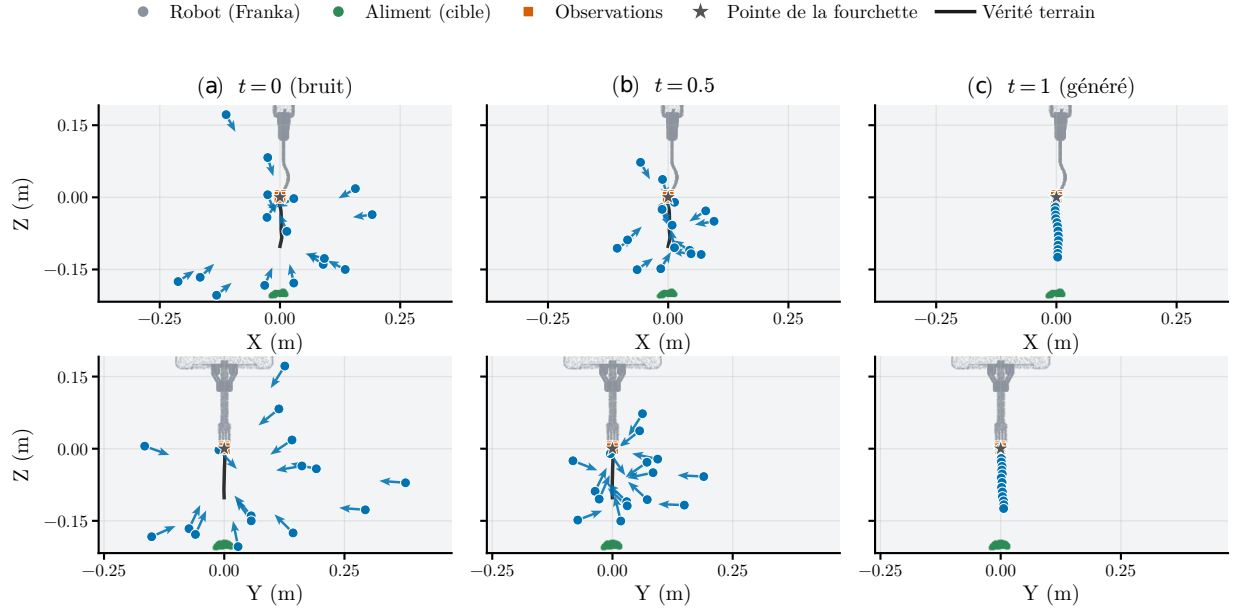


Figure 5.9 Intégration du flot pour une scène de la tâche d'alimentation : segment  $X_t$  à  $t = 0$  (bruit),  $t = 0,5$  et  $t = 1$  (généré), dans les plans  $XZ$  (haut) et  $YZ$  (bas).

### 5.3.1 Résultats agrégés avec et sans filtre

À compléter.

Tableau 5.3 Comparaison avec et sans filtre SDF-CBF, agrégée sur  $N$  essais par cellule : taux de succès, collisions, marge minimale  $h_{\min}$  et déviation de trajectoire (métriques de la Section 5.1.3).

| Scénario        | Condition   | Succès | Collisions | $h_{\min}$ (mm) | Déviation (cm) |
|-----------------|-------------|--------|------------|-----------------|----------------|
| Prisme statique | sans filtre | ...    | ...        | ...             | —              |
|                 | avec filtre | ...    | ...        | ...             | ...            |
| Sphère mobile   | sans filtre | ...    | ...        | ...             | —              |
|                 | avec filtre | ...    | ...        | ...             | ...            |

La Figure 5.10 illustre la comparaison sur un essai représentatif : trajectoire de la pointe de l'outil dans la carte persistante, avec et sans filtre, jusqu'à la cible. À compléter.

### 5.3.2 Étude détaillée : tâche d'alimentation

Les sous-sections suivantes analysent en profondeur un essai représentatif de la tâche d'alimentation en présence d'obstacles, en suivant la chaîne de commande du Chapitre 4 : la



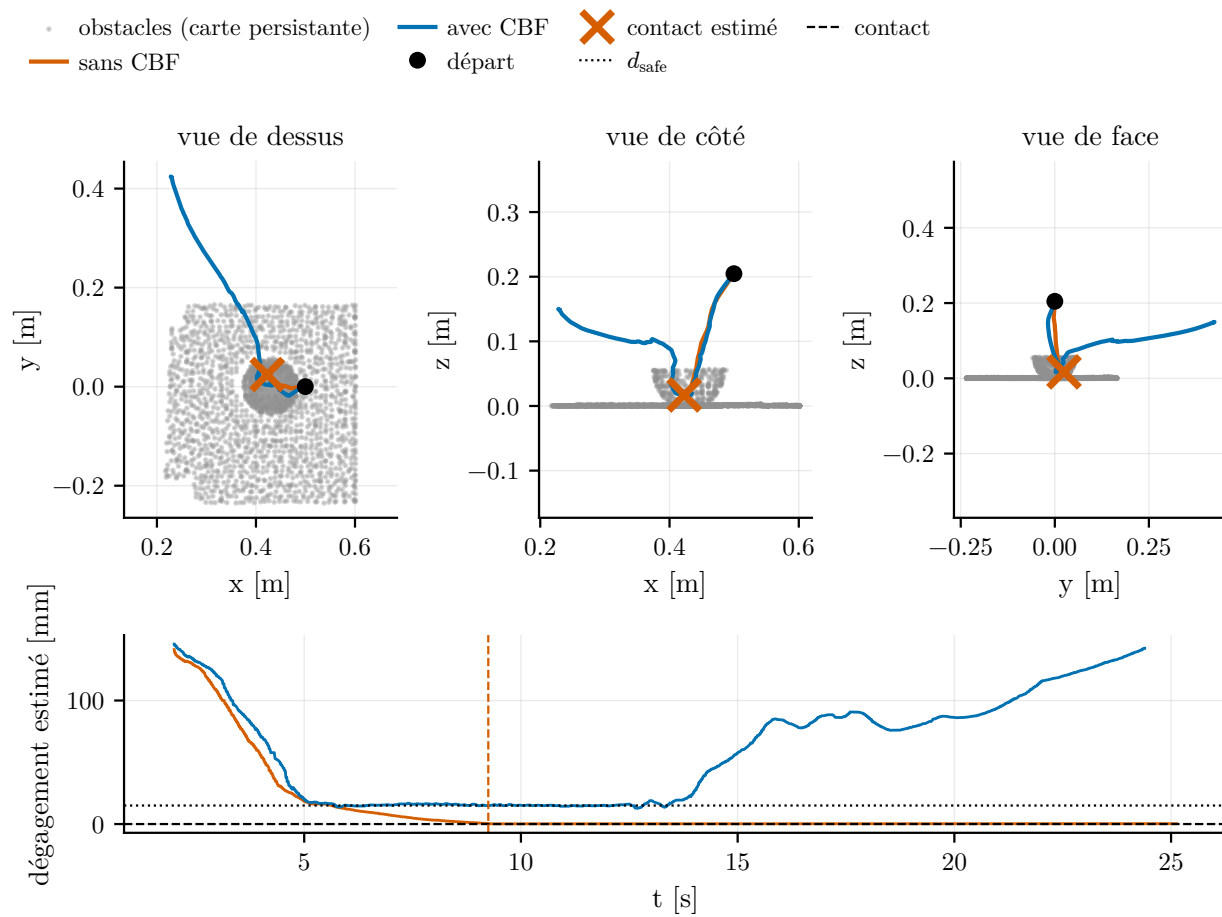


Figure 5.10 Trajectoire de la pointe de l'outil avec et sans filtre SDF-CBF pour un essai représentatif, superposée au nuage d'obstacles de la carte persistante et à la cible.

barrière et la contrainte d’abord, puis la décomposition des vitesses qui révèle l’action de chaque étage du filtre.

### Évolution de la barrière et de la contrainte de sécurité

La Figure 5.11 présente l’évolution de la barrière  $h_{\text{soft}}$  et de sa valeur compensée  $h_{\text{corr}}$  au cours d’un essai représentatif. Les bandes vertes marquent les cycles où la projection corrige la vitesse ( $\Delta \dot{\mathbf{q}}_{\text{CBF}} \neq 0$ ) ; les bandes jaunes, ceux où seule la réparation intervient : la vitesse instantanée satisfait la contrainte, mais la commande filtrée, qui porte la mémoire des cycles précédents, échoue à la revérification et est corrigée par  $\Delta \dot{\mathbf{q}}_{\text{rep}}$ . Après la phase d’approche, le système opère en régime de suivi de frontière :  $h_{\text{corr}}$  se stabilise à quelques millimètres de la frontière pendant que le robot progresse tangentiellement le long des surfaces vers la cible, conformément au comportement attendu de la projection préservant la composante tangentielle (Section 4.7.5).

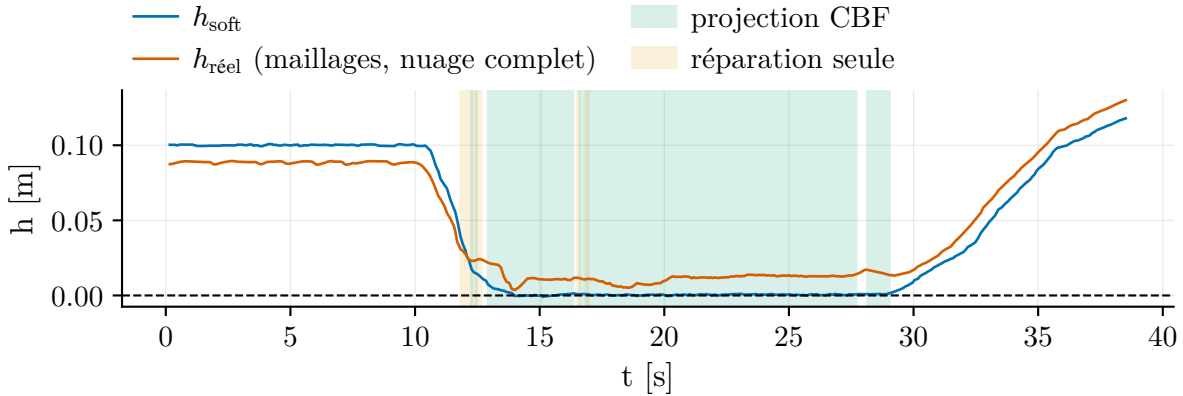


Figure 5.11 Évolution de la barrière  $h_{\text{soft}}$  et de sa valeur compensée  $h_{\text{corr}}$  pour une tâche en présence d’obstacles ; les bandes vertes indiquent les cycles où la projection corrige la vitesse, les bandes jaunes ceux où seule la réparation intervient.

La Figure 5.12 compare la valeur de la contrainte de sécurité évaluée sur la vitesse nominale (avant projection) et sur la commande finale (après filtrage et réparation). La courbe nominale plonge sous zéro dès que le robot opère à proximité des obstacles : laissée seule, la trajectoire générée par *Flow Matching* violerait continuellement la condition de sécurité. La courbe finale ne descend jamais sous  $-m_{\text{CBF}}$  (pire valeur  $-5 \times 10^{-4}$ ) : le résidu introduit par le filtrage et les saturations est entièrement absorbé par la marge numérique, de sorte que la condition CBF proprement dite,  $\nabla h^\top \dot{\mathbf{q}} + \kappa_{\text{eff}} h \geq 0$ , est satisfaite à chaque cycle de l’essai. L’étape de réparation (Section 4.7.5) est loin d’être marginale dans ce résultat : elle

intervient sur la quasi-totalité des cycles où la projection est active, le filtre de sortie érodant systématiquement une partie de la correction au sein du même cycle. L'écart entre les deux courbes constitue la mesure directe de l'intervention du filtre de sécurité.

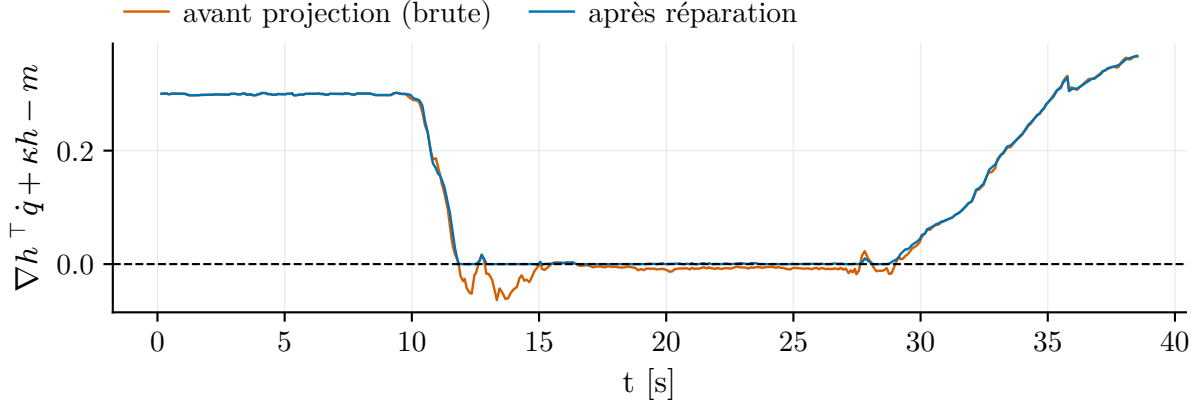


Figure 5.12 Contrainte de sécurité évaluée sur la vitesse nominale (avant projection) et sur la commande finale (après réparation). Les valeurs négatives de la courbe nominale correspondent aux violations qu'aurait commises la trajectoire générée sans filtre.

## Décomposition des vitesses articulaires

Cette sous-section déroule, articulation par articulation, la construction de la vitesse nominale des Sections 4.7.4 à 4.7.5 : le feedforward retimé  $\dot{\mathbf{q}}_{ff}$ , la correction de suivi  $\dot{\mathbf{q}}_{fb}$  et la vitesse nominale filtrée  $\dot{\mathbf{q}}_{base}$  qui en résulte, entrée du filtre de sécurité.

Trois observations quantitatives ressortent des signaux enregistrés, indépendamment de leur représentation graphique. Premièrement,  $\|\dot{\mathbf{q}}_{ff}\|$  est constante à 0,25 rad/s sur toute la tâche : le retiming à vitesse articulaire constante (Section 4.6.3) remplit son rôle et fournit au filtre une référence régulière. Deuxièmement, la correction de projection est strictement nulle hors des zones d'activation : le filtre est transparent dès que la contrainte est satisfaite, ce qui confirme le mécanisme de transmission directe. On remarque d'ailleurs des épisodes où  $\Delta \dot{\mathbf{q}}_{rep}$  est active alors que  $\Delta \dot{\mathbf{q}}_{CBF}$  demeure nulle (vers 5 s et 22 s) : la barrière y effleure la frontière sans la presser ; la projection juge la vitesse instantanée admissible, mais la commande filtrée, qui porte la mémoire des cycles précédents, échoue à la revérification et est corrigée par la réparation. Troisièmement, la vitesse nominale  $\dot{\mathbf{q}}_{base}$  présente un bruit important dont l'analyse spectrale situe la puissance dominante à 12 Hz, soit exactement la fréquence de régénération du planificateur : chaque nouvelle trajectoire générée déplace légèrement la consigne, et le gain de suivi  $K_p$  transforme ces sauts en impulsions de vitesse.

### Action du filtre de sécurité sur la vitesse commandée

La Figure 5.13 superpose, pour chacune des sept articulations, la vitesse nominale  $\dot{\mathbf{q}}_{\text{base}}$  (ce que le robot ferait en l'absence d'obstacle), la correction de projection et la vitesse effectivement commandée  $\dot{\mathbf{q}}_{\text{final}}$  : l'écart entre les courbes se lit directement comme l'amplitude de l'intervention du filtre, et la correction se concentre sur les articulations ayant le plus d'autorité sur la barrière.

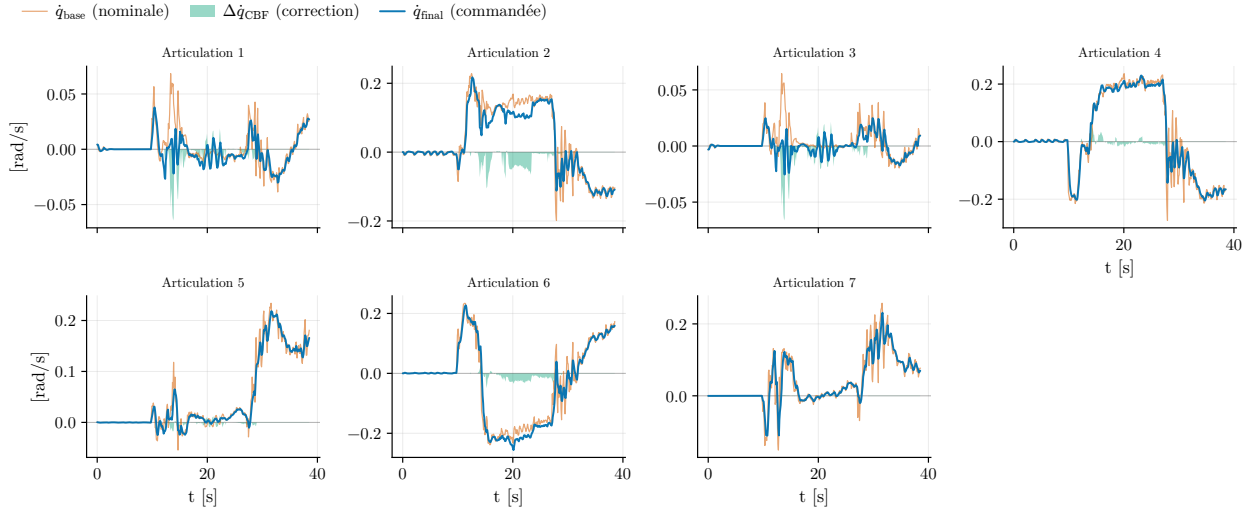


Figure 5.13 Vitesse nominale, correction de projection et vitesse commandée pour chacune des sept articulations.

Hors des zones d'activation, la vitesse commandée demeure légèrement sous la nominale ( $\approx 5\%$  en moyenne sur la norme) : ce n'est pas une intervention de sécurité mais l'effet du filtre passe-bas de sortie ( $\tau_{\text{safe}} = 0,08\text{ s}$ ).

### Effet du rééchantillonnage du planificateur sur la vitesse exécutée

La décomposition révèle un effet systémique du fonctionnement en horizon glissant. Bien que  $\|\dot{\mathbf{q}}_{\text{ff}}\| = 0,25\text{ rad/s}$  et que la correction de suivi soit en moyenne alignée avec le feed-forward (composante moyenne de  $+0,03\text{ rad/s}$  le long de  $\dot{\mathbf{q}}_{\text{ff}}$ ), la vitesse nominale filtrée ne moyenne que  $\approx 0,15\text{ rad/s}$ . La cause n'est pas un freinage mais un phénomène vectoriel : à chaque régénération, la *direction* instantanée de  $\dot{\mathbf{q}}_{\text{ff}}$  saute (rotations atteignant  $80^\circ$  entre cycles consécutifs au  $95^{\text{e}}$  centile), et le filtre passe-bas, en moyennant des vecteurs dont la direction alterne à  $12\text{ Hz}$ , produit un vecteur de norme inférieure à la moyenne des normes. Près de  $40\%$  de la vitesse demandée est ainsi perdue par annulation vectorielle, indépendamment de tout obstacle. Ce constat motive l'agrégation temporelle des trajectoires générées

(moyennage pondéré des dernières générations) comme amélioration future du planificateur.

### Analyse de l'interface de commande

L'instrumentation du rapport entre le taux de variation de la barrière commandé ( $\nabla h^\top \dot{\mathbf{q}}_{\text{final}}$ ) et celui réellement exécuté ( $\nabla h^\top \dot{\mathbf{q}}_{\text{mes}}$ ) a guidé le choix de l'interface de commande et justifie a posteriori celui de la Section 4.7.5. Deux interfaces fondées sur une consigne de *position*, évaluées en cours de développement, présentaient chacune un mode de défaillance. Avec une consigne ré-ancrée à chaque cycle sur la position mesurée, le rapport exécuté/commandé mesuré était proche de zéro : les corrections de sécurité étaient réabsorbées avant d'être réalisées, et le robot demeurait immobilisé contre la frontière malgré une commande de dégagement soutenue. À l'inverse, avec une référence intégrée à un pas supérieur à la période réelle de la boucle, la référence avançait plus vite que la vitesse certifiée et le rapport devenait négatif et supérieur à l'unité en valeur absolue : le robot traversait la frontière en oscillant, produisant des violations répétées de la marge. L'interface retenue élimine ces deux régimes en supprimant l'intermédiaire : la vitesse certifiée est transmise directement au contrôleur de vitesse articulaire, la référence de position intégrée n'étant conservée qu'à des fins de supervision (Section 4.7.5).

La Figure 5.14 rapporte ce même rapport avec l'interface retenue : le taux de variation de la barrière réellement exécuté suit le taux commandé, ce qui confirme que les corrections certifiées par le filtre sont effectivement réalisées par le contrôleur de vitesse bas niveau.

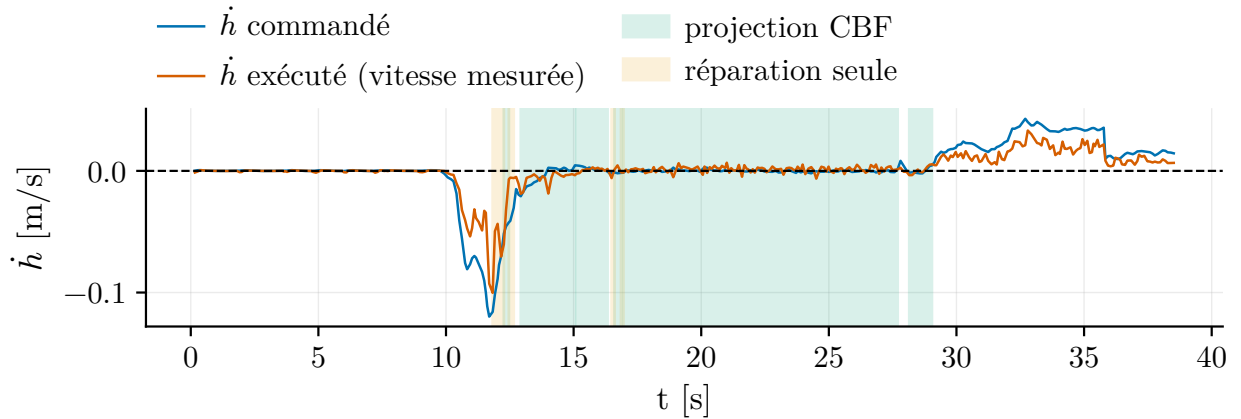


Figure 5.14 Taux de variation de la barrière commandé ( $\nabla h^\top \dot{\mathbf{q}}_{\text{final}}$ ) et réellement exécuté ( $\nabla h^\top \dot{\mathbf{q}}_{\text{mes}}$ ) avec l'interface de commande retenue.

### 5.3.3 Étude détaillée : tâche de saisie et dépôt

À compléter.

## 5.4 Ablations des mécanismes

Chaque mécanisme de conception du Chapitre 4 est isolé par une ablation dédiée : le système complet est comparé au même système dont le seul mécanisme évalué est désactivé, sur le scénario qui sollicite ce mécanisme. Chaque ablation suit le même gabarit : la décision de conception qu'elle valide, le résultat agrégé, le constat.

### 5.4.1 Carte persistante contre perception instantanée

La carte persistante du Chapitre 4 découple la sécurité du champ de vision instantané : une caméra au poignet perd l'obstacle dès que le bras s'en détourne, précisément au moment où il s'en approche. L'ablation alimente le filtre directement par le nuage instantané nettoyé plutôt que par la carte, sur le scénario d'occlusion du banc d'alimentation : sans mémoire spatiale, l'obstacle occulté disparaît des contraintes du filtre au moment où elles importent le plus. À compléter.

### 5.4.2 Métrique de tâche contre métrique euclidienne

La projection en métrique de tâche  $\mathbf{W}$  (Section 4.7.5) répartit la correction de sécurité préférentiellement dans le noyau de la tâche afin de préserver le mouvement de l'outil. L'ablation la remplace par la métrique euclidienne, à scénario et réglages par ailleurs identiques (saisie et dépôt, prisme statique) : à sécurité égale, la déviation de l'outil (DTW) entre trajectoires exécutée et nominale mesure ce que la métrique de tâche préserve de l'intention. À compléter.

### 5.4.3 Dégagement postural : neutralité de sécurité et progression

Le dégagement postural (Section 4.7.6) est le seul mécanisme de déblocage du filtre, et son statut est celui d'une heuristique de progression : il n'entre que dans l'objectif du problème de projection, jamais dans ses contraintes, de sorte que la sécurité ne peut structurellement pas en dépendre. L'ablation vérifie empiriquement cette neutralité et mesure ce que le terme achète : à scénarios appariés (conflit porté par le corps du bras, corridor de l'outil libre), la condition ablatée ( $k_{\text{clear}} = 0$ ) est comparée à la condition nominale sur les colonnes de sécurité, attendues indiscernables, et sur les colonnes de progression (taux et temps de complétion, taux d'arrêt sûr sans progression), attendues favorables au dégagement. À compléter.

#### 5.4.4 Resserrement dynamique face à un obstacle mobile

Le resserrement dynamique (Section 4.7.4) fait céder le robot proportionnellement à la vitesse d’approche estimée de l’environnement, avant même que la barrière ne décroisse. L’ablation le désactive face à la sphère mobile et rapporte  $h_{\min}$  et la durée de violation en fonction de la vitesse de la sphère. À compléter.

Conformément à la portée des garanties établie à la Section 4.7.7, aucune garantie formelle n’est revendiquée face aux obstacles mobiles : le resserrement dynamique est une robustification dont l’apport est ici évalué empiriquement.

#### 5.4.5 Réparation de la commande filtrée

L’étude détaillée de la Section 5.3.2 montre que la réparation intervient sur la quasi-totalité des cycles où la projection est active, le filtre passe-bas de sortie érodant une partie de la correction au sein même du cycle. Cette ablation la désactive pour quantifier les violations que ce lissage causerait seul :  $h_{\min}$  et la durée de violation sont comparées au système complet sur le banc d’alimentation avec obstacles statiques. À compléter.

### 5.5 Généralisation inter-embodiment

Le transfert vers le xArm7 n’introduit aucune méthodologie nouvelle : la chaîne du Chapitre 4 est agnostique à l’embodiment. La politique de *Flow Matching* génère des poses de l’outil dans l’espace de la tâche et son conditionnement perceptif est inchangé ; seuls les éléments propres au bras sont substitués, à savoir le modèle cinématique (URDF et cinématique inverse) et les SDF de Bernstein par lien, réentraînés sur les maillages du xArm7 par le protocole de la Section 4.7.1. Le filtre de sécurité, ses barrières et ses réglages sont identiques. Le Tableau 5.4 compare les deux bras sur les scénarios communs de la tâche de saisie et dépôt. À compléter.

Tableau 5.4 Comparaison Panda et xArm7 sur les scénarios communs de la tâche de saisie et dépôt ( $N$  essais par cellule, métriques de la Section 5.1.3).

| Scénario                | Robot | Succès | Collisions | $h_{\min}$ (mm) | Déviaton (cm) |
|-------------------------|-------|--------|------------|-----------------|---------------|
| Sans obstacle critique  | Panda | ...    | ...        | ...             | ...           |
|                         | xArm7 | ...    | ...        | ...             | ...           |
| Obstacle(s) statique(s) | Panda | ...    | ...        | ...             | ...           |
|                         | xArm7 | ...    | ...        | ...             | ...           |

## 5.6 Performance et temps de calcul

Le Tableau 5.5 rapporte les statistiques des temps de calcul par étape du pipeline, mesurés en cours d’exécution sur la plateforme de la Section 4.1.1. Les étapes se rattachent à la hiérarchie de fréquences du Chapitre 4 : à la fréquence de commande, **cbf\_total** couvre le cycle complet du bouclier, dont la projection et la réparation (**cbf\_correction**) constituent le coût dominant ; à la cadence de cartographie, l’intégration et l’éviction de la carte persistante (**persistent\_integrate**, **persistent\_evict**) et la sélection des points critiques (**critical\_point\_selection**) ; à la cadence de replanification, l’encodage du nuage de conditionnement (**fm\_encode**), la génération (**fm\_generation**) et la cinématique inverse (**ik**) ; la perception sémantique (**sam3\_detect**, **sam2\_track**) s’exécute de manière asynchrone.

Trois constats se dégagent. D’abord, le bouclier tient son budget en régime nominal : la médiane de **cbf\_total** (8,60 ms) reste sous la période de 10 ms de la boucle à 100 Hz, mais sa queue (22,06 ms au 95<sup>e</sup> centile) la dépasse ; lors de ces cycles, le pas de discrétisation effectif s’allonge au-delà de la valeur budgétée dans la marge  $d_{\text{safe}}$  (Section 4.7.7). Ensuite, la sélection des points critiques (médiane à 39,62 ms) s’exécute en pratique légèrement sous la cadence visée de 30 Hz ; l’obsolescence de l’ensemble actif qui en résulte demeure du même ordre que la valeur prise en compte dans le budget de la marge. Enfin, la replanification tient largement sa cadence : la génération (62,07 ms en moyenne) et l’encodage s’inscrivent dans la période de 100 ms à 10 Hz, et la cinématique inverse est négligeable (0,27 ms).

La ligne **sam3\_detect** ne compte que deux occurrences, et c’est là un comportement nominal plutôt qu’un déficit statistique : SAM 3 n’est invoqué qu’à l’acquisition initiale de la cible puis à chaque réacquisition après une perte de suivi, le suivi continu étant assuré par SAM 2 (**sam2\_track**). Le nombre d’occurrences compte donc les (ré)acquisitions de l’essai, et le coût de chacune (jusqu’à 335 ms) est absorbé hors du chemin critique par l’architecture asynchrone, la carte persistante continuant d’alimenter le filtre pendant la détection.

## 5.7 Cas d’échec et limites

Les échecs observés pendant les campagnes sont documentés ici sans complaisance, chacun rattaché à une limitation déjà annoncée, qu’il s’agisse de la portée des garanties (Section 4.7.7) ou des exclusions du Chapitre 3 : aucun d’eux ne contredit une propriété revendiquée. À compléter.



Tableau 5.5 Statistiques descriptives des temps de calcul par étape du pipeline (durées en millisecondes).

| Étape                    | n    | Moyenne | Écart-type | Médiane | p95    | Max    |
|--------------------------|------|---------|------------|---------|--------|--------|
| cbf_command              | 1660 | 1,88    | 2,12       | 1,38    | 4,98   | 31,10  |
| cbf_correction           | 1660 | 7,58    | 5,41       | 6,50    | 17,65  | 41,44  |
| cbf_total                | 1660 | 9,91    | 6,43       | 8,60    | 22,06  | 57,81  |
| critical_point_selection | 832  | 42,27   | 18,35      | 39,62   | 75,49  | 148,81 |
| fm_encode                | 263  | 1,51    | 2,21       | 1,02    | 2,85   | 20,87  |
| fm_generation            | 263  | 62,07   | 6,40       | 61,62   | 73,89  | 83,89  |
| ik                       | 255  | 0,27    | 0,09       | 0,25    | 0,43   | 0,93   |
| persistent_evict         | 709  | 3,17    | 1,74       | 2,76    | 5,90   | 22,08  |
| persistent_integrate     | 1753 | 1,02    | 1,06       | 0,84    | 1,80   | 23,85  |
| sam2_track               | 132  | 53,85   | 15,10      | 52,77   | 78,66  | 91,14  |
| sam3_detect              | 2    | 182,45  | 152,72     | 182,45  | 319,89 | 335,16 |

## 5.8 Discussion

À compléter.

## CHAPITRE 6 CONCLUSION

Texte / Text.

### 6.1 Synthèse des travaux / Summary of Works

Texte / Text.

### 6.2 Limitations de la solution proposée / Limitations

### 6.3 Améliorations futures / Future Research

Texte / Text.

## RÉFÉRENCES

- [1] S. Karaman et E. Frazzoli, “Sampling-Based Algorithms for Optimal Motion Planning,” *The International Journal of Robotics Research (IJRR)*, vol. 30, n°. 7, p. 846–894, 2011.
- [2] I. A. Şucan, M. Moll et L. E. Kavraki, “The Open Motion Planning Library,” *IEEE Robotics & Automation Magazine*, vol. 19, n°. 4, p. 72–82, 2012.
- [3] S. Liu et P. Liu, “Robot Motion Planning Benchmarking and Optimization using Motion Planning Pipeline,” 2021, arXiv preprint.
- [4] M. Zucker *et al.*, “CHOMP : Covariant Hamiltonian Optimization for Motion Planning,” *The International Journal of Robotics Research (IJRR)*, vol. 32, n°. 9-10, p. 1164–1193, 2013.
- [5] J. Schulman *et al.*, “Motion Planning with Sequential Convex Optimization and Convex Collision Checking,” *The International Journal of Robotics Research (IJRR)*, vol. 33, n°. 9, p. 1251–1270, 2014.
- [6] J. J. Kuffner et S. M. LaValle, “RRT-Connect : An Efficient Randomized Spatio-temporal Planner,” dans *IEEE International Conference on Robotics and Automation (ICRA)*, 2000, p. 995–1001.
- [7] L. E. Kavraki *et al.*, “Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces,” *IEEE Transactions on Robotics and Automation*, vol. 12, n°. 4, p. 566–580, 1996.
- [8] C. G. Atkeson et S. Schaal, “Robot Learning from Demonstration,” dans *International Conference on Machine Learning (ICML)*, 1997, p. 12–20.
- [9] S. Schaal, “Is Imitation Learning the Way to Eager Robotics?” *Trends in Cognitive Sciences*, vol. 3, n°. 6, p. 233–242, 1999.
- [10] B. D. Argall *et al.*, “A survey of robot learning from demonstration,” 2009, arXiv preprint.
- [11] D. A. Pomerleau, “ALVINN : An Autonomous Land Vehicle in a Neural Network,” dans *Advances in Neural Information Processing Systems (NeurIPS)*, 1989, p. 305–313.
- [12] M. Bain et C. Sammut, “A Framework for Behavioural Cloning,” dans *Machine Intelligence 15*, 1995, p. 103–129.
- [13] T. Z. Zhao *et al.*, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” 2023, arXiv preprint.

- [14] P. Sundaresan, S. Belkhale et D. Sadigh, “Learning Visuo-Haptic Skewering Strategies for Robot-Assisted Feeding,” 2022, arXiv preprint.
- [15] R. Liu, A. Bhaskar et P. Tokekar, “Adaptive Visual Imitation Learning for Robotic Assisted Feeding Across Varied Bowl Configurations and Food Types,” 2024, arXiv preprint.
- [16] S. Levine, “Supervised learning of behaviors,” Lecture notes, CS 285 : Deep Reinforcement Learning, UC Berkeley, 2026, accessed : 2026-07-05. [En ligne]. Disponible : <https://rail.eecs.berkeley.edu/deeprlcourse/static/slides/lec-2.pdf>
- [17] P. Florence *et al.*, “Implicit Behavioral Cloning,” 2021, arXiv preprint.
- [18] D. Jarrett, I. Bica et M. v. d. Schaar, “Strictly Batch Imitation Learning by Energy-based Distribution Matching,” 2021, arXiv preprint.
- [19] P. Florence, L. Manuelli et R. Tedrake, “Self-Supervised Correspondence in Visuomotor Policy Learning,” 2019, arXiv preprint.
- [20] J. Luo *et al.*, “Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning,” 2025, arXiv preprint.
- [21] A. Gupta *et al.*, “Relay Policy Learning : Solving Long-Horizon Tasks via Imitation and Reinforcement Learning,” 2019, arXiv preprint.
- [22] S. M. Khansari-Zadeh et A. Billard, “Learning Stable Nonlinear Dynamical Systems With Gaussian Mixture Models,” 2011, arXiv preprint.
- [23] R. Wolf *et al.*, “Diffusion Models for Robotic Manipulation : A Survey,” *arXiv preprint arXiv :2504.08438*, 2025.
- [24] J. Cho, M. Kim et S. Jung, “A Survey on Flow Matching for Robotic Trajectory Generation and Control,” 2025, arXiv preprint.
- [25] C. Chi *et al.*, “Diffusion Policy : Visuomotor Policy Learning via Action Diffusion,” dans *Proceedings of Robotics : Science and Systems (RSS)*, 2023.
- [26] J. Ho, A. Jain et P. Abbeel, “Denoising Diffusion Probabilistic Models,” 2020, arXiv preprint.
- [27] M. Janner *et al.*, “Planning with Diffusion for Flexible Behavior Synthesis,” 2022, arXiv preprint.
- [28] S. Huang *et al.*, “Diffusion-based Generation, Optimization, and Planning in 3D Scenes,” 2023, arXiv preprint.
- [29] Y. Ze *et al.*, “3D Diffusion Policy : Generalizable Visuomotor Policy Learning via Simple 3D Representations,” 2024, arXiv preprint.

- [30] G. Lu *et al.*, “ManiCM : Real-time 3D Diffusion Policy via Consistency Model for Robotic Manipulation,” 2025, arXiv preprint.
- [31] A. Prasad *et al.*, “Consistency Policy : Accelerated Visuomotor Policies via Consistency Distillation,” 2024, arXiv preprint.
- [32] Z. Wang *et al.*, “One-Step Diffusion Policy : Fast Visuomotor Policies via Diffusion Distillation,” 2024, arXiv preprint.
- [33] Y. Lipman *et al.*, “Flow Matching for Generative Modeling,” dans *International Conference on Learning Representations (ICLR)*, 2023.
- [34] —, “Flow matching guide and code,” 2024. [En ligne]. Disponible : <https://arxiv.org/abs/2412.06264>
- [35] A. Tong *et al.*, “Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport,” *Transactions on Machine Learning Research (TMLR)*, 2024.
- [36] X. Liu, C. Gong et Q. Liu, “Flow Straight and Fast : Learning to Generate and Transfer Data with Rectified Flow,” dans *International Conference on Learning Representations (ICLR)*, 2023.
- [37] E. Chisari *et al.*, “Learning Robotic Manipulation Policies from Point Clouds with Conditional Flow Matching,” 2024, arXiv preprint.
- [38] G. Yan *et al.*, “ManiFlow : A General Robot Manipulation Policy via Consistency Flow Training,” 2025, arXiv preprint.
- [39] Z. Geng *et al.*, “Mean Flows for One-step Generative Modeling,” 2025, arXiv preprint.
- [40] H. Fang *et al.*, “OMP : One-step Meanflow Policy with Directional Alignment,” 2026, arXiv preprint.
- [41] S. Jiang *et al.*, “Streaming Flow Policy : Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories,” 2025, arXiv preprint.
- [42] K. Nguyen *et al.*, “FlowMP : Learning Motion Fields for Robot Planning with Conditional Flow Matching,” 2025, arXiv preprint.
- [43] D. Soleymanzadeh, X. Liang et M. Zheng, “Flow Motion Policy : Manipulator Motion Planning with Flow Matching Models,” 2026, arXiv preprint.
- [44] M. Braun *et al.*, “Riemannian Flow Matching Policy for Robot Motion Learning,” 2024, arXiv preprint.
- [45] A. Brohan *et al.*, “RT-2 : Vision-Language-Action Models Transfer Web Knowledge to Robotic Control,” 2023, arXiv preprint.

- [46] M. J. Kim *et al.*, “OpenVLA : An Open-Source Vision-Language-Action Model,” 2024, arXiv preprint.
- [47] R. Sapkota *et al.*, “Vision-Language-Action (VLA) Models : Concepts, Progress, Applications and Challenges,” 2026, arXiv preprint.
- [48] D. Li *et al.*, “What Foundation Models can Bring for Robot Learning in Manipulation : A Survey,” *The International Journal of Robotics Research*, 2025.
- [49] J. Wen *et al.*, “TinyVLA : Towards Fast, Data-Efficient Vision-Language-Action Models for Robotic Manipulation,” 2025, arXiv preprint.
- [50] S. Liu *et al.*, “Vision-language model-driven scene understanding and robotic object manipulation,” 2024, arXiv preprint.
- [51] O. Mees, L. Hermann et W. Burgard, “What Matters in Language Conditioned Robotic Imitation Learning over Unstructured Data,” 2022, arXiv preprint.
- [52] N. Ingelhart *et al.*, “A Robotic Skill Learning System Built Upon Diffusion Policies and Foundation Models,” 2024, arXiv preprint.
- [53] G. Yin *et al.*, “CodeDiffuser : Attention-Enhanced Diffusion Policy via VLM-Generated Code for Instruction Ambiguity,” 2025, arXiv preprint.
- [54] A. Kirillov *et al.*, “Segment Anything,” dans *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, p. 4015–4026.
- [55] N. Ravi *et al.*, “SAM 2 : Segment Anything in Images and Videos,” *arXiv preprint arXiv :2408.00714*, 2024.
- [56] Meta AI Research, “SAM 3 : Segment Anything with Concepts,” *arXiv preprint*, 2025.
- [57] D. Seita *et al.*, “ToolFlowNet : Robotic Manipulation with Tools via Predicting Tool Flow from Point Clouds,” 2022, arXiv preprint.
- [58] H. Li *et al.*, “Language-Guided Object-Centric Diffusion Policy for Generalizable and Collision-Aware Manipulation,” 2025, arXiv preprint.
- [59] R. Yang *et al.*, “FP3 : A 3D Foundation Policy for Robotic Manipulation,” 2025, arXiv preprint.
- [60] Y. Fu *et al.*, “CordViP : Correspondence-based Visuomotor Policy for Dexterous Manipulation in Real-World,” 2025, arXiv preprint.
- [61] R. Yu *et al.*, “No Need for Real 3D : Fusing 2D Vision with Pseudo 3D Representations for Robotic Manipulation Learning,” 2025, arXiv preprint.
- [62] Z. Ni *et al.*, “VO-DP : Semantic-Geometric Adaptive Diffusion Policy for Vision-Only Robotic Manipulation,” 2025, arXiv preprint.

- [63] J. Jones *et al.*, “Beyond Sight : Finetuning Generalist Robot Policies with Heterogeneous Sensors via Language Grounding,” 2025, arXiv preprint.
- [64] J. Morris *et al.*, “DynoSAM : Open-Source Smoothing and Mapping Framework for Dynamic SLAM,” 2025, arXiv preprint.
- [65] Y. Jin *et al.*, “SE(3)-PoseFlow : Estimating 6D Pose Distributions for Uncertainty-Aware Robotic Manipulation,” 2025, arXiv preprint.
- [66] A. Elfes, “Using Occupancy Grids for Mobile Robot Perception and Navigation,” *Computer*, vol. 22, n<sup>o</sup>. 6, p. 46–57, 1989.
- [67] A. Hornung *et al.*, “OctoMap : An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, n<sup>o</sup>. 3, p. 189–206, 2013.
- [68] B. Curless *et al.*, “A Volumetric Method for Building Complex Models from Range Images,” dans *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH ’96. ACM, 1996, p. 303–312.
- [69] R. A. Newcombe *et al.*, “KinectFusion : Real-time Dense Surface Mapping and Tracking,” dans *Proceedings of the IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011, p. 127–136.
- [70] H. Oleynikova *et al.*, “Voxblox : Incremental 3D Euclidean Signed Distance Fields for On-Board MAV Planning,” dans *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, p. 1366–1373.
- [71] A. Millane *et al.*, “nvblox : GPU-Accelerated Incremental Signed Distance Field Mapping,” dans *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [72] L. Han *et al.*, “FIESTA : Fast Incremental Euclidean Distance Fields for Online Motion Planning of Aerial Robots,” dans *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, p. 4423–4430.
- [73] Y. Pan *et al.*, “Voxfield : Non-Projective Signed Distance Fields for Online Planning and 3D Reconstruction,” dans *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, p. 10 173–10 180.
- [74] W. B. Bedada *et al.*, “Real Time Collision Avoidance with GPU-Computed Distance Maps,” 2024, arXiv preprint.
- [75] J. Ortiz *et al.*, “iSDF : Real-Time Neural Signed Distance Fields for Robot Perception,” dans *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, p. 4654–4663.
- [76] E. Sucar *et al.*, “iMAP : Implicit Mapping and Positioning in Real-Time,” dans *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2021, p. 6229–6238.

- [77] Z. Zhu *et al.*, “NICE-SLAM : Neural Implicit Scalable Encoding for SLAM,” dans *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, p. 12 776–12 786.
- [78] B. Sundaralingam *et al.*, “CuRobo : Parallelized Collision-Free Robot Motion Generation,” dans *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, p. 8112–8119.
- [79] J. Lee *et al.*, “Learning Fast, Tool-Aware Collision Avoidance for Collaborative Robots,” *IEEE Robotics and Automation Letters*, 2025.
- [80] Y. Li *et al.*, “Representing robot geometry as distance fields : Applications to whole-body manipulation,” dans *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024, p. 15 351–15 357.
- [81] B. Liu *et al.*, “Collision-free Motion Generation Based on Stochastic Optimization and Composite Signed Distance Field Networks of Articulated Robot,” *IEEE Robotics and Automation Letters*, vol. 8, n<sup>o</sup>. 11, p. 7082–7089, 2023.
- [82] X. Zhu *et al.*, “Efficient collision detection framework for enhancing collision-free robot motion,” dans *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, p. 16 162–16 168.
- [83] A. Govoni *et al.*, “Real-time collision avoidance with robot distance fields in a task-priority framework,” *Robotics and Autonomous Systems*, vol. 200, p. 105394, 2026.
- [84] K. Chen *et al.*, “Samp : Spatial anchor-based motion policy for collision-aware robotic manipulators,” *arXiv preprint arXiv :2509.11185*, 2025.
- [85] Y. Li *et al.*, “Configuration space distance fields for manipulation planning,” *arXiv preprint arXiv :2406.01137*, 2024.
- [86] O. Khatib, “Real-Time Obstacle Avoidance for Manipulators and Mobile Robots,” *The International Journal of Robotics Research (IJRR)*, vol. 5, n<sup>o</sup>. 1, p. 90–98, 1986.
- [87] H. Ding *et al.*, “Towards Safe Imitation Learning via Potential Field-Guided Flow Matching,” dans *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, p. 11 693–11 700.
- [88] A. D. Ames *et al.*, “Control Barrier Functions : Theory and Applications,” 2019, arXiv preprint.
- [89] P. Wieland et F. Allgöwer, “CONSTRUCTIVE SAFETY USING CONTROL BARRIER FUNCTIONS,” 2007, arXiv preprint.
- [90] A. D. Ames *et al.*, “Control Barrier Function Based Quadratic Programs for Safety Critical Systems,” 2017, arXiv preprint.



- [91] W. S. Cortez *et al.*, “Control Barrier Functions for Mechanical Systems : Theory and Application to Robotic Grasping,” 2019, arXiv preprint.
- [92] B. Stellato *et al.*, “OSQP : An Operator Splitting Solver for Quadratic Programs,” *Mathematical Programming Computation*, vol. 12, n<sup>o</sup>. 4, p. 637–672, 2020.
- [93] L. M. Bregman, “The Method of Successive Projection for Finding a Common Point of Convex Sets,” *Soviet Mathematics Doklady*, vol. 6, p. 688–692, 1965.
- [94] H. H. Bauschke et J. M. Borwein, “On Projection Algorithms for Solving Convex Feasibility Problems,” *SIAM Review*, vol. 38, n<sup>o</sup>. 3, p. 367–426, 1996.
- [95] R. L. Dykstra, “An Algorithm for Restricted Least Squares Regression,” *Journal of the American Statistical Association*, vol. 78, n<sup>o</sup>. 384, p. 837–842, 1983.
- [96] J. P. Boyle et R. L. Dykstra, “A Method for Finding Projections onto the Intersection of Convex Sets in Hilbert Spaces,” dans *Advances in Order Restricted Statistical Inference*, ser. Lecture Notes in Statistics. Springer, 1986, vol. 37, p. 28–47.
- [97] C. Hildreth, “A Quadratic Programming Procedure,” *Naval Research Logistics Quarterly*, vol. 4, n<sup>o</sup>. 1, p. 79–85, 1957.
- [98] K.-C. Hsu, H. Hu et J. F. Fisac, “The Safety Filter : A Unified View of Safety-Critical Control in Autonomous Systems,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 7, p. 47–72, 2024.
- [99] K. P. Wabersich et M. N. Zeilinger, “A Predictive Safety Filter for Learning-Based Control of Constrained Nonlinear Dynamical Systems,” *Automatica*, vol. 129, p. 109597, 2021.
- [100] K. P. Wabersich *et al.*, “Data-Driven Safety Filters : Hamilton-Jacobi Reachability, Control Barrier Functions, and Predictive Methods for Uncertain Systems,” *IEEE Control Systems Magazine*, vol. 43, n<sup>o</sup>. 5, p. 137–177, 2023.
- [101] R. Römer *et al.*, “From Demonstrations to Safe Deployment : Path-Consistent Safety Filtering for Diffusion Policies,” 2026, arXiv preprint.
- [102] A. Agrawal et K. Sreenath, “Discrete Control Barrier Functions for Safety-Critical Control of Discrete Systems with Application to Bipedal Robot Navigation,” dans *Robotics : Science and Systems (RSS)*, 2017.
- [103] J. Breeden, K. Garg et D. Panagou, “Control Barrier Functions in Sampled-Data Systems,” *IEEE Control Systems Letters*, vol. 6, p. 367–372, 2022.
- [104] L. E. Beaver, “Optimal Control Barrier Functions : Maximizing the Action Space Subject to Control Bounds,” 2024, arXiv preprint.

- [105] X. Dai *et al.*, “SafeFlow : Safe Robot Motion Planning with Flow Matching via Control Barrier Functions,” 2025, arXiv preprint.
- [106] J. Long *et al.*, “Constraining Streaming Flow Models for Adapting Learned Robot Trajectory Distributions,” 2026, arXiv preprint.
- [107] Y. Song *et al.*, “OmniGuide : Universal Guidance Fields for Enhancing Generalist Robot Policies,” 2026, arXiv preprint.
- [108] K. Mizuta et K. Leung, “CoBL-Diffusion : Diffusion-Based Conditional Robot Planning in Dynamic Environments Using Control Barrier and Lyapunov Functions,” 2024, arXiv preprint.
- [109] Z. Yang *et al.*, “UniConFlow : A Unified Constrained Flow-Matching Framework for Certified Motion Planning,” 2026, arXiv preprint.
- [110] T.-Y. Huang *et al.*, “SAD-Flower : Flow Matching for Safe, Admissible, and Dynamically Consistent Planning,” 2026, arXiv preprint.
- [111] K. Mizuta et K. Leung, “Unified Generation-Refinement Planning : Bridging Guided Flow Matching and Sampling-Based MPC for Social Navigation,” 2026, arXiv preprint.
- [112] J. Yang, S. Jang et S. Han, “SafeFlowMatcher : Safe and Fast Planning using Flow Matching with Control Barrier Functions,” 2026, arXiv preprint.
- [113] E. Welte *et al.*, “FlowCorrect : Efficient Interactive Correction of Generative Flow Policies for Robotic Manipulation,” 2026, arXiv preprint.
- [114] A. Liao *et al.*, “Delay-Aware Diffusion Policy : Bridging the Observation-Execution Gap in Dynamic Tasks,” 2025, arXiv preprint.
- [115] X. Ye *et al.*, “RA-DP : Rapid Adaptive Diffusion Policy for Training-Free High-frequency Robotics Replanning,” 2025, arXiv preprint.
- [116] J. Carpentier *et al.*, “The Pinocchio C++ Library : A Fast and Flexible Implementation of Rigid Body Dynamics Algorithms and Their Analytical Derivatives,” dans *IEEE/SICE International Symposium on System Integration (SII)*, 2019, p. 614–619.
- [117] F. Xiang *et al.*, “SAPIEN : A simulated part-based interactive environment,” dans *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.
- [118] Y. Zhou *et al.*, “On the Continuity of Rotation Representations in Neural Networks,” 2020, arXiv preprint.
- [119] A. Vaswani *et al.*, “Attention Is All You Need,” dans *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

- [120] Y. Nakamura et H. Hanafusa, “Inverse Kinematic Solutions With Singularity Robustness for Robot Manipulator Control,” *Journal of Dynamic Systems, Measurement, and Control*, vol. 108, n<sup>o</sup>. 3, p. 163–171, 1986.
- [121] C. W. Wampler, “Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, n<sup>o</sup>. 1, p. 93–101, 1986.
- [122] A. Li’geois, “Automatic Supervisory Control of the Configuration and Behavior of Multibody Mechanisms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 7, n<sup>o</sup>. 12, p. 868–871, 1977.
- [123] U. Rosolia, A. Singletary et A. D. Ames, “Unified Multirate Control : From Low-Level Actuation to High-Level Planning,” *IEEE Transactions on Automatic Control*, vol. 67, n<sup>o</sup>. 12, p. 6627–6640, 2022.
- [124] International Organization for Standardization, “Robots and Robotic Devices — Collaborative Robots,” ISO, Rapport technique ISO/TS 15066 :2016, 2016.

## ANNEXE A    PROTOCOLE DÉTAILLÉ D’ACQUISITION DES DÉMONSTRATIONS

Cette annexe détaille la mise en œuvre du protocole de collecte de la Section 4.4.1 : la procédure opératoire, l’architecture logicielle d’acquisition et le format de stockage des enregistrements. Ce format, défini en début de projet, est autoporteur : les données sont lisibles sans dépendance à ROS (JSON, PNG, NumPy), ce qui permet au pipeline d’entraînement de les consommer directement, là où un conteneur standard de type `rosvbag` aurait exigé une étape d’extraction préalable.

### Procédure opératoire

Chaque démonstration réelle suit la séquence suivante :

1. montage de la caméra RGB-D sur le support officiel Franka à l’effecteur ;
2. positionnement du robot dans la configuration articulaire initiale  $\mathbf{q}_0$  donnée à la Section 4.4.1 ;
3. passage du robot en mode de guidage manuel, permettant à l’opérateur de le déplacer librement ;
4. exécution par guidage kinesthésique de la trajectoire à apprendre, depuis la posture initiale jusqu’à la cible, selon le schéma d’approche de la Section 4.4.1 ;
5. arrêt de l’enregistrement et écriture des données de la trajectoire dans un fichier JSON.

### Architecture d’acquisition

L’enregistrement repose sur deux nœuds ROS distincts. Le premier, écrit en C++ pour minimiser la latence de lecture, lit l’état du robot et publie :

- la pose du TCP dans le repère de base ( $1 \times 7$ ) : position cartésienne (trois premières composantes) et orientation en quaternion (quatre dernières) ;
- la vitesse du TCP dans le repère de base ( $1 \times 6$ ) : vitesses linéaires (trois premières composantes) puis angulaires (trois dernières).

Le second nœud, écrit en Python, s’abonne à ces messages ainsi qu’au flux RGB-D de la caméra à l’effecteur, et écrit l’ensemble sur disque à la cadence de 10 Hz. La Figure A.1 illustre cette architecture.

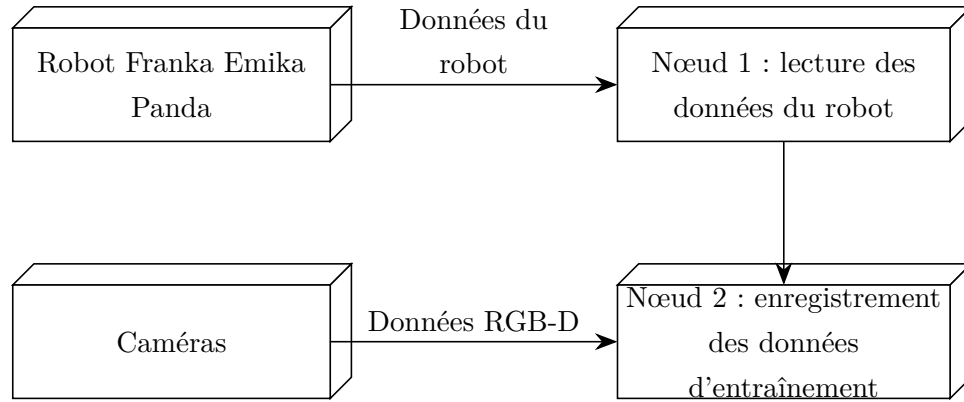


Figure A.1 Architecture du système d'acquisition des données d'entraînement.

### Format d'enregistrement

À chaque pas d'échantillonnage, le second nœud consigne dans le fichier JSON, en plus de la pose et de la vitesse du TCP, les champs structurels suivants :

- `step_number` : l'indice de l'échantillon dans la trajectoire ;
- `time_step` : l'instant de la prise de données depuis le début de la trajectoire ;
- `rgb` : le nom du fichier de l'image couleur, nommé par convention `ee_rgb_step_{step_number}.png` ;
- `depth_npy` : le nom du fichier de la carte de profondeur, nommé par convention `ee_depth_step_{step_number}.npy`.

Les images et cartes de profondeur sont écrites dans un sous-dossier dédié ; le fichier JSON n'en contient que les noms.

### Organisation des fichiers

Chaque trajectoire enregistrée occupe un dossier `Trajectory_X`, où `X` est le numéro de la trajectoire, regroupant le fichier JSON des données numériques et le sous-dossier des images et cartes de profondeur (Figure A.2).

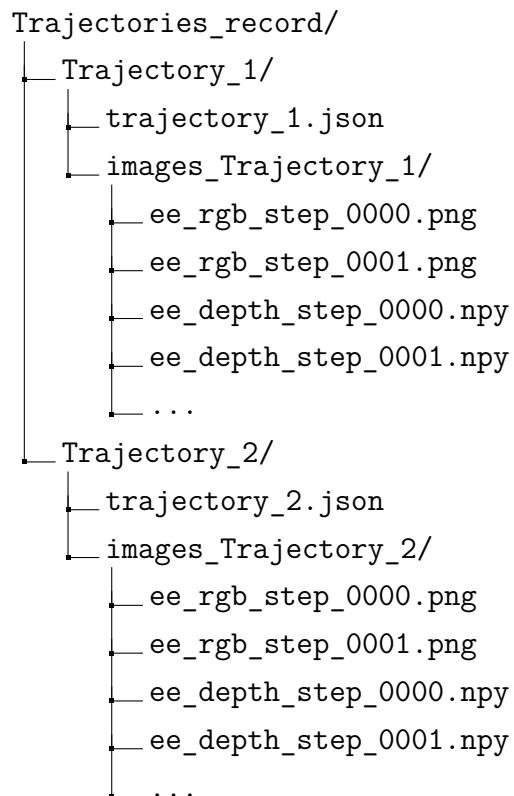


Figure A.2 Structure des dossiers du jeu de données brut.

Le prétraitement décrit à la Section 4.4.2 produit un second dossier racine, miroir du premier : les nuages de points conditionnels fusionnés y sont enregistrés sous le nom `Merged_{step_number}.npy` dans un sous-dossier `Merged_Trajectory_X`, et le fichier JSON de la trajectoire y est copié, augmenté de la pose de l'outil dans le repère de base (Figure A.3).

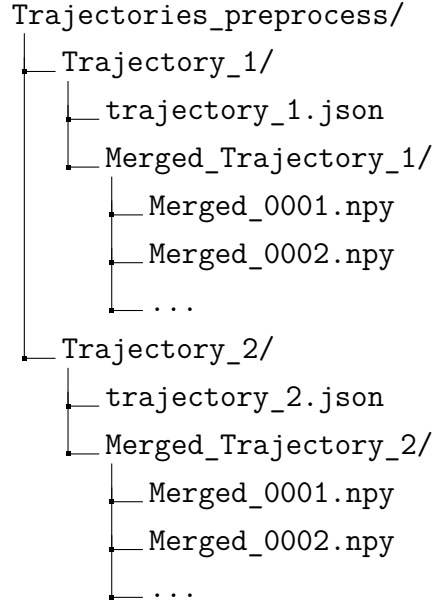


Figure A.3 Structure des dossiers du jeu de données prétraité.

## Collecte en simulation

Les collectes simulées reprennent l'infrastructure d'enregistrement ci-dessus à l'identique (cadence de 10 Hz en temps simulé, nomenclature et organisation des fichiers identiques), en l'enrichissant : les environnements *ManiSkill 3* sont vectorisés (plusieurs environnements simulés en parallèle sur GPU, commande en mode `pd_ee_delta_pose`), 25 pas de stabilisation physique précèdent l'enregistrement, et le fichier JSON contient en plus les positions et vitesses articulaires, le flux d'une seconde caméra statique, les matrices extrinsèques des deux caméras (converties de la convention SAPIEN vers la convention OpenCV) ainsi que des métadonnées de scène (tâche, type de cible, type de support).

Pour la tâche d'alimentation, le robot est le Panda muni de la fourchette, obtenu par composition d'URDF, la caméra de poignet ( $640 \times 480$ , champ de vue de  $42^\circ$ ) répliquant le montage réel. La cible est un maillage rigide tiré parmi sept aliments (pomme de terre, poitrine de poulet, brocoli, sushi, rouleau de sushi, cube de saumon, saucisse), placé aléatoirement dans un rectangle de  $12 \times 16$  cm du plan de travail avec un lacet uniforme sur  $[-\pi, \pi]$  ; une fois sur deux, il repose sur un piédestal de hauteur uniforme entre 4 et 10 cm. La politique experte procède en trois phases : approche avec alignement du lacet sur l'axe principal de la cible (abordée du côté opposé au robot, tangage de  $-10^\circ$ ), survol à 3 cm puis piquage vertical d'une profondeur  $\min(2h_z - 2 \text{ mm}, 15 \text{ mm})$ , où  $h_z$  est la demi-hauteur de l'aliment, bornée par la surface d'appui ; redressement du poignet le long d'un arc de rayon 3,5 cm jusqu'à un

tangage de  $80^\circ$  ; transport vers une pose de livraison fixe. Dès le piquage, l'aliment est cinématiquement solidarisé à la fourchette (pose relative verrouillée réappliquée à chaque pas), idéalisant un piquage ferme. Trois environnements parallèles sur cinq épisodes produisent les 15 trajectoires (Figure 4.4).

Pour la tâche de saisie et dépôt, le robot est le Panda standard et la saisie est physique : la politique ne commande que la position et la pince, l'orientation verticale de la pince étant fixée par la configuration initiale et maintenue par le contrôleur. Le cube (dynamique, 4 cm, lacet aléatoire) apparaît dans un rectangle de  $10 \times 10$  cm sous l'empreinte initiale de la caméra de poignet, ce qui garantit sa visibilité dans la première image ; la boîte de dépôt ( $16 \times 16 \times 1$  cm) apparaît dans une zone aléatoire à la droite du robot couvrant les positions utilisées au déploiement. La machine à états à huit phases (survol, descente, saisie, dégagement, transport, dépôt, relâchement, retrait) suit ses segments de chemin par jalon : le paramètre d'avancement ne progresse que lorsque le TCP a rejoint le point courant du chemin, de sorte que les portions raides de l'arc de transport sont effectivement parcourues plutôt que coupées. Comme pour la tâche d'alimentation, le cube et la boîte sont segmentés par SAM 3 dans la première image et leurs nuages de points sont reconstruits par la chaîne de rétroprojection de la Section 4.4.2. Quatre environnements parallèles sur dix épisodes produisent les 40 trajectoires (Figure 4.5).



## ANNEXE B DÉTAILS DE L'ARCHITECTURE DU RÉSEAU DE VITESSE

Cette annexe rassemble les détails architecturaux du modèle génératif de la Section 4.5 : les encodeurs du conditionnement, le plongement du temps d'écoulement, puis le réseau de vitesse **ConditionalUnet1D** et son bloc résiduel conditionnel. Ces composants sont repris des architectures publiées de *Diffusion Policy* [25] et de *3D Diffusion Policy* [29] ; leur conception n'est pas une contribution de ce mémoire, et ils sont documentés ici pour rendre le document autonome et reproductible.

### Encodeurs du conditionnement

Le nuage de points conditionnel ( $1024 \times 3$ ) traverse l'encodeur léger de *3D Diffusion Policy* [29] : un perceptron multicouche partagé entre les points ( $3 \rightarrow 64 \rightarrow 128 \rightarrow 256$ , normalisation de couche et ReLU), suivi d'un max-pooling sur l'ensemble des points, qui rend le descripteur invariant à leur ordre, puis d'une projection linéaire vers  $\mathbb{R}^{64}$ . La proprioception des deux derniers pas traverse quant à elle un perceptron ( $20 \rightarrow 128 \rightarrow 64$ , activation Mish). Les deux descripteurs concaténés forment le vecteur de conditionnement  $c \in \mathbb{R}^{128}$  de la Section 4.5.

### Plongement du temps d'écoulement

Le temps d'écoulement  $t$  est fourni au réseau par le plongement sinusoidal introduit avec l'architecture Transformer [119] :

$$\gamma(t) = [\sin(\omega_1 t'), \dots, \sin(\omega_{d/2} t'), \cos(\omega_1 t'), \dots, \cos(\omega_{d/2} t')], \quad \omega_i = 10000^{-\frac{i-1}{d/2-1}} \quad (\text{B.1})$$

où  $d = 256$  est la dimension du plongement et  $t' = 1000 t$  une mise à l'échelle qui ramène l'intervalle  $[0, 1]$  à la plage d'indices pour laquelle ces fréquences ont été conçues. La progression géométrique des pulsations  $\omega_i$  résout ainsi  $t$  à toutes les échelles, des variations grossières aux plus fines, tout en demeurant une fonction lisse. Ce plongement est raffiné par un perceptron ( $256 \rightarrow 1024 \rightarrow 256$ , activation Mish), puis concaténé au vecteur  $c$  pour constituer le conditionnement global  $c_g \in \mathbb{R}^{384}$  ( $128 + 256$ ).

## Réseau de vitesse et bloc résiduel conditionnel

La Figure B.1 présente la vue d'ensemble du réseau de vitesse : chaque triplet y indique la forme du tenseur qui transite entre les blocs, soit la taille du lot  $B$ , le nombre de canaux et la longueur temporelle, et le segment  $X_t$  y est noté  $x_t$ .

Chaque bloc résiduel conditionnel enchaîne deux convolutions 1D (noyau 3), chacune suivie d'une normalisation de groupe et de l'activation Mish

$$\text{Mish}(x) = x \cdot \tanh(\ln(1 + e^x)) \quad (\text{B.2})$$

le conditionnement global  $c_g$  y étant appliqué par modulation affine de type FiLM : un gain  $\gamma \in \mathbb{R}^{C_{\text{out}}}$  et un biais  $\beta \in \mathbb{R}^{C_{\text{out}}}$ , prédits par un perceptron à partir de  $c_g$ , modulent canal par canal les cartes de caractéristiques entre les deux convolutions. La Figure B.2 détaille ce bloc.  $x$  y désigne la carte de caractéristiques entrant dans le bloc, de forme  $(B, C_{\text{in}}, H)$ , où  $C_{\text{in}}$  et  $C_{\text{out}}$  sont les nombres de canaux à l'entrée et à la sortie du bloc et  $H$  la longueur temporelle au niveau de résolution courant, divisée par deux à chaque descente ; la connexion résiduelle traverse une convolution  $1 \times 1$  lorsque  $C_{\text{in}} \neq C_{\text{out}}$  afin d'accorder les dimensions.

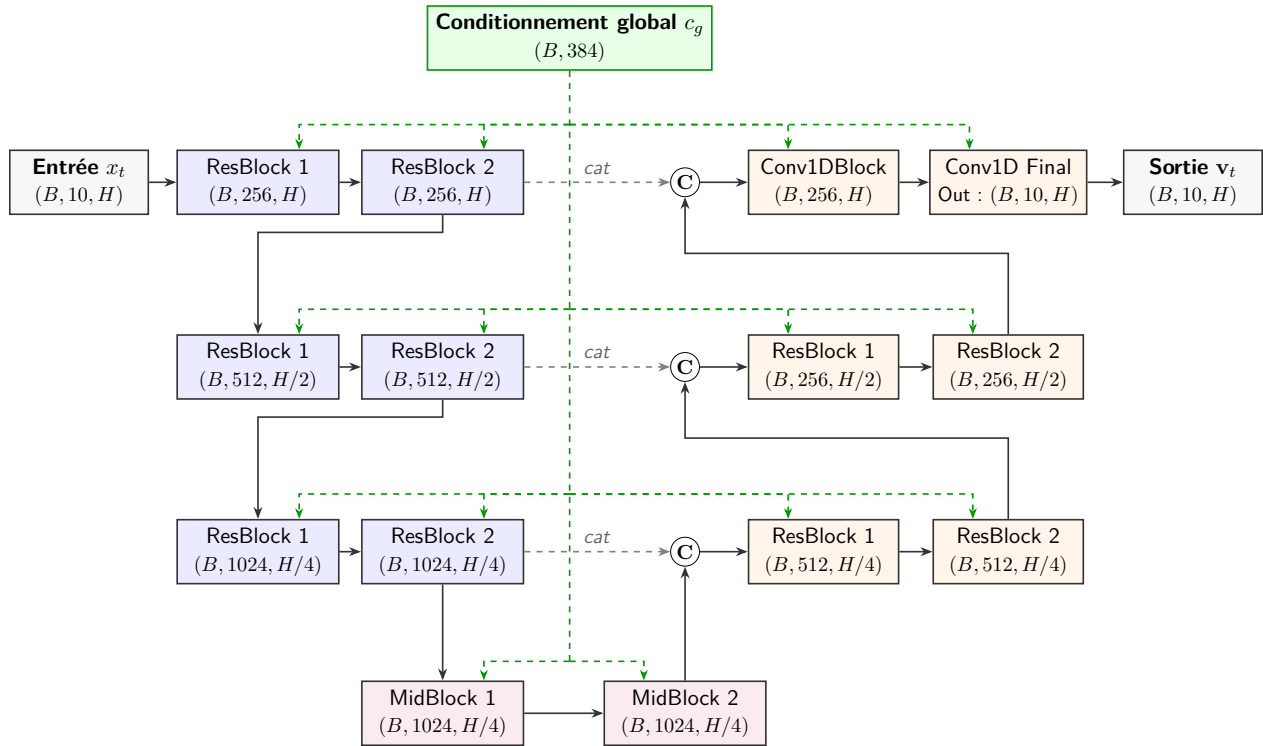


Figure B.1 Architecture du réseau de vitesse **ConditionalUnet1D** : encodeur-décodeur 1D à trois niveaux de résolution, connexions de saut et injection du conditionnement global dans chaque bloc résiduel. Chaque triplet indique la forme du tenseur transitant entre les blocs : taille de lot  $B$ , nombre de canaux, longueur temporelle ( $H = 16$ ).

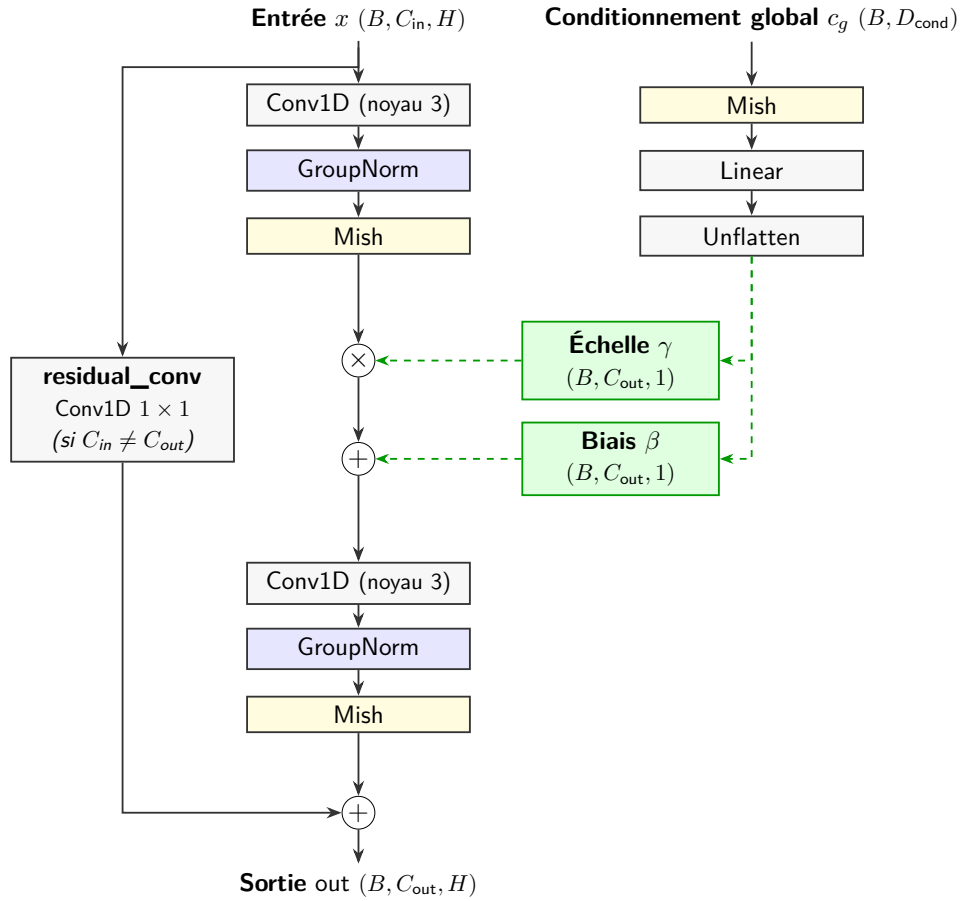


Figure B.2 Architecture du bloc résiduel conditionnel (**ConditionalResidualBlock1D**) du réseau de vitesse, avec sa modulation affine par le conditionnement global.  $B$  désigne la taille du lot,  $C_{in}$  et  $C_{out}$  les nombres de canaux à l'entrée et à la sortie du bloc,  $H$  la longueur temporelle du niveau de résolution courant et  $D_{cond} = 384$  la dimension du conditionnement global  $c_g$ .

## ANNEXE C    ENTRAÎNEMENT DU CHAMP DE DISTANCE SIGNÉE DU ROBOT

Cette annexe rassemble le détail de l'apprentissage du modèle de distance de la Section 4.7.1. La génération des étiquettes et la régression récursive reproduisent le protocole *Robot Distance Fields* de Li et al. [80] et son implémentation publique ; elles ne constituent pas une contribution de ce mémoire et sont reproduites ici pour rendre le document autonome. Seuls les hyperparamètres du raffinement géométrique, introduit à la Section 4.7.1, sont propres à ce travail.

### Génération des étiquettes de distance

Pour chaque lien protégé, le maillage est d'abord normalisé selon l'équation (4.11), puis un jeu d'étiquettes de distance signée exacte est construit suivant le protocole de [80], lui-même hérité de DeepSDF :  $8 \times 10^5$  points concentrés près de la surface, obtenus par balayages virtuels du maillage, le signe étant déterminé par les normales, et autant de points tirés uniformément dans le cube  $[-1, 1]^3$ , chacun étiqueté de sa distance signée exacte au maillage. Les étiquettes négatives situées hors de la boîte englobante du lien, artefacts du calcul de signe par normales, sont écartées.

### Régression récursive des poids

La sortie du modèle étant linéaire en ses paramètres  $\mathbf{w}_l$ , l'ajustement est un problème de moindres carrés, résolu de manière récursive pour éviter l'inversion d'une matrice de  $n^3$  lignes assemblée sur l'ensemble du jeu de données. À chaque itération, un lot de points encodés  $\Phi_k$ , d'étiquettes  $\mathbf{d}_k$ , met à jour les poids en forme close :

$$\mathbf{K}_k = \mathbf{B}_{k-1} \Phi_k^\top \left( \mathbf{I} + \Phi_k \mathbf{B}_{k-1} \Phi_k^\top \right)^{-1} \quad (\text{C.1})$$

$$\mathbf{B}_k = \mathbf{B}_{k-1} - \mathbf{K}_k \Phi_k \mathbf{B}_{k-1} \quad (\text{C.2})$$

$$\mathbf{w}_k = \mathbf{w}_{k-1} + \mathbf{K}_k (\mathbf{d}_k - \Phi_k \mathbf{w}_{k-1}) \quad (\text{C.3})$$

avec l'initialisation  $\mathbf{B}_0 = 10^2 \mathbf{I}$ , équivalente à une régularisation de type *ridge* de la solution. Ces mises à jour correspondent à l'algorithme d'apprentissage récursif de [80] ; 80 itérations sont conduites par lien.

## Hyperparamètres du raffinement géométrique

Le raffinement géométrique de la Section 4.7.1 part des poids issus de la régression récursive et les affine par descente de gradient sur la fonction de coût (4.15). Le jeu de supervision compte 20 000 rayons normaux vérifiés par lien : les décalages  $\delta$  sont concentrés à 75 % dans la coquille critique  $[1, 10]$  mm, cohérente avec la marge  $d_{\text{safe}} = 15$  mm, et étendus jusqu'à 50 mm ; un candidat n'est retenu que si sa distance exacte recalculée coïncide avec le décalage à 0,25 mm près et que la direction exacte du gradient s'écarte de moins de  $5^\circ$  de la normale de départ. Le tableau C.1 rassemble les valeurs numériques.

Tableau C.1 Hyperparamètres d'entraînement du modèle de distance du robot.

| Symbole                                         | Description                                      | Valeur                            |
|-------------------------------------------------|--------------------------------------------------|-----------------------------------|
| <i>Régression de valeur (protocole de [80])</i> |                                                  |                                   |
| —                                               | points près de la surface / uniformes            | $8 \times 10^5$ / $8 \times 10^5$ |
| $\mathbf{B}_0$                                  | initialisation des moindres carrés récursifs     | $10^2 \mathbf{I}$                 |
| —                                               | itérations récursives par lien                   | 80                                |
| <i>Raffinement géométrique (contribution)</i>   |                                                  |                                   |
| —                                               | rayons normaux vérifiés par lien                 | 20 000                            |
| $\delta$                                        | décalages : coquille critique / maximum          | 75 % dans $[1, 10]$ mm / 50 mm    |
| —                                               | tolérances de vérification (distance, direction) | 0,25 mm, $5^\circ$                |
| $\lambda_{\text{ray}}, \lambda_{\text{eik}}$    | poids des rayons et du terme eikonal             | 2, 0,05                           |
| $\lambda_{\text{dir}}, \lambda_{\text{vec}}$    | poids d'alignement du gradient                   | 0,2, 0,25                         |
| $\omega$                                        | pondération de proximité                         | $1 + 4e^{-\bar{d}/\bar{\delta}}$  |
| —                                               | optimiseur, ordonnancement                       | Adam, recuit en cosinus           |
| —                                               | époques de raffinement                           | 1500                              |

## ANNEXE D COMPLÉMENTS DU FILTRE DE SÉCURITÉ SDF-CBF

Cette annexe rassemble les compléments d'implémentation du filtre de sécurité de la Section 4.7 : les formes closes du lissage et de la réparation qui transforment la vitesse projetée en commande publiée, ainsi que les garde-fous qui bornent cette réparation. La dérivation du gradient analytique de la contrainte et la forme close de la projection sur un demi-espace figurent au chapitre principal (Sections 4.7.3 et 4.7.5) ; les notations sont celles du chapitre.

### Formes closes du post-traitement de la commande

**Saturation et lissage.** La vitesse projetée est saturée aux limites articulaires,  $\dot{\mathbf{q}}_{\text{proj,sat}} = \text{sat}(\dot{\mathbf{q}}_{\text{proj}})$ , puis filtrée par un passe-bas exponentiel de constante de temps  $\tau_{\text{safe}}$  :

$$\dot{\mathbf{q}}_{f,k} = (1 - \gamma_{\text{safe}}) \dot{\mathbf{q}}_{f,k-1} + \gamma_{\text{safe}} \dot{\mathbf{q}}_{\text{proj,sat},k}, \quad \gamma_{\text{safe}} = \frac{\Delta t}{\Delta t + \tau_{\text{safe}}}.$$

L'ordre saturation puis filtrage évite que le filtre considère une vitesse physiquement irréalisable.

**Réparation finale.** La contrainte est réévaluée sur la vitesse filtrée, avec le même second membre saturé et resserré que la projection principale,  $c_f = \nabla h^\top \dot{\mathbf{q}}_f - b(h) - \eta$ . Si  $c_f < 0$  dans la bande d'activation, une réparation est appliquée :

$$\Delta \dot{\mathbf{q}}_{\text{rep}} = \frac{\max(0, -c_f)}{\nabla h^\top \mathbf{W}^{-1} \nabla h + \varepsilon} \mathbf{W}^{-1} \nabla h.$$

Cette réparation est une projection corrective sur la même contrainte CBF linéarisée : elle réutilise la valeur de barrière  $h$ , le gradient  $\nabla h$ , la borne  $b(h) + \eta$  et la métrique  $\mathbf{W}$  de la projection principale, de sorte qu'elle répartit sa correction de la même manière que le solveur au lieu de défaire, par un pas euclidien, la préférence pour le noyau de tâche.

Deux garde-fous bornent cette réparation. D'une part, sa norme est plafonnée : à  $\Delta v_{\text{max}}$  tant que la barrière est positive, et à un plafond distinct  $\Delta v_{\text{max}}^{\text{rec}}$ , plus permissif, lorsque  $h < 0$  et qu'une sortie rapide de la zone de violation devient prioritaire. Ce plafond couvre notamment le scénario où la carte persistante fait apparaître de nouveaux points obstacles déjà très proches du robot, voire déjà dans la zone de violation : la violation instantanée  $-c_f$  peut alors être grande, et une réparation non bornée se traduirait par une commande de vitesse violente en un seul cycle. D'autre part, la réparation est modulée par un seuil de fiabilité du

gradient : son amplitude croît linéairement de zéro, lorsque  $\|\nabla h\| = g_{\min}$ , jusqu'à sa pleine valeur à  $3g_{\min}$ . La direction  $\nabla h/\|\nabla h\|$  n'est en effet fiable que si la norme du gradient est significative : face à un obstacle mince, les gradients des points situés de part et d'autre du lien se compensent dans l'agrégation soft-min et  $\|\nabla h\|$  s'effondre ; une réparation de pleine amplitude pousserait alors le bras dans une direction dépourvue de sens géométrique. Le filtre s'en remet dans ce cas à la vitesse issue de la projection pour le cycle courant, la violation résiduelle étant résorbée aux cycles suivants, dès que la géométrie redevient non dégénérée. La même limite  $\Delta v_{\max}$  s'applique enfin à chaque pas de projection de la boucle de balayage elle-même : lorsque cette borne est atteinte, la solution retournée n'est plus la projection exacte de (4.10), mais une solution dont la correction par cycle est bornée, le résidu étant repris par la réévaluation finale puis par les cycles suivants. Face à plusieurs contraintes simultanément actives, la réparation ne porte que sur la contrainte critique réévaluée (le lien de plus faible barrière, avec son second membre courant), les autres demi-espaces ayant été traités par les balayages ; le résidu de contrainte définitif est mesuré sur la commande effectivement publiée, après la saturation finale, et exporté à chaque cycle comme diagnostic (`constr_final`), de sorte que l'effet conjoint de la troncature, des plafonds et de la saturation est observé plutôt que supposé nul.



## ANNEXE E    RÉSULTATS COMPLÉMENTAIRES

Cette annexe rassemble les résultats de support qui appuient les analyses du Chapitre 5 sans en constituer l'argument principal : elle est le réceptacle des figures et tableaux détaillés que le corps du chapitre ne cite que ponctuellement, afin d'en préserver la lisibilité. Le chapitre porte la démonstration ; l'annexe porte la preuve détaillée.

À compléter.